

January 2026

Wednesday, January 28, 2026 4:15 PM

Date	Experiment	Description	Page #
Jan 5 – Jan 12, 2026	Shapemapper2 on Dada2 output	• ShapeMapper2 successfully ran on variant-specific FASTA files after primer adjustments. • Variant-level ShapeMapper2 outputs reproduced expected mutation rate trends (U1 and V1). • Environment manager issues were resolved, restoring pipeline usability.	001
(Jan 12 – Jan 19, 2026)	Denoising U1 data with dada2	Re-trimming and reprocessing of U1 sequencing data validated a robust ShapeMapper2 → RNP-MaP workflow with consistent Sm-site binding signals, while larger-scale dataset processing, variant interpretation, and cross-dataset comparisons remain the primary next steps.	002
Jan 19 – Jan 26, 2026	U1 snRNA Replicate Processing and Denoising Validation Using DADA2, ShapeMapper2, and RNP-MaP	Processing of the second U1 replicate confirmed that revised trimming and DADA2 denoising improved replicate concordance and preserved biological signal, while remaining variance sources and broader snRNA clustering analyses are the next priorities.	003
Jan 26 – Feb 2, 2026	snRNA Sequencing Preparation, PCA Analysis, and Variant Parsing	Sequencing preparation, PCA generation across snRNA samples, and variant parsing were completed successfully this week, while downstream DADA2 interpretation and fastp → ShapeMapper2 → RNP-MaP processing were delayed by sequencing workload, leaving denoising completion, pipeline refinement, and cross-sample PCA interpretation as the primary next steps.	004

February 2026

Wednesday, January 28, 2026

4:58 PM

Date	Experiment	Description	Page #
02/03	Learning how to use Snakemake	Notes from Snakemake tutorial using features for resource control, config-driven sample handling, wildcard-dependent inputs, structured logging, and intermediate file management to make the workflow more scalable and robust.	006
02/04	Snakemake tutorial code	Code from snakemake tutorials (basic and advanced)	007
02/05	Designing Snakemake Workflow	Determine how to design a pipeline to remove perform the various adapter trimming, ASV detection, and UMI extraction and detection necessary to run RNP-MaP and Shapemapper2	008
02/09	Workflow Update and Pipeline version 1		009
02/12	Workflow Update and Pipeline version 2		010
02/(14-21)	Workflow Update and Pipeline version update		011
02/23	Implementing pipeline on U1 snRNA Samples	Running U1 snRNA through snakemake pipeline to obtain U1 Amplicon Sequencing Variant Sequences	012
02/24	Implementing pipeline on all snRNA Samples	Running U11, U12, U4, U4atac, and U5 (A/B) snRNA through snakemake pipeline to obtain respective Amplicon Sequencing Variants (ASVs) Sequence	013
02/25-03/09	Running Shapemapper 2 on all snRNA Samples and Analyzing U1 snRNA via Correlation and Covariation		014
03/17	PCA Analysis on all snRNA Data from snakemake pipeline		015

March 2026

Thursday, April 2, 2026 4:30 PM

Date	Experiment	Description	Page #
03/17	PCA Analysis on all snRNA Data from snakemake pipeline		015
03/26	RNP-MaP Step 1 -2		016
03/25	Hierarchical Clustering and Dendrogram Plot		017
03/30	Recreation of the snRNA Nucleotide Frequency Ratio		018

April 2026

Thursday, April 2, 2026 4:30 PM

Date	Experiment	Description	Page #
04/2026	Dimensionality Analysis of snRNA data		019

May 2026

Friday, May 8, 2026

3:48 PM

Date	Experiment	Description	Page #
05/26	Multivariate Analysis of snRNA Data		020
05/26	UMAP analysis on SM and Non-SM Sites		021
05/26	Using DTW to observe effects of sequence differences		022

June 2026

Friday, May 8, 2026

3:48 PM

Date	Experiment	Description	Page #
06/01	Shapemapper2 and RNP-MaP on 04022026 sequencing run and redoing U5 samples	RNP-MaP reactivity profiling of spliceosomal snRNAs, ShapeMapper2 mutation rate, RNP-MaPper2 Sm site calling (U11 RANSAC failure noted), and RStudio Sm profile visualization (U1 variability observed).	023
06/02			

001 (Jan 5 – Jan 12, 2026)

Monday, January 5, 2026 1:48 PM

Goals for the week:

- Update lab notebook to current date
- Run Shapemapper2 with fasta files from dada2 output to determine if binned/parsed profiles will be generated

Tuesday

Objective:

- Organize myself
- Run fasta files through shapemapper2
- Look into the steps that need to be taken to organize my notebook

What I Did:

- Unfortunately, the Cluster is down until tomorrow, but I created the sbatch files needed.

Results / Observations:

Attachments:

Thursday

Objective:

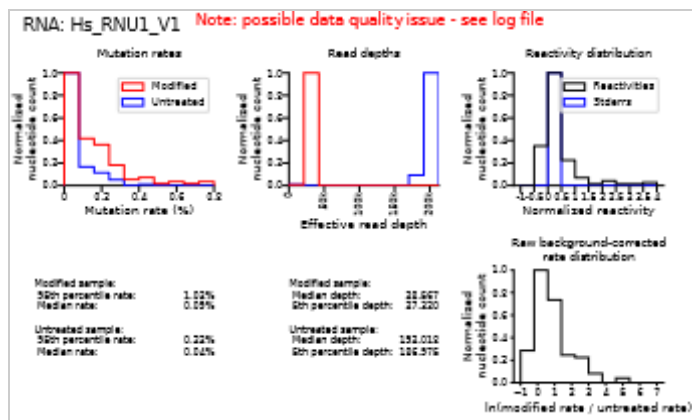
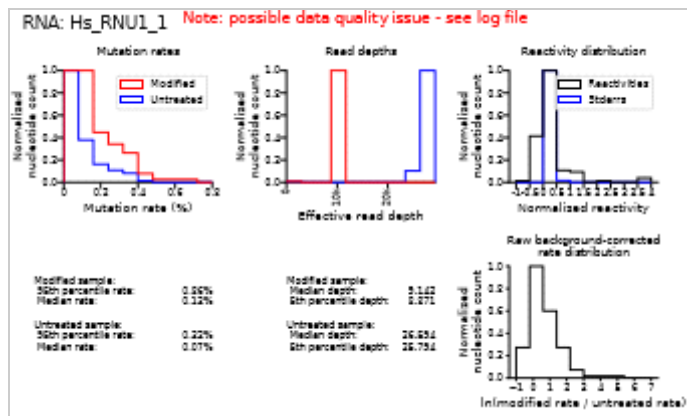
- Run fasta files through shapemapper2
- Look into the steps that need to be taken to organize my notebook

What I Did:

- Ran shapemapper2 using variant fasta's

Results / Observations:

- Issues occurred with matching the primer sequences with each variant. After adjusting each primer sequence, shapemapper2 ran. An error was called stating that the less abundant sequences didn't reach the required depth of 4000, rendering an error. Advice was given by Chase to get rid of any variants that fall below the threshold rather than lowering it. This maintains the trustworthiness of the data we are getting. Also, Considering that all the primers are varying in a specific region and we can logically conclude that they are the same, we are trimming off the end sequences that overlap with the primer to maintain the variant regions.
- Looking at the shapemapper2 data I was capable of generating, U1 and V1 showed to maintain the increased mutation rate seen with the combined U1 run. This indicates that the method appears to work.



Future:

- Tomorrow I will
 - o Trim reads
 - o Rerun dada2
 - o Rerun shapemapper2

Attachments:

FRIDAY

Objective:

Fix environment manager

What I Did:

Fixed environment manager

Results / Observations:

Environment manager issues were resolved, allowing pipelines to be run without activation or dependency errors.

Future:

Tomorrow I will

- o Trim reads
- o Rerun dada2

- Rerun shapemapper2

Attachments:

Weekly Summary:

What worked:

- ShapeMapper2 successfully ran on variant-specific FASTA files after primer adjustments.
- Variant-level ShapeMapper2 outputs reproduced expected mutation rate trends (U1 and V1).
- Environment manager issues were resolved, restoring pipeline usability.

What didn't:

- Cluster downtime delayed initial analyses.
- Low-abundance variants failed to meet ShapeMapper2 read depth requirements.

Problems / Troubleshooting:

- Primer–variant mismatches required manual correction.
- ShapeMapper2 depth threshold errors required exclusion of low-abundance variants rather than parameter relaxation.

Next steps:

- Trim reads to remove primer-overlapping regions.
- Rerun DADA2 with updated trimming strategy.
- Rerun ShapeMapper2 on filtered, high-confidence variants.
- Continue organizing and updating the lab notebook to reflect finalized workflows and decisions.

002 (Jan 12 – Jan 19, 2026)

Monday, January 12, 2026 10:26 AM

Goals for the week:

- Prepare samples for sequencing
 - o Maybe grow cells?
 - o Check RNA quality to make timeline
- Re-trim ends from U1 reads then dada2 --> make fasta --> Shapemapper2 -- RNP-MaP
- Try to plan committee meeting

Monday

Objective:

- Get through splicing paper
- Look into top hit U1 variant
- Check Quality of all RNA stock
- 1-on-1 with Chase

What I Did:

Results / Observations:

Attachments:

Tuesday

Objective:

- Read papers on splicing

What I Did:

- Read Papers on splicing

Results / Observations:

Attachments:

Wednesday

Objective:

Trim R1 and R2 reads for U1 and rerun through dada2. Also, rim reads from different cell line (from Chase) and run through dada2. Want to determine the difference between variant calls to see if top hit splicing variant is HeLa specific or HeLa splicing factor

What I Did:

Results / Observations:

- To remove the additional sequences to run dad2, I used the following code black on my twice cutadapt trimmed reads:

```
cutadapt -j 4 \  
-u 25 -U 22 \  
-o "$out1" -p "$out2" \  
"$r1" "$r2"
```

- -j 4 → use 4 threads per file pair for faster processing.
- -u 25 → trim 25 nt from the 5' end of Read 1.
- -U 22 → trim 22 nt from the 5' end of Read 2.
- -o → output for R1
- -p → output for R2
- "\$r1" "\$r2" → input paired-end FASTQ files.

Notes: The negative numbers (-25 and -22) indicate trimming from the **3' end**. Positive numbers would trim from the 5' end.

This provided me with the following output:

This is cutadapt 5.2 with Python 3.12.12

Command line parameters: -j 4 -u 25 -U 22 -o /home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads/Trimmed_U1_Reads/trimmed_reads_part_2/Trimm_3_01142026/trimmed_reads_part_3-trimmed_reads_part_2-Trimmed_U1_Reads-HeLa-U1-DMSOaq-Rep2_S22_L001_R1_001.fastq.gz -p /home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads/Trimmed_U1_Reads/trimmed_reads_part_2/Trimm_3_01142026/trimmed_reads_part_3-trimmed_reads_part_2-Trimmed_U1_Reads-HeLa-U1-DMSOaq-Rep2_S22_L001_R2_001.fastq.gz trimmed_reads_part_2-Trimmed_U1_Reads-HeLa-U1-DMSOaq-Rep2_S22_L001_R1_001.fastq.gz trimmed_reads_part_2-Trimmed_U1_Reads-HeLa-U1-DMSOaq-Rep2_S22_L001_R2_001.fastq.gz

Processing paired-end reads on 4 cores ...

=== Summary ===

Total read pairs processed: 495,020
Pairs written (passing filters): 495,020 (100.0%)
Total base pairs processed: 145,154,438 bp

Read 1: 72,935,388 bp
 Read 2: 72,219,050 bp
 Total written (filtered): 121,889,567 bp (84.0%)
 Read 1: 60,560,741 bp
 Read 2: 61,328,826 bp

25 nucleotide from the 5' end of R1 and 22 from the 5' end of R2 were trimmed off because R1 doesn't contain the additional 5 error prone nucleotides that were added from RT, the primer sequence (15), and the random 5 nucleotides that were added from Step 1. For R2, there are no error prone nucleotides, so we are only removing the 5 random nucleotides and the primer sequence (17). The processed reads were then ran through dada2, providing the following out file:

```

Loading required package: Rcpp
[1] "dada2_u1_R_code_12092025.sh"
[2] "dada2_U1.R"
[3] "DADA2_wjarret_39872415.out"
[4] "DADA2_wjarret_39872419.out"
[5] "filtered"
[6] "trimmed_reads_part_3-trimmed_reads_part_2-Trimmed_U1_Reads-HeLa-U1-DMSOaq-Rep2-F_S22_L001_R1_001.fastq.gz"
[7] "trimmed_reads_part_3-trimmed_reads_part_2-Trimmed_U1_Reads-HeLa-U1-DMSOaq-Rep2-R_S22_L001_R2_001.fastq.gz"
[1] "trimmed"
[1] "trimmed"
Removing ALL files inside filtered/:
[1] "/home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads/Trimmed_U1_Reads/trimmed_reads_part_2/Trimm_3_01142026/dada2_processing_01142026/filtered/trimmed_F_filt.fastq.gz"
[2] "/home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads/Trimmed_U1_Reads/trimmed_reads_part_2/Trimm_3_01142026/dada2_processing_01142026/filtered/trimmed_R_filt.fastq.gz"
[1] TRUE TRUE

                                trimmed
"/home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads/Trimmed_U1_Reads/trimmed_reads_part_2/Trimm_3_01142026/dada2_processing_01142026/filtered/trimmed_F_filt.fastq.gz"

                                trimmed
"/home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads/Trimmed_U1_Reads/trimmed_reads_part_2/Trimm_3_01142026/dada2_processing_01142026/filtered/trimmed_R_filt.fastq.gz"
Contents of filtered/ immediately before filterAndTrim:
character(0)

                                reads.in
trimmed_reads_part_3-trimmed_reads_part_2-Trimmed_U1_Reads-HeLa-U1-DMSOaq-Rep2-F_S22_L001_R1_001.fastq.gz 495020
                                reads.out
trimmed_reads_part_3-trimmed_reads_part_2-Trimmed_U1_Reads-HeLa-U1-DMSOaq-Rep2-F_S22_L001_R1_001.fastq.gz 494754
60555115 total bases in 494754 reads from 1 samples will be used for learning the error rates.
61322541 total bases in 494754 reads from 1 samples will be used for learning the error rates.
Warning messages:
1: In scale_y_log10() : log-10 transformation introduced infinite values.
2: In scale_y_log10() : log-10 transformation introduced infinite values.
null device
1
Warning messages:
1: In scale_y_log10() : log-10 transformation introduced infinite values.
2: In scale_y_log10() : log-10 transformation introduced infinite values.
null device
1
Sample 1 - 494754 reads in 25158 unique sequences.
Sample 1 - 494754 reads in 35777 unique sequences.
478789 paired-reads (in 69 unique pairings) successfully merged out of 491300 (in 651 pairings) input.
[1] 1 69
114 127 129 146 151
1 56 3 7 2
Wrote Excel file to: /home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads/Trimmed_U1_Reads/trimmed_reads_part_2/Trimm_3_01142026/dada2_processing_01142026/sequence_table_with_sequences_U1.xlsx
=== Job finished at 18:07:16 ===

```

This generated the attached excel sheet with the U1 variants. Error plots are also attached below from the dada2 run. Data interpretation will be done in the future.

Future:

- Tomorrow I will:
 - o interpret the dada2 results
 - o Trim and process Chase's data from a different cell line

- Potentially do step 1 and step 2 for other snRNA

Attachments:



sequence_t
able_with...



U1_error_
plot_R



U1_error_
plot_F

Thursday

Objective:

- interpret the dada2 results
- Trim and process Chase's data from a different cell line
- Potentially do step 1 and step 2 for other snRNA

What I Did:

- Interpreted the dada2 results and ran top 99% of variants through shapemapper2
- Began trimming and processing sequences from Chase

Results / Observations:

- Due to the additional trimming, dada2 processed the data faster, and called less variants. Considering that Each variant was ran through shapemapper2 and will be subsequently ran through RNP-MaP to obtain the binding profiles.
 - Looking at the two shapmapper2 profiles, one containing the U1 file with the variants parsed out and the other unprocessed, we can see an change in the overall profile. What's important to note, is that the sm binding site remained similar in both cases.
- Following dada2 analysis, I ran the files chase provided through the first trimming process. Containing 20 million reads, this has taken some time, so I will be continuing this tomorrow.

Attachments:



U1_DMSO
_int_2026...



U1_DMSO
_int_2025...

FRIDAY

Objective:

- Take Shapemapper2 output and run through RNP-MaP
- See what the Sm-Site profile moving average looks like
- Run Chases Hek293 U1 data through dada2

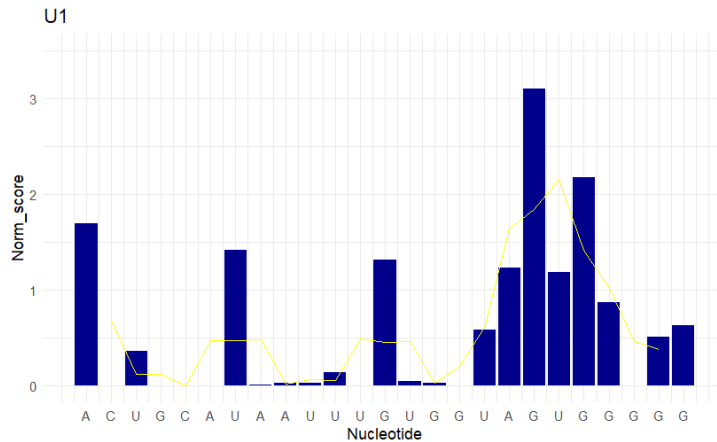
What I Did:

- All tasks complete

Results / Future:

- Looking at the attached RNP-MaP data, it appears that Sm-site binding profile resembles the other U1 profiles. I will be performing this exact same process with data from a different U1 sequencing run. Following, I will perform a PCA to determine the variation between this data and previous U1 profiles before denoising.

Attachments:



Weekly Summary:

What worked

- Successfully re-trimmed U1 paired-end reads using cutadapt with adjusted 5' trimming parameters, which streamlined downstream processing.
- Re-running DADA2 on the additionally trimmed reads completed efficiently, with high read retention and successful merging, and produced a clean variant table for U1.
- Parsing the top ~99% of U1 variants into ShapeMapper2 and subsequently into RNP-MaP was effective, yielding interpretable binding profiles.
- Comparative analysis between processed and unprocessed ShapeMapper2 profiles showed that, despite global profile differences, the Sm-site binding region remained consistent.
- Initial trimming and processing of Chase's data from an alternative cell line progressed as planned, despite larger data volume.
- End-to-end completion of ShapeMapper2 → RNP-MaP for U1 on Friday confirmed pipeline robustness.

What didn't

- Processing of Chase's ~20 million read dataset was time-intensive, preventing full completion within the planned day.
 - Required 11 hours, completed on Saturday morning
- Interpretation of DADA2 error plots and biological significance of all variants was deferred due to time constraints.

Problems / Troubleshooting

- DADA2 generated warnings related to log-scale transformations (infinite values), which did not halt execution but will require consideration during interpretation.
- Additional trimming reduced the number of variants called, necessitating careful evaluation of whether low-abundance variants are being biologically filtered or technically excluded.
- Managing multiple rounds of trimming and intermediate directories required careful bookkeeping to avoid confusion between datasets.

Next steps

- Complete trimming and full DADA2 processing of Chase's HEK293 data and compare variant calls across cell lines.
- Perform deeper interpretation of DADA2 outputs, including variant frequency distributions and error profiles.
- Run the same ShapeMapper2 and RNP-MaP pipeline on additional U1 sequencing runs for consistency checks.
- Conduct PCA on RNP-MaP outputs to assess variance between datasets prior to denoising.
- Extend steps 1 and 2 of the pipeline to other snRNAs as time permits.
- Continue planning and scheduling the committee meeting once sequencing timelines are clearer.

003 Jan 19 – Jan 26, 2026

Monday, January 19, 2026 12:35 PM

Goals for the week:

- Run other U1 Replicate through dada2
- Run PCA with outputs
- Prepare snRNA for sequencing

Monday

Objective:

- Finish abstract for RNA symposium

What I Did:

- Finish abstract for RNA symposium

Results / Observations:

Attachments:

Tuesday

Objective:

- Run Replicate 2 U1 data through dada2
- Perform bead clean up and run tape station

What I Did:

Today, I performed adapter trimming and primer trimming on U1 data. Following, the output fasta files were put into dada2.

Results / Observations:

Samples inputted into the 07072025 sequencing run were ran on a 300 mi-seq kit. Not requiring trimming as the inserts were not short than 150 nt, samples were still subjected to adapter trimming. After the first trimming process in cutadapt, targeting i7 and i5 adapter sequences. The following output was provided:

status	in_reads out2_bp	in_bp	too_short	too_long	too_many_n	out_reads	w/adapters	qualtrim_bp	out_bp w/adapters2	qualtrim2_bp
WARN	1164956	374681393	0	0	0	1164956	367320 0	185339188	427245 0	184725417

From <https://greatlakes.arc-ts.umich.edu/pun/sys/dashboard/files/fs//home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads_rep2/cutadapt/wjarret_40478737_4294967294.out>

Following, the trimmed outputs were ran again through cutadapt targeting Tn7 and Tn5 sequences. The following output was provided:

status	in_reads out2_bp	in_bp	too_short	too_long	too_many_n	out_reads	w/adapters	qualtrim_bp	out_bp w/adapters2	qualtrim2_bp
OK	1164956	370064605	0	0	0	1164956	103286 0	184671065	621697 0	180630097

From <https://greatlakes.arc-ts.umich.edu/pun/sys/dashboard/files/fs//home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads_rep2/trim_2_transposase_sequences/cutadapt/wjarret_40479219_4294967294.out>

After, a third cutadapt run was performed, targeting the 5 prime regions of R1 and R2. 25 nucleotide from the 5' end of R1 and 22 from the 5' end of R2 were trimmed off because R1 doesn't contain the additional 5 error prone nucleotides that were added from RT, the primer sequence (15), and the random 5 nucleotides that were added from Step 1. For R2, there are no error prone nucleotides, so we are only removing the 5 random nucleotides and the primer sequence (17). The processed reads were then ran through dada2, providing the following out file:

This is cutadapt 5.2 with Python 3.12.12

Command line parameters: -j 4 -u 25 -U 22 -o /home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads_rep2/trim_2_transposase_sequences/Trim_3_end_trimming_5_prime/trimmed_reads_part_3-Trimmed_U1_Reads_2-Trimmed_U1_Reads-JW-HeLa-U1-DMSO-Aq_S49_L001_R1_001.fastq.gz -p /home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads_rep2/trim_2_transposase_sequences/Trim_3_end_trimming_5_prime/trimmed_reads_part_3-Trimmed_U1_Reads_2-Trimmed_U1_Reads-JW-HeLa-U1-DMSO-Aq_S49_L001_R2_001.fastq.gz Trimmed_U1_Reads_2-Trimmed_U1_Reads-JW-HeLa-U1-DMSO-Aq_S49_L001_R1_001.fastq.gz Trimmed_U1_Reads_2-Trimmed_U1_Reads-JW-HeLa-U1-DMSO-Aq_S49_L001_R2_001.fastq.gz

Processing paired-end reads on 4 cores ...

=== Summary ===

Total read pairs processed: 1,164,956
Pairs written (passing filters): 1,164,956 (100.0%)
Total basepairs processed: 365,301,162 bp
Read 1: 184,671,065 bp
Read 2: 180,630,097 bp
Total written (filtered): 310,548,237 bp (85.0%)
Read 1: 155,547,172 bp
Read 2: 155,001,065 bp

After last round trimming, we were left with the following stats:

file	format	type	num_seqs	sum_len	min_len	avg_len	max_len
Trimmed_U1_Reads_2-Trimmed_U1_Reads-JW-HeLa-U1-DMSO-Aq_S49_L001_R1_001.fastq.gz	FASTQ	DNA	1,164,956	184,671,065	18	158.5	216

The trimmed reads from this experiment were run through dada2 overnight

Discussion

Sequencing reads of rep2:

file	format	type	num_seqs	sum_len	min_len	avg_len	max_len
trimmed_reads_part_2-Trimmed_U1_Reads-HeLa-U1-DMSOaq-Rep2-F_S22_L001_R1_001.fastq.gz	FASTQ	DNA	495,020	72,935,388	13	147.3	151

Attachments:

Wednesday

Objective:

What I Did:

Results / Observations:

Comparing the stats from the last trimming output, there are non-negligible differences in the overall trimming process:

Metric	Replicate 1	Replicate 2	
num_seqs	1,164,956	495,020	
sum_len (bp)	184,671,065	60,560,741	
min_len (bp)	18	0	
avg_len (bp)	158.5	122.3	
max_len (bp)	216	126	
Q1 (bp)	169	120	
Median (Q2)	173	120	
Q3 (bp)	174	126	
N50 (bp)	174	120	N50 describes the shortest sequence that contains the majority of the base content in your dataset. This is different from average as it can be heavily skewed at its tails due to increased number of reads. N50 is taking the sequence length distribution weighted by total base content, not by the number of reads.
N50_num	43	5	
Q20 (%)	97.32	98.37	
Q30 (%)	96.36	97.81	
AvgQual	29.92	31.8	
GC (%)	53.96	51.99	
sum_n	70	0	

Looking at the variant output file for dada2, there is a considerable difference when compared to the previous two runs. About 1/6 of the sequences appear to have the full length U1 sequences in them, leading me to believe that the trimming wasn't as successful as we had hoped.

Considering the variability trimming outcomes, I looked back to the overall similarities in the Bowtie aligner output:

849035 reads; of these:

- | 70653 (8.32%) were paired; of these:
- | 47429 (67.13%) aligned concordantly 0 times
- | 23223 (32.87%) aligned concordantly exactly 1 time

```
| 1 (0.00%) aligned concordantly >1 times
| ----
| 47429 pairs aligned concordantly 0 times; of these:
| 171 (0.36%) aligned discordantly 1 time
| ----
| 47258 pairs aligned 0 times concordantly or discordantly; of these:
| 94516 mates make up the pairs; of these:
| 56776 (60.07%) aligned 0 times
| 37740 (39.93%) aligned exactly 1 time
| 0 (0.00%) aligned >1 times
| 778382 (91.68%) were unpaired; of these:
| 18900 (2.43%) aligned 0 times
| 759474 (97.57%) aligned exactly 1 time
| 8 (0.00%) aligned >1 times
| 91.77% overall alignment rate
```

From <https://greatlakes.arc-ts.umich.edu/pun/sys/dashboard/files/fs//home/wjarret/scratchjw/20250525_HeLa_U1_RNPMaP/shapemapper_wjarret_26347612_0.out>

This data indicates that 8.32% of overall reads were paired. Comparing this to the other U1 run, this is smaller, though the second run has a lower overall alignment rate

```
143128 reads; of these:
| 18010 (12.58%) were paired; of these:
| 16067 (89.21%) aligned concordantly 0 times
| 1943 (10.79%) aligned concordantly exactly 1 time
| 0 (0.00%) aligned concordantly >1 times
| ----
| 16067 pairs aligned concordantly 0 times; of these:
| 5 (0.03%) aligned discordantly 1 time
| ----
| 16062 pairs aligned 0 times concordantly or discordantly; of these:
| 32124 mates make up the pairs; of these:
| 27968 (87.06%) aligned 0 times
| 4156 (12.94%) aligned exactly 1 time
| 0 (0.00%) aligned >1 times
| 125118 (87.42%) were unpaired; of these:
| 2446 (1.95%) aligned 0 times
| 122660 (98.04%) aligned exactly 1 time
| 12 (0.01%) aligned >1 times
| 81.13% overall alignment rate
```

From <https://greatlakes.arc-ts.umich.edu/pun/sys/dashboard/files/fs//home/wjarret/scratchjw/20250707_HeLa_U1-12RNPMaP/shapemapper_wjarret_28024685_1.out>

- **Concordant alignment:**
 - Both R1 and R2 align to the reference **in the correct relative orientation and distance**.
 - Bowtie considers the pair “concordantly aligned.”
- **Discordant alignment:**
 - Both reads align, but **orientation or distance** is unusual or outside expected insert size.
- **Unpaired / single-end alignment:**
 - Only one read aligns (the other is missing or too short).

The alignments from each sequencing run proved successful, showing 91% and 81% overall alignment respectively. One thing that was noted is that the samples from 05252025 was ran on a 500 kit. Two additional elements that have been left out in both trimming processes. One, given the expected length of the amplicons, and the absence of read merging, smaller and lower quality reads are left in during the dada2 run. Two, upon 5' end trimming of the R1 and R2 reads, a complementary 3' overhang is present on each read. To obtain better quality data, the two steps were performed; step 1 in fastp and step 2 in cutadapt. The following output was put into cutadapt.

For fastp, the following code was ran:

```
fastp \
-i /home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads_rep2/trim_2_transposase_sequences/Trim_3_end_trimming_5_prime/fastp_quality_trimming_01212026/Trimmed_U1_Reads_2-Trimmed_U1_Reads-JW-HeLa-U1-DMSO-Aq_S49_L001_R1_001.fastq.gz \
-l /home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads_rep2/trim_2_transposase_sequences/Trim_3_end_trimming_5_prime/fastp_quality_trimming_01212026/Trimmed_U1_Reads_2-Trimmed_U1_Reads-JW-HeLa-U1-DMSO-Aq_S49_L001_R2_001.fastq.gz \
-o /home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads_rep2/trim_2_transposase_sequences/Trim_3_end_trimming_5_prime/fastp_quality_trimming_01212026/fastp_trimmed-Trimmed_U1_Reads_2-Trimmed_U1_Reads-JW-HeLa-U1-DMSO-Aq_S49_L001_R1_001.fastq.gz \
-O /home/wjarret/scratchjw/U1_Trimming_and_Processing_for_dada2/U1_reads_rep2/trim_2_transposase_sequences/Trim_3_end_trimming_5_prime/fastp_quality_trimming_01212026/fastp_trimmed-Trimmed_U1_Reads_2-Trimmed_U1_Reads-JW-HeLa-U1-DMSO-Aq_S49_L001_R2_001.fastq.gz \
--correction \
-q 20 \
--n_base_limit 0 \
--thread 4
#-i = input directory
#-o = output directory
```



```
#--correction = base correction
#-q 20 = quality score threshold
#--n_base_limit 0 = maximum number of N bases allowed in reads
#eval "${tasks[0]}"
```

From https://greatlakes.arc-ts.umich.edu/pun/sys/dashboard/files/fs//home/wjarret/scratchiw/U1_Trimming_and_Processing_for_dada2/U1_reads_rep2/trim_2_transposase_sequences/Trim_3_end_trimming_5_prime/fastp_quality_trimming_01212026/fastp_script_quality_trimming_01212026.sh

In this run, a base correction was performed, replacing low quality bases with higher quality counterparts. A threshold of 20 was placed for the Q score, throwing out all reads that fell below. All reads containing non-called (N) nucleotides were removed. The following .out file showed the effect of the editing.

```
Read1 before filtering:
total reads: 1164956
total bases: 184671065
Q20 bases: 179713068(97.3152%)
Q30 bases: 177944275(96.3574%)
Read2 before filtering:
total reads: 1164956
total bases: 180630097
Q20 bases: 173721739(96.1754%)
Q30 bases: 171925202(95.1808%)
Read1 after filtering:
total reads: 1143005
total bases: 181283747
Q20 bases: 177202977(97.749%)
Q30 bases: 175654080(96.8946%)
Read2 after filtering:
total reads: 1143005
total bases: 177155639
Q20 bases: 172024480(97.1036%)
Q30 bases: 170535762(96.2632%)
Filtering result:
reads passed filter: 2286010
reads failed due to low quality: 43902
reads failed due to too many N: 0
reads failed due to too short: 0
reads with adapter trimmed: 448
bases trimmed due to adapters: 5368
reads corrected by overlap analysis: 89569
bases corrected by overlap analysis: 118664
Duplication rate: 6.62834%
Insert size peak (evaluated by paired-end reads): 174
```

From https://greatlakes.arc-ts.umich.edu/pun/sys/dashboard/files/fs//home/wjarret/scratchiw/U1_Trimming_and_Processing_for_dada2/U1_reads_rep2/trim_2_transposase_sequences/Trim_3_end_trimming_5_prime/fastp_quality_trimming_01212026/fastp_wjarret_40652552.out

Overall, the use of fastp quality trimming provided minimal improvement, removing 43902 reads from the quality filter, and corrected 89569 reads by overlap analysis, and 118664 bases by overlap analysis. Following this, cutadapt was used to remove both 5' primer regions, 5 random nucleotides at the beginning of the sequence, and for R1, 5 error prone nucleotides at the start of RT, and the resulting 3' overhang from each sequence.

```
for r1 in *_R1_*.fastq.gz; do
  r2="${r1/_R1/_R2_}"
  if [[ -f "$r2" ]]; then
    out1="/home/wjarret/scratchiw/U1_Trimming_and_Processing_for_dada2/U1_reads_rep2/trim_2_transposase_sequences/Trim_3_end_trimming_5_prime/fastp_quality_trimming_01212026/cutadapt_trimming_5_and_3_prime_ends/trimmed_reads_part_3-$(basename "$r1")"
    out2="/home/wjarret/scratchiw/U1_Trimming_and_Processing_for_dada2/U1_reads_rep2/trim_2_transposase_sequences/Trim_3_end_trimming_5_prime/fastp_quality_trimming_01212026/cutadapt_trimming_5_and_3_prime_ends/trimmed_reads_part_3-$(basename "$r2")"
    cutadapt -j 4 \
      -u 25 -U 22 \
      -u -22 -U -25 \
      --minimum-length 100 \
      --maximum-length 180 \
      -o "$out1" -p "$out2" \
      "$r1" "$r2"
  else
    echo "Warning: Paired file $r2 not found for $r1, skipping."
  fi
done
```

From https://greatlakes.arc-ts.umich.edu/pun/sys/dashboard/files/fs//home/wjarret/scratchiw/U1_Trimming_and_Processing_for_dada2/U1_reads_rep2/trim_2_transposase_sequences/Trim_3_end_trimming_5_prime/fastp_quality_trimming_01212026/cutadapt_trimming_5%20and_3_prime_ends/Cutadapt_script_trimming_01212026_primers.sbatch

The following .out file was provided:

=== Summary ===

Total read pairs processed: 1,143,005
 == Read fate breakdown ==
 Pairs that were too short: 203,485 (17.8%)
 Pairs that were too long: 0 (0.0%)
 Pairs written (passing filters): 939,520 (82.2%)
 Total basepairs processed: 358,439,386 bp
 Read 1: 181,283,747 bp
 Read 2: 177,155,639 bp
 Total written (filtered): 232,239,586 bp (64.8%)
 Read 1: 117,818,887 bp
 Read 2: 114,420,699 bp

From <https://greatlakes.arc-ts.umich.edu/pun/sys/dashboard/files/fs//home/wjarret/scratchiw/U1_Trimming_and_Processing_for_dada2/U1_reads_rep2/trim_2_tansposase_sequences/Trim_3_end_trimming_5_prime/fastp_quality_trimming_01212026/cutadapt_trimming_5%20and_3_prime_ends/cutadapt_wjarret_40652953_4294967294.out>

Seqkit provided an overview of the effect of the overall trimming. When comparing side by side with the data from the 300 kit, the trimmed control that contains the expected insert size with error after trimming, we see that the trimming appears to work despite difference in average length and N50. This is to be expected as there is a difference in the number of reads, with the 500 kit providing a higher number of sequences than the 300 kit.

Sample	U1 300 kit (07072025)	U1 500 kit (052522025)
File	trimmed_reads_part_3-fastp_trimmed-trimmed_reads_part_2-Trimmed_U1_Reads-HeLa-U1-DMSOaq-Rep2_S22_L001_R1_001.fastq.gz	trimmed_reads_part_3-fastp_trimmed-Trimmed_U1_Reads_2-Trimmed_U1_Reads-JW-HeLa-U1-DMSO-Aq_S49_L001_R1_001.fastq.gz
Format	FASTQ	FASTQ
Type	DNA	DNA
Number of sequences	482,768	939,520
Total length (bp)	59,136,052	117,818,887
Min length	100	100
Average length	122.5	125.4
Max length	126	169
Q1 length	120	123
Median (Q2) length	120	127
Q3 length	126	127
Sum gap	0	0
N50	120	127
N50 count	7	43
Q20 (%)	98.80	97.87
Q30 (%)	98.33	97.10
Average quality	32.84	30.82
GC (%)	51.99	52.28
Sum N	0	0

After obtaining the final fastq, each sample was ran through dada2 providing the following excel file containing the ASVs (attached below).

Tomorrow:

- Run sequence alignment with variants
- Trim the DMSO aqueous and SDA interface samples
- Run samples through shapemapper2 with respective variant fasta files
- Run through RNP-MaP
- Plot moving average curves
- Run PCA on Data

Attachments:



U1_07072
025_dada...



U1_07072
025_dada...



dada2_U1_
replicate_...



U1_07072
025_dada...



dada2_U1_
_replicate...

Thursday/Friday

Objective:

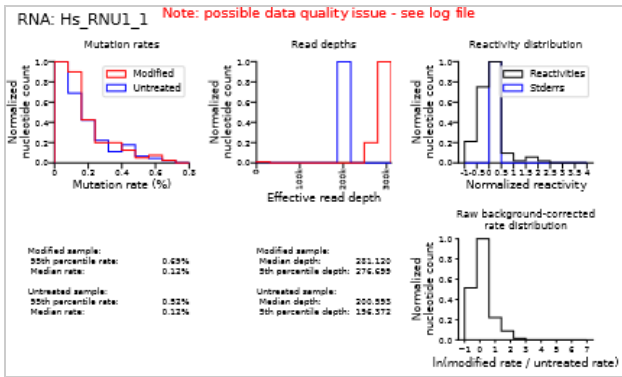
- Run adapter trimming, fastp quality trimming, and dada2 on DMSO U1 samples
- Run Through Shapemapper2 and RNP-MaP

What I Did:

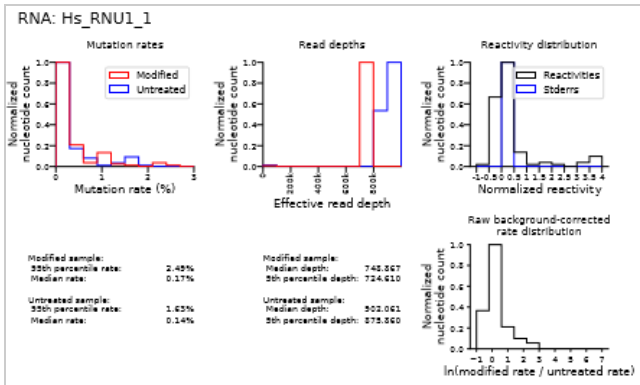
- Ran adapter trimming, fastp quality trimming, and dada2 on DMSO U1 samples
- Ran Through Shapemapper2 and RNP-MaP

Results / Observations:

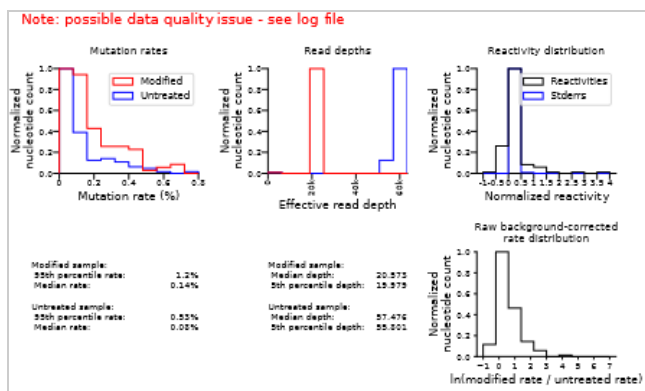
denoised_histograms_05252025



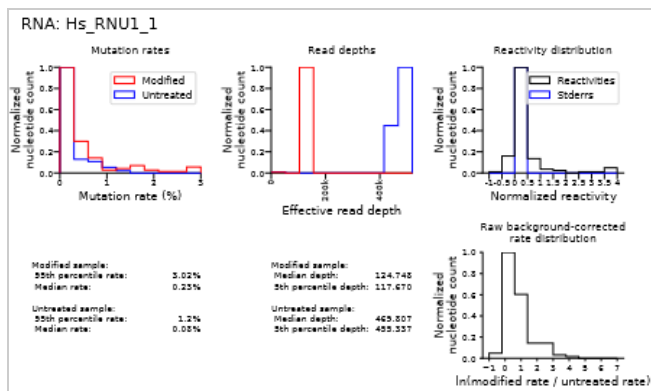
Original_histogram_05252025



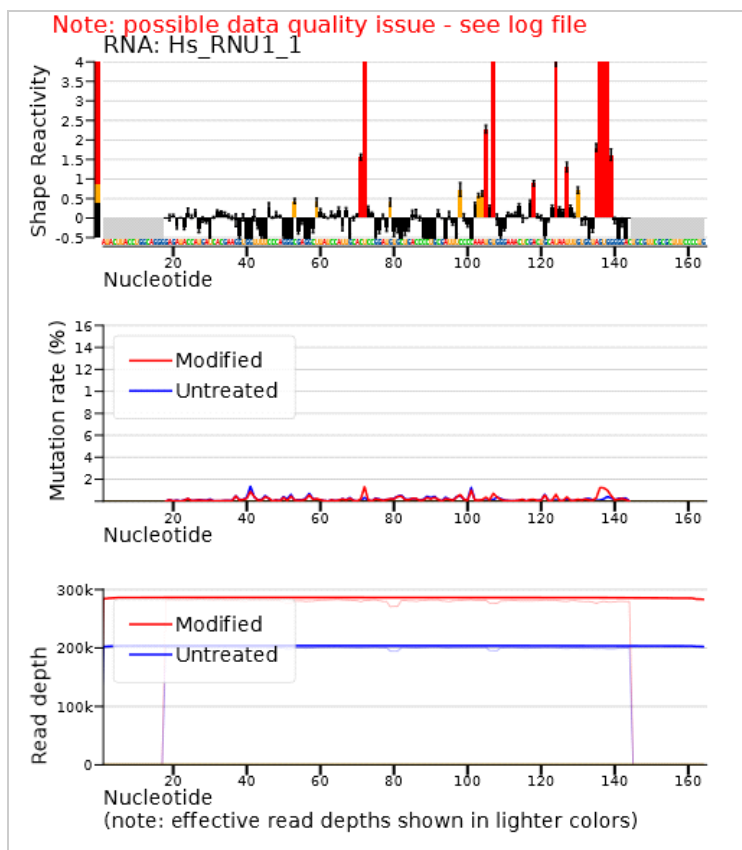
denoised_histograms_07072025



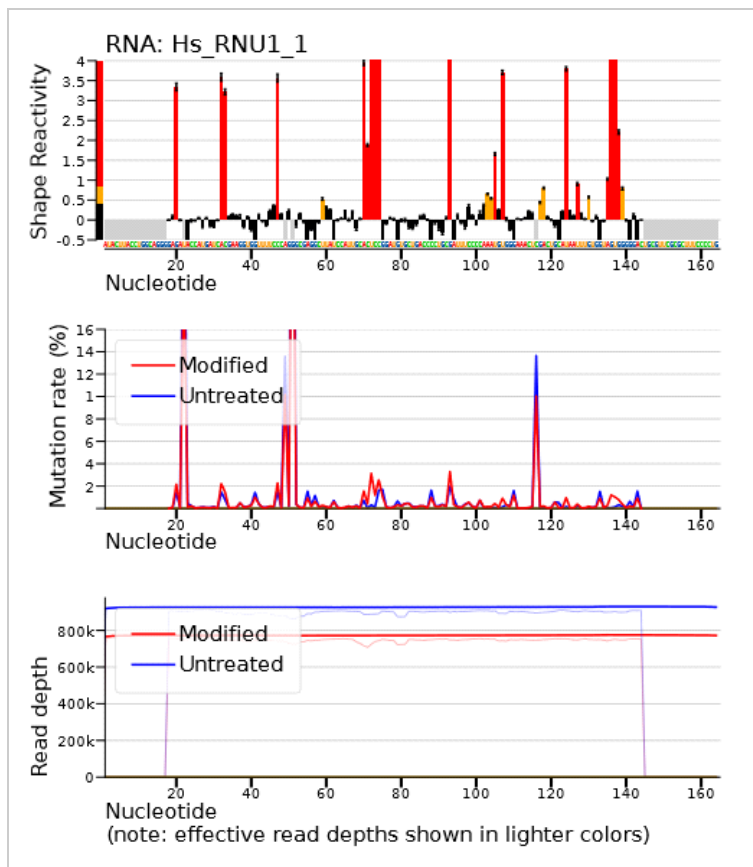
Original_histogram_07072025



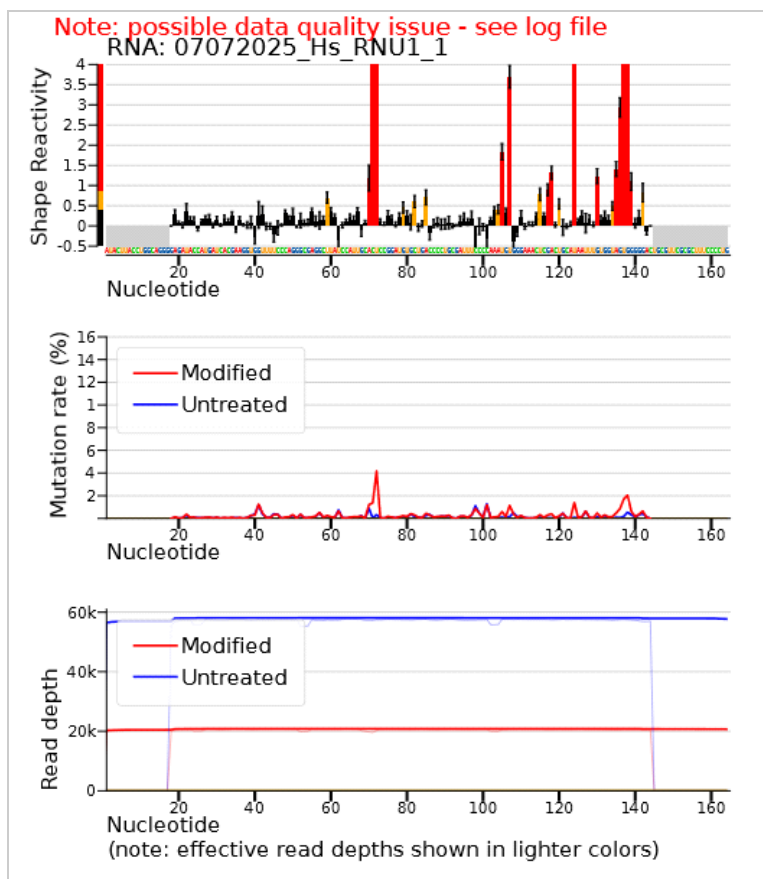
denoised_profile_05252025

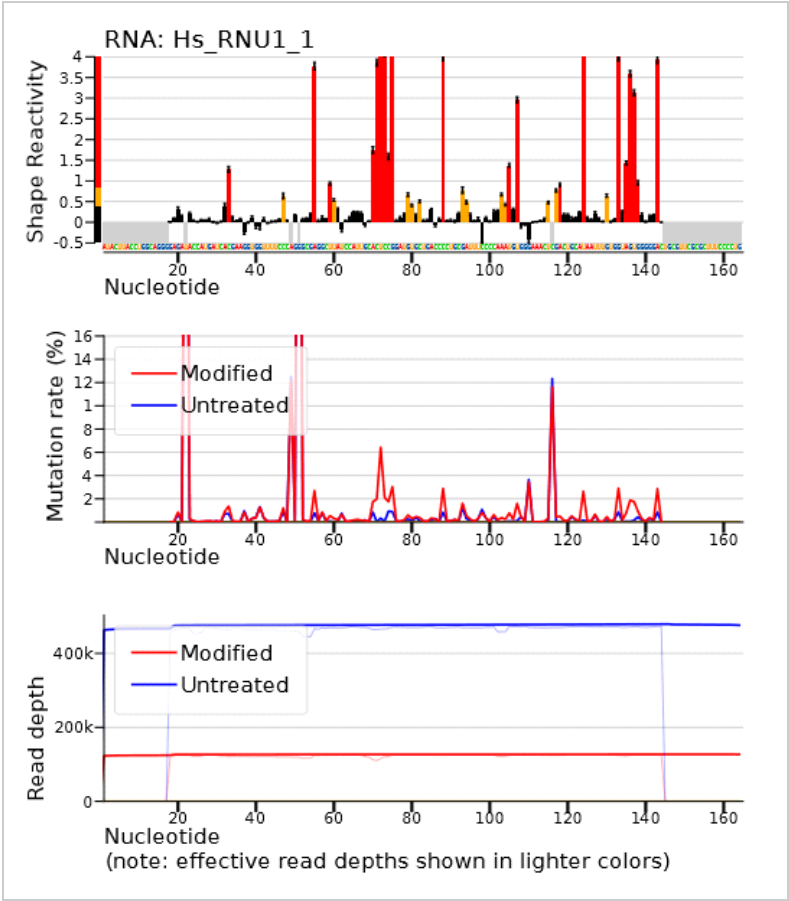


Original_profile_05252025

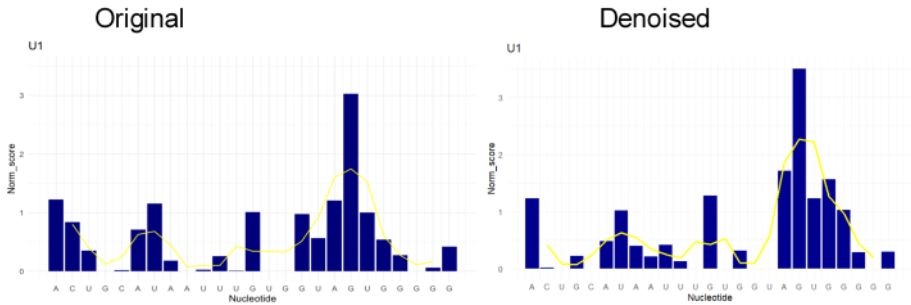


denoised_profile_07072025

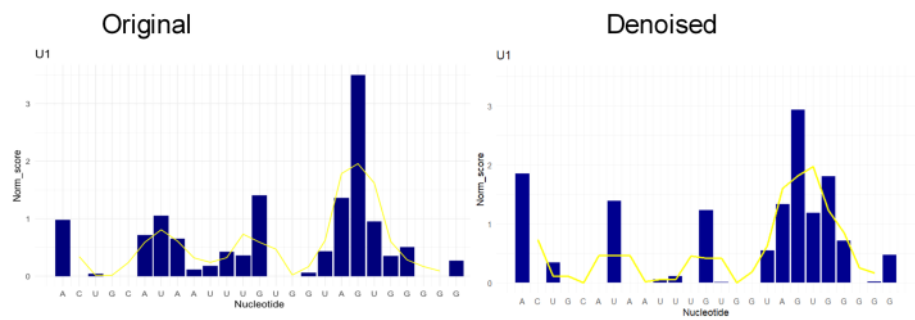




RNP_MaP Profile (05/25)



RNP_MaP Profile (07/07)

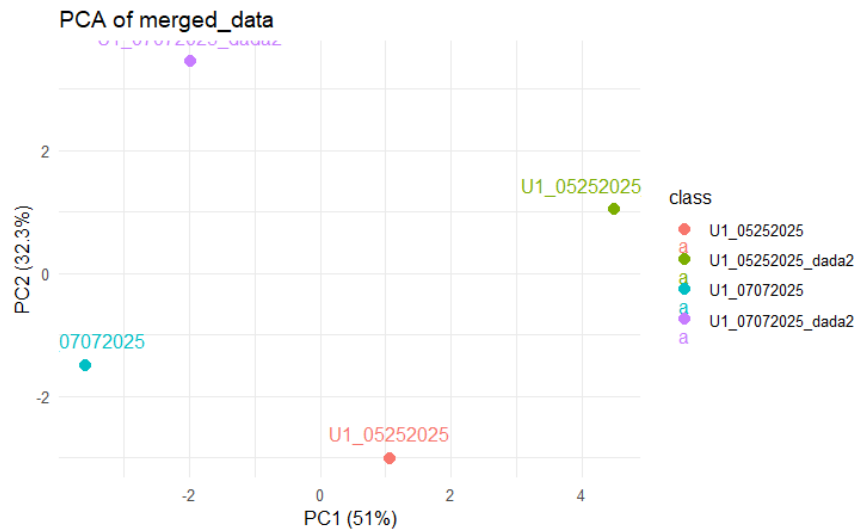


PCA Analysis (Correlation Matrix)

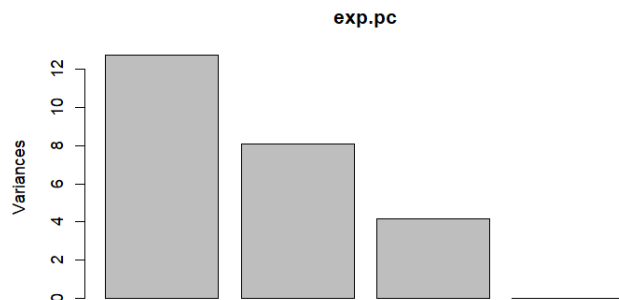
	U1_05252025	U1_05252025_dada2	U1_07072025	U1_07072025_dada2
U1_05252025	1.00			
U1_05252025_dada2	0.92	1.00		
U1_07072025	0.90	0.85	1.00	
U1_07072025_dada2	0.84	0.92	0.84	1.00

PCA Analysis (Co-Variance Matrix)

	U1_05252025	U1_05252025_dada2	U1_07072025	U1_07072025_dada2
U1_05252025	0.59			
U1_05252025_dada2	0.62	0.76		
U1_07072025	0.47	0.50	0.46	
U1_07072025_dada2	0.51	0.64	0.45	0.63



Bar graph Plot of exp.pc



Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	3.571	2.8427	2.0418	1.157e-15
Proportion of Variance	0.510	0.3232	0.1668	0.000e+00
Cumulative Proportion	0.510	0.8332	1.0000	1.000e+00

The shapemapper2 output reduced the mutation rate in both sequences, being accounted for by the reduction of reads as a result of denoising. Observing the shapemapper2 profiles, changes are evident between, denoised and original samples. Both sets of samples were ran through RNP_MaP to determine the impact of denoising on the moving average profile. Visual observations show retention of the profiles structure. The correlation matrix indicate a stronger correlation between biological samples after denoising than before with a correlation value of 0.92 vs 0.90. Looking at the correlation between biological samples before and after denoising, the 05252025 samples have a correlation of 0.92 while the 07072025 samples are 0.84. This indicates that while the denoising changed the profile shape, the overall relationship shape of the profiles don't significantly vary. Looking at the Co-variance matrix, we observe that the co-variance between denoised samples is higher than ASV inclusive samples, 0.47 vs 0.64. The higher covariance values between samples indicate that the profiles show higher similarity in nucleotide reactivity, indicating the interaction patterns are more similar. This indicates that the biological interactions observed in the denoised profiles are preserved more than the original samples. Looking at the PCA analysis, we can see that each sample remove varying directions in the respective PC's with the 05252025 sample moving across PC1 and 07072025 samples moving along PC2. It is important to note that the variation of the original 07072025 sample is not explained by PC1 or PC2, but PC3. Overall, while this data provides a visual explanation of the variance between these samples, performing clustering with all snRNA samples will indicate more information on their clustering. The denoising appeared to work and in the future, fastp quality trimming will be performed after 3' and 5' end trimming to prevent variable changes in the size of the samples, as accidental over trimming can affect the dada2 ASV calling.

Attachments:



denoised_h
istograms...



denoised_h
istograms...



Original_p
rofile_070...



Original_p
rofile_052...



Original_hi
stogram_...



Original_hi
stogram_...



denoised_p
rofile_070...



denoised_p
rofile_052...



denoised_h
istograms...

Weekly Summary: U1 Data Processing & Denoising Analysis

This week focused on processing the second replicate of **U1 snRNA** data through the **DADA2** pipeline and comparing the results of denoised profiles versus original (ASV-inclusive) profiles using **ShapeMapper2** and **RNP-MaP**.

What Worked

- **Pipeline Completion:** Successfully ran U1 Replicate 2 through a multi-stage trimming process (Cutadapt), quality filtering (fastp), and DADA2 denoising.
- **Abstract Submission:** Completed the abstract for the RNA symposium on Monday.
- **Profile Comparison:** Successfully generated and compared reactivity profiles and histograms for both replicates (05252025 and 07072025) in both "Original" and "Denoised" states.
- **Statistical Validation:** Correlation matrices confirmed that **denoising improved the correlation** between biological replicates (from 0.90 to 0.92) and increased covariance (from 0.47 to 0.64), suggesting that biological interaction patterns are better preserved after removing noise.

What Didn't (and Observations)

- **Initial Trimming Variability:** Replicate 2 initially showed significant differences in sequence length and N50 compared to Replicate 1. About 1/6 of sequences appeared to retain full-length U1 sequences, indicating the initial trimming parameters were not aggressive enough.
- **Kit Differences:** You noted that the 05252025 samples were run on a 500 kit while others were on a 300 kit. This led to differences in average length and total sequence counts, though the overall alignment rates remained strong (81%–91%).
- **PCA Variance:** While PCA showed distinct clustering, the original 07072025 sample's variance was not captured by PC1 or PC2, requiring PC3 for explanation.

Problems & Troubleshooting

- **3' Overhangs:** Identified that 5' end trimming of R1 and R2 left complementary 3' overhangs.
 - **Solution:** Implemented a two-step fix using **fastp** for base correction/quality filtering followed by **Cutadapt** to specifically target the 5' and 3' ends.
- **Accidental Over-trimming:** Recognized that quality trimming before end-trimming can cause variable sample sizes.
 - **Process Change:** In the future, you will perform **fastp quality trimming after 3' and 5' end trimming** to ensure more consistent DADA2 ASV calling.

Next Steps

- **Clustering:** Perform comprehensive clustering with **all snRNA samples** to get a broader view of the data relationships beyond just the U1 replicates.
- **Sequencing Prep:** Prepare snRNA for the upcoming sequencing run.

Would you like me to help draft the specific command-line scripts for the revised trimming order (End-trimming then Fastp) for your next run?

From <<https://gemini.google.com/app/376c80114aa63a8c>>

004 Jan 26 – Feb 2, 2026

Monday, January 26, 2026 1:04 PM

Goals for the week:

- Run PCA on with all sample snRNA
- Prepare samples for sequencing
- Denoise all snRNA

Monday

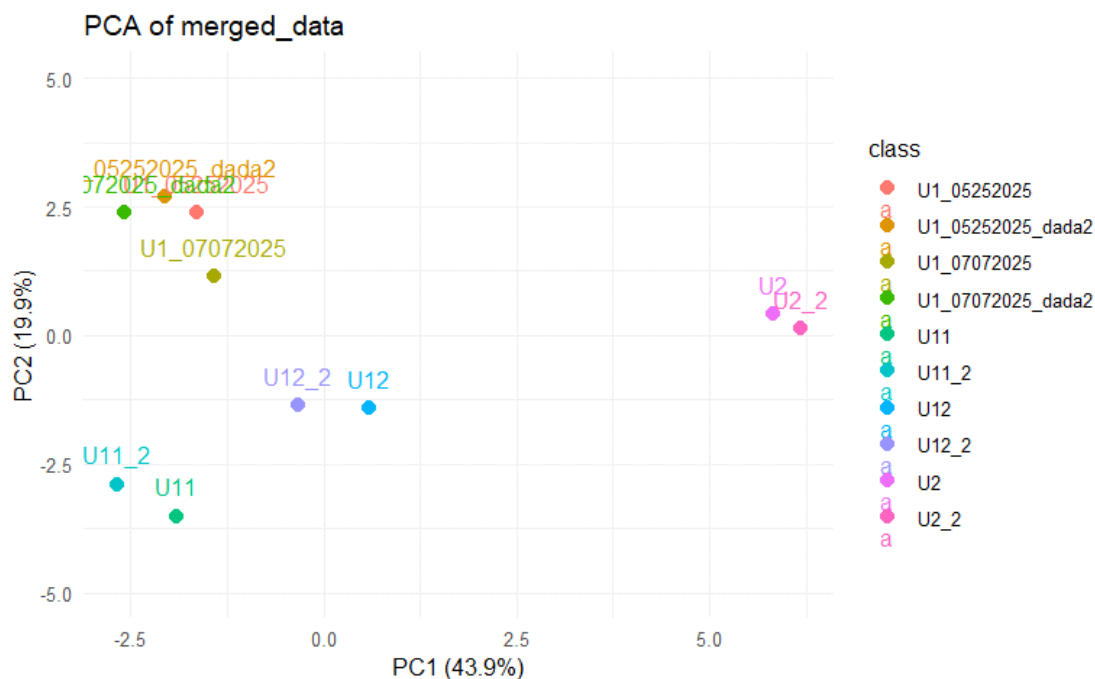
Objective:

- Prepare samples for sequencing
- Run all sample PCA

What I Did:

- Performed step 2 and Bead clean up
- Tape station and qubit samples
- Ran PCA on all snRNA samples

Results / Observations:



Attachments:

Tuesday

Objective:

- Analyze Tapes and pool libraries
- Perform variant parsing on all snRNA samples

What I Did:

- Analyze Tapes and pool libraries

- Perform variant parsing on all snRNA samples

Results / Observations:

- Analyzation of dada2 outputs will take place tomorrow

Attachments:

Wednesday

Objective:

- Perform sequencing run

What I Did:

- Perform sequencing ru

FRIDAY

Objective:

- Analyze dada2_output
- Run sequencing reads through fastp, Shapemapper2, and RNP_MaP

What I Did:

- Performed sequencing
- Looked into ways of developing a pipeline for analyzing snRNA data

Results / Observations:

Attachments:

Here's a concise weekly summary based on your notes:

Weekly Summary

What worked

Sample preparation for sequencing progressed smoothly, including bead cleanup, TapeStation/Qubit validation, library pooling, and completion of the sequencing run. PCA across all snRNA samples was successfully generated, and variant parsing for all snRNA datasets was completed, advancing the datasets toward denoising and downstream analysis. Initial exploration of a standardized snRNA analysis pipeline also began.

What didn't

Planned analysis of DADA2 outputs and downstream processing (fastp → ShapeMapper2 → RNP-MaP) was not completed within the week due to sequencing timing and preparation workload.

Problems / Troubleshooting

No major experimental failures occurred, but sequencing preparation and run coordination shifted analysis tasks later than planned. Pipeline design considerations for multi-snRNA processing remain unresolved and require further implementation work.

Next steps

Analyze DADA2 outputs from the sequencing run, complete denoising for all snRNA datasets, run fastp → ShapeMapper2 → RNP-MaP processing on new reads, refine a reproducible snRNA analysis pipeline, and interpret PCA structure across all samples.

006 02032026 Learning how to use snakemake

Wednesday, January 28, 2026 4:56 PM

Objective:

- Learn how to use Snakemake
- See if I can integrate with GitHub

NOTES

Basic_Snakemake_Tutorial

I worked through a Snakemake tutorial to understand how to structure a reproducible sequencing analysis workflow and how Snakemake determines execution order using file-based dependencies. I learned that Snakemake workflows are composed of **rules**, where each rule defines:

- required **input files**
- produced **output files**
- the **shell command** or script that transforms input into output

Each rule represents a single logical step in the pipeline, and complex workflows are built by chaining rules together.

I examined how Snakemake **automatically infers dependencies** by matching filenames between rule outputs and rule inputs. If one rule produces a file that another rule requires, Snakemake creates a dependency edge between those jobs without explicit instruction.

I practiced using **brace notation** (e.g., {input}, {output}, {wildcards.sample}) inside shell commands to reference rule components dynamically.

Target files, DAGs, and execution logic

I learned that Snakemake is **target-driven**, meaning workflows are executed by requesting output files (target files), not by specifying which rules to run. Snakemake builds a **directed acyclic graph (DAG)** of jobs that are required to generate the requested target, starting from existing files and working upstream.

Using dry runs (-n) and command printing (-p), I inspected how Snakemake would construct execution plans without actually running commands. I confirmed that:

- Snakemake runs *all required upstream jobs* for a target
- Individual rules cannot be run in isolation
- Each concrete execution of a rule is a **job**
- Dependency edges exist strictly because of **file requirements**

I also learned that Snakemake will **not re-run jobs** unless an input file is newer than its corresponding output.

Parallelization and resource control

I documented how Snakemake manages parallel execution using:

- --cores N to define total available cores
- threads: within rules to specify per-job resource usage

Snakemake schedules jobs efficiently to respect both limits. I clarified for myself the distinction between CPUs, cores, and threads to avoid misconfiguring resource requests.

Wildcards and rule generalization

I generalized mapping rules using **wildcards**, allowing the same rule definition to apply to multiple

samples. I verified that Snakemake substitutes wildcard values based on requested target filenames and supports:

- multiple explicit targets
- brace expansion ({A,B})
- scalable workflows without rewriting rules

BAM processing and directory structure

I implemented downstream steps including:

- sorting BAM files with samtools sort
- indexing BAM files with samtools index

I noted that Snakemake automatically creates missing output directories before job execution. I also reinforced the best practice of separating workflow stages into distinct output directories to improve readability and dependency resolution efficiency.

DAG visualization

I generated a visual representation of the workflow DAG using `--dag` and Graphviz, confirming my understanding of job dependencies and execution order.

Aggregation with `expand()` and variant calling

I learned how to use `expand()` to collect multiple input files programmatically, especially when dealing with:

- multiple samples
- replicates
- multiple wildcards

This approach avoids file overwrites, keeps replicate structure explicit, and scales cleanly for experiments like RNP-MaP that involve many conditions.

I used this mechanism to define a variant-calling rule that aggregates sorted BAM files and their indexes into a single VCF output.

Config-driven workflows

I explored using a `config.yaml` file to store reference paths (e.g., genome FASTA, GTF), allowing the workflow to be more portable and easier to maintain compared to hard-coding filenames directly into rules.

Custom scripts and Python integration

I tested integrating custom Python scripts into Snakemake using both:

- the `script:` directive (accessing inputs and outputs via `snakemake.input` / `snakemake.output`)
- the `shell:` directive to call scripts that already use `argparse`

I noted that using `script:` avoids boilerplate argument parsing and cleanly separates **workflow logic** from **analysis logic**, while `shell:` execution is useful when reusing existing command-line scripts.

Target rules and workflow completion

Finally, I learned how to define **target rules** (e.g., rule `all`) that contain fully expanded filenames and no wildcards. These rules serve as logical workflow endpoints and allow Snakemake to run a complete analysis with a single command.

Key takeaways

- Dependencies are inferred purely from files
- Rules with wildcards cannot be directly targeted by name
- Target rules provide clean, reproducible workflow endpoints

- Snakemake scales naturally to multi-sample, multi-replicate experimental designs

Advanced_Snakemake_Tutorial

Thread management, configuration, and advanced rule features in Snakemake

I extended my understanding of Snakemake by learning how to control computational resources, externalize workflow configuration, and handle more complex input logic.

Thread specification and resource scheduling

I learned how to specify per-rule CPU usage using the threads directive and how this interacts with the global `--cores` argument at runtime. Each rule can declare how many threads a job requires, and Snakemake schedules jobs so that the total number of used threads does not exceed the available cores. This clarified how to efficiently parallelize workflows while preventing resource oversubscription, especially for multithreaded tools like bwa mem.

Using config files for workflow flexibility

I learned how to use a config.yaml file to make the workflow configurable and easier to adapt to new datasets. Snakemake loads the config file at startup and stores it in a global dictionary (config), allowing sample definitions and file paths to be modified without editing the Snakefile.

By moving sample definitions into the config file, I removed hard-coded sample lists from the workflow and used `expand()` with `config["samples"]` to dynamically generate input file lists for aggregation rules such as variant calling. This reinforced the distinction between workflow logic and dataset-specific information.

Input functions and execution phases

I learned that Snakemake workflows execute in distinct phases (initialization, DAG construction, scheduling) and that not all inputs can be resolved during initialization. When input files depend on wildcard values that are only known once jobs are instantiated, `expand()` is insufficient.

To address this, I implemented **input functions**, which defer input resolution until wildcard values are known during DAG construction. This allowed per-sample FASTQ paths to be retrieved dynamically from the config file, including paired-end reads. I documented when to use `expand()` versus input functions and why incorrect usage can cause conceptual or execution failures.

Rule parameters

I learned how to use the `params` directive to define arbitrary, non-file parameters for rules. This is useful for passing tool-specific options (e.g., read group information) into shell commands without encoding them into filenames or inputs. I confirmed that `params` values can be accessed consistently across shell commands, run blocks, and scripts.

Logging job output

I added structured logging to rules using the `log` directive so that `stderr` output from each job is written to a dedicated log file instead of the terminal. This is especially important for debugging when many jobs run in parallel. I followed best practices by organizing logs in a `logs/` directory and naming them by rule and sample.

Temporary and protected outputs

I learned how to manage disk usage by marking intermediate files with `temp()`, allowing Snakemake to automatically delete them once all downstream jobs have completed. Conversely, I used `protected()` to prevent critical final outputs from being accidentally deleted or overwritten. This clarified how

Snakemake enforces file lifecycle management based on workflow dependencies.

Key takeaways

- threads and --cores work together to control parallelism
- Config files separate data specification from workflow logic
- Input functions are required when inputs depend on wildcards
- params, log, temp(), and protected() improve robustness, debuggability, and storage efficiency
- Understanding Snakemake's execution phases is critical for designing correct workflows

007 02042026 Create Snakemake files

Tuesday, February 3, 2026 5:47 PM

Objective:

- Write code from snakemake Tutorials

NOTES

Website: <https://snakemake.readthedocs.io/en/stable/index.html>

#Snakemake basic tutorial

#This is a simple Snakemake workflow that demonstrates the basic structure and functionality of Snakemake.

#Structuring a code block

SAMPLES = ["A", "B"]

rule trim:

input:

"raw/{sample}.fastq.gz"

output:

"trimmed/{sample}.fastq.gz"

shell:

"fastp -i {input} -o {output}"

#Element of the rule in the shell command are in brace notation

rule align:

input:

"trimmed/{sample}.fastq.gz"

output:

"aligned/{sample}.bam"

shell:

"bwa mem ref.fa {input} > {output}"

#Use of distinct file names prefixes increases snakemake's efficiency

rule bwa_map:

input:

"data/genome.fa",

"data/samples/{sample}.fastq"

```

output:
"mapped_reads/{sample}.bam"
shell:
"mkdir -p mapped_reads && bwa mem {input} | samtools view -Sb > {output}"

```

```

rule samtools_sort:
input:
"mapped_reads/{sample}.bam"
output:
"sorted_reads/{sample}.bam"
shell:
"""
mkdir -p sorted_reads
samtools sort -T sorted_reads/{wildcards.sample} -O bam {input} > {output}
"""

```

#Indexing read alignments and visualizing the DAG of jobs

```

rule samtools_index:
input:
"sorted_reads/{sample}.bam"
output:
"sorted_reads/{sample}.bam.bai"
shell:
"samtools index {input}"

```

```

rule bcftools_call:
input:
fa="data/genome.fa",
bam=expand("sorted_reads/{sample}.bam", sample=SAMPLES),
bai=expand("sorted_reads/{sample}.bam.bai", sample=SAMPLES)
output:
"calls/all.vcf"
shell:
"""
mkdir -p calls
bcftools mpileup -f {input.fa} {input.bam} | bcftools call -mv - > {output}
"""

```

```

rule plot_qual:
input:
"calls/all.vcf"
output:
"plots/quals.svg"

```

```

log:
"logs/plot_qual.log"
script:
"""
    scripts/plot-quals.py
"""

#OUTPUTs

#Shell Output: snakemake -np mapped_reads/A.bam -s Smakemake_tutorial_code.smk

#Building DAG of jobs...
# Job stats:
# job      count
# -----
# bwa_map   1
# total     1

# [Wed Feb  4 12:13:26 2026]
# rule bwa_map:
#   input: data/genome.fa, data/samples/A.fastq
#   output: mapped_reads/A.bam
#   jobid: 0
#   reason: Missing output files: mapped_reads/A.bam
#   resources: tmpdir=<TBD>
# Shell command: mkdir -p mapped_reads && bwa mem data/genome.fa data/samples/A.fastq | samtools view -Sb > mapped_reads/A.bam
# Job stats:
# job      count
# -----
# bwa_map   1
# total     1

# Reasons:
#   (check individual jobs above for details)
#   output files have to be generated:
#     bwa_map
# This was a dry-run (flag -n). The order of jobs does not reflect the order of execution.

#Shell Output: snakemake -np sorted_reads/B.bam -s Smakemake_tutorial_code.smk

# host: gl3026.arc-ts.umich.edu

```

Building DAG of jobs...

Job stats:

# job	count
-------	-------

# -----	-----
---------	-------

# bwa_map	1
-----------	---

# samtools_sort	1
-----------------	---

# total	2
---------	---

[Wed Feb 4 14:03:14 2026]

rule bwa_map:

input: data/genome.fa, data/samples/B.fastq

output: mapped_reads/B.bam

jobid: 1

reason: Missing output files: mapped_reads/B.bam

wildcards: sample=B

resources: tmpdir=<TBD>

Shell command: mkdir -p mapped_reads && bwa mem data/genome.fa data/samples/B.fastq | samtools view -Sb > mapped_reads/B.bam

[Wed Feb 4 14:03:14 2026]

rule samtools_sort:

input: mapped_reads/B.bam

output: sorted_reads/B.bam

jobid: 0

reason: Missing output files: sorted_reads/B.bam; Input files updated by another job: mapped_reads/B.bam

wildcards: sample=B

resources: tmpdir=<TBD>

Shell command:

mkdir -p sorted_reads

samtools sort -T sorted_reads/B -O bam mapped_reads/B.bam > sorted_reads/B.bam

Job stats:

# job	count
-------	-------

# -----	-----
---------	-------

# bwa_map	1
-----------	---

# samtools_sort	1
-----------------	---

# total	2
---------	---

Reasons:

(check individual jobs above for details)

input files updated by another job:

samtools_sort

output files have to be generated:

bwa_map, samtools_sort

This was a dry-run (flag -n). The order of jobs does not reflect the order of execution.

#Shell command: snakemake sorted_reads/{A/B}.bam.bai --dag | dot -Tsvg > dag.svg

#Generated dag visual

#Shell Command: snakemake -np -s snakemake.smk plots/quals.svg

host: gl-login4.arc-ts.umich.edu

Building DAG of jobs...

Job stats:

# job	count
-------	-------

# -----	-----
---------	-------

# bcftools_call	1
-----------------	---

# plot_quals	1
--------------	---

# total	2
---------	---

[Wed Feb 4 15:25:42 2026]

rule bcftools_call:

input: data/genome.fa, sorted_reads/A.bam, sorted_reads/B.bam, sorted_reads/A.bam.bai, sorted_reads/B.bam.bai

output: calls/all.vcf

jobid: 1

reason: Missing output files: calls/all.vcf

resources: tmpdir=<TBD>

Shell command:

mkdir -p calls

bcftools mpileup -f data/genome.fa sorted_reads/A.bam sorted_reads/B.bam | bcftools call -mv - > calls/all.vcf"

[Wed Feb 4 15:25:42 2026]

rule plot_quals:

input: calls/all.vcf

output: plots/quals.svg

jobid: 0

reason: Missing output files: plots/quals.svg; Input files updated by another job: calls/all.vcf

resources: tmpdir=<TBD>

Shell command: None

Job stats:

# job	count
-------	-------

# -----	-----
---------	-------

# bcftools_call	1
-----------------	---

```

# plot_qual      1
# total          2

# Reasons:
#   (check individual jobs above for details)
#   input files updated by another job:
#   plot_qual
#   output files have to be generated:
#   bcftools_call, plot_qual
# This was a dry-run (flag -n). The order of jobs does not reflect the order of execution.

```

#Snakemake advanced tutorial

#Config file implementation

configfile: "config.yaml"

#This is a simple Snakemake workflow that demonstrates the basic structure and functionality of Snakemake.

rule all:

input:

"calls/all.vcf"

#Structuring a code block

SAMPLES = ["A", "B"]

rule trim:

input:

"raw/{sample}.fastq.gz"

output:

"trimmed/{sample}.fastq.gz"

shell:

```
"fastp -i {input} -o {output}"
```

#Element of the rule in the shell command are in brace notation

rule align:

input:

```
"trimmed/{sample}.fastq.gz"
```

output:

```
"aligned/{sample}.bam"
```

shell:

```
"bwa mem ref.fa {input} > {output}"
```

#Use of distinct file names prefixes increases snakemake's efficiency

```
def get_bwa_map_input_fastqs(wildcards):
```

```
    return config["samples"][wildcards.sample]
```

rule bwa_map:

input:

```
"data/genome.fa",
```

```
    get_bwa_map_input_fastqs
```

output:

```
    temp("mapped_reads/{sample}.bam")
```

params:

```
rg=r"@RG\tID:{sample}\tSM:{sample}"
```

log:

```
"logs/bwa_mem/{sample}.log"
```

threads: 8

```
shell: "mkdir -p mapped_reads && bwa mem -R '{params.rg}' -t {threads} {input} | samtools view -Sb > {output}"
```

rule samtools_sort:

input:

```
"mapped_reads/{sample}.bam"
```

output:

```
protected("sorted_reads/{sample}.bam")
```

shell:

```
"""
```

```
mkdir -p sorted_reads
```

```
samtools sort -T sorted_reads/{wildcards.sample} -O bam {input} > {output}
```

```
"""
```

#Indexing read alignments and visualizing the DAG of jobs

rule samtools_index:

input:

"sorted_reads/{sample}.bam"

output:

"sorted_reads/{sample}.bam.bai"

shell:

"samtools index {input}"

rule bcftools_call:

input:

fa="data/genome.fa",

bam=expand("sorted_reads/{sample}.bam", sample=config["samples"].keys()),

bai=expand("sorted_reads/{sample}.bam.bai", sample=config["samples"].keys())

params: rate=config["prior_mutation_rate"]

log:

"logs/bcftools_call/all.log"

output:

"calls/all.vcf"

shell:

"""

mkdir -p calls

bcftools mpileup -f {input.fa} {input.bam} | bcftools call -mv -P {params.rate} - > {output}

"""

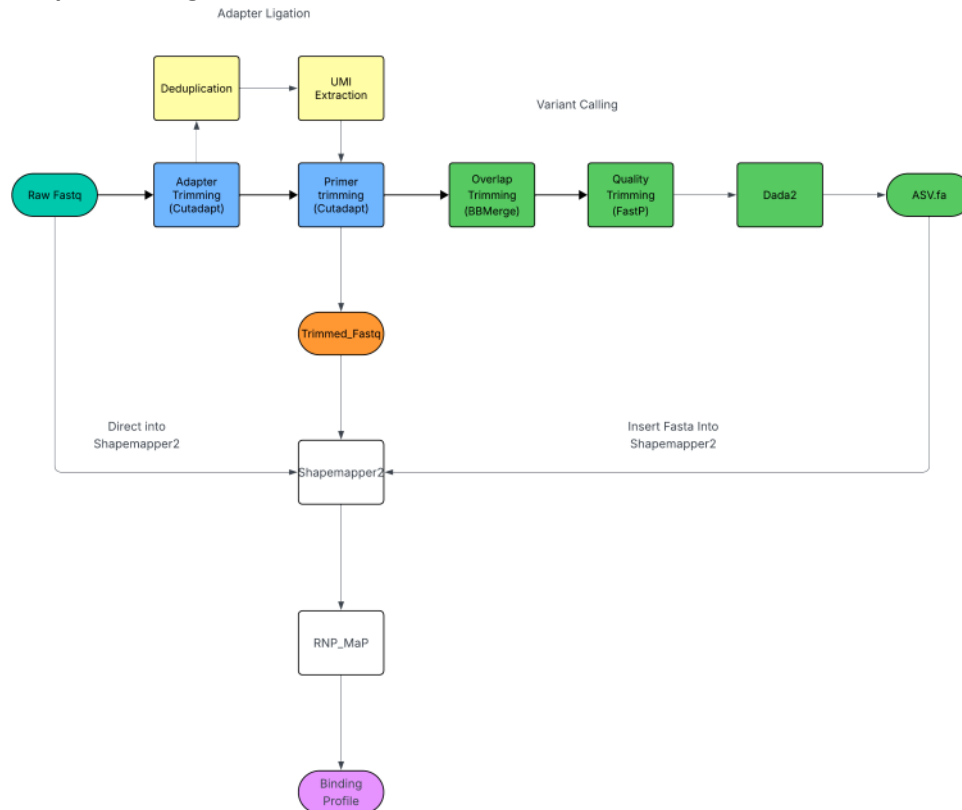
008 02052026 Designing Workflow

Friday, February 06, 2026 4:59 PM

Objective:

- Determine how to design a pipeline to remove perform the various adapter trimming, ASV detection, and umi extraction and detection necessary to run RNP_MaP and Shapemapper2

Graphical Design



009 02092026 Workflow Update and Pipeline version 1

Monday, February 09, 2026 10:57 PM

Objective:

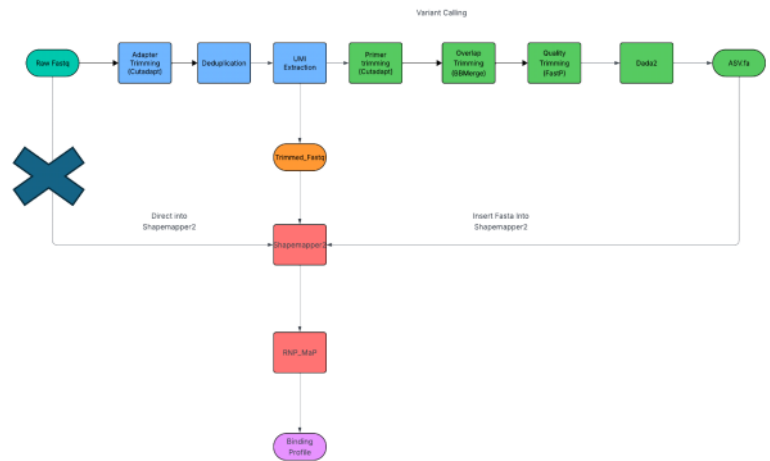
- Make first snakemake version
- Update workflow schematic

Future:

- Finish second part of pipeline
- Learn how to integrate BBMerge

Other Important Notes:

BBtools has a tool call BBmerge that will allow us to remove any overhang post primer trimming. The function [tbo=f] (trimbyoverlap) allows us to trim overlapping reads to remove rightmost (3') non-overlapping portion, instead of joining. The removes the necessity of physically removing 3' end bases after primer removal.



Snakemake Pipeline version 1 (Not complete)

#RNP_MaP Snakemake Pipeline

#Created February 2026

##author: wjarret and chat GPT 5.2

###Config. file

configfile: "config_RNP_MaP.yaml"

samples = config["samples"]

Index_adapters = config["Index_adapters"]

Cutadapt = config["Cutadapt"]

###Workflow

###Rules

rule copy_fastq:

input:

r1="{sample}_R1_001.fastq.gz",

r2="{sample}_R2_001.fastq.gz"

output:

r1="raw/{sample}_R1_001.fastq.gz",

r2="raw/{sample}_R2_001.fastq.gz"

shell:

"""

mkdir -p raw

cp {input.r1} {output.r1}

cp {input.r2} {output.r2}

"""

#Adapter Trimming cutadapt (Index Primer sequence and transposase)

rule cutadapt_step1:

input:

r1="raw/{sample}_R1_001.fastq.gz",

r2="raw/{sample}_R2_001.fastq.gz"

output:

r1="index_adapter_trimming/step1_{sample}_R1_001.fastq.gz",

r2="index_adapter_trimming/step1_{sample}_R2_001.fastq.gz"

threads: 8

log: "logs/cutadapt_step1/step1_{sample}.log"

shell:

"""

mkdir -p Index_adapters_trimming

mkdir -logs

cutadapt \

-j {threads} \

-a [file:{Index_adapters\[Non_Transposase\]}](#) \

```
-A file:{Index\_adapters\[Non\_Transposase\]} \
-e {Cutadapt[error_rate]} \
--report=minimal \
-o {output.r1} -p {output.r2} \
{input.r1} {input.r2} > {log} 2>&1
====
```

rule cutadapt_step2:

```
input:
    r1="index_adapter_trimming/step1_{sample}_R1_001.fastq.gz",
    r2="index_adapter_trimming/step1_{sample}_R2_001.fastq.gz"
output:
    r1="index_adapter_trimming/step2_{sample}_R1_001.fastq.gz",
    r2="index_adapter_trimming/step2_{sample}_R2_001.fastq.gz"
threads: 8
log="logs/cutadapt_step2/step2_{sample}.log"
shell:
    ====

    mkdir -p Index_adapters_trimming
    cutadapt \
        -j {threads} \
        -a file:{Index\_adapters\[Transposase\]} \
        -A file:{Index\_adapters\[Transposase\]} \
        -e {Cutadapt[error_rate]} \
        --report=minimal \
        -o {output.r1} -p {output.r2} \
        {input.r1} {input.r2} > {log} 2>&1
    =====
```

#UMI extraction ** Might be worthwhile to add the ability for the user to directly input UMI length ** (fastp):

rule umi_extraction:

```
input:
    r1="index_adapter_trimming/step2_{sample}_R1_001.fastq.gz",
    r2="index_adapter_trimming/step2_{sample}_R2_001.fastq.gz"
output:
    r1="umi_removed/{sample}_R1.fastq.gz",
    r2="umi_removed/{sample}_R2.fastq.gz"
threads: 2
log="logs/fastp_umi/{sample}.log"
shell:
    ====

    fastp \
        --thread {threads} \
        --in1 {input.r1} \
        --in2 {input.r2} \
        --out1 {output.r1} \
        --out2 {output.r2} \
        --umi \
        --umi_loc per_read \
        --umi_len {config["fastp"]["umi_len"]} \
        --umi_prefix UMI \
        -j umi_removed/{wildcards.sample}.json \
        -h umi_removed/{wildcards.sample}.html \
        > {log} 2>&1
    =====
```

After this primer trimming, samples can either be moved directly into the variant calling or directly into shapemapper2

Variant calling (only DMSO samples)

#Trimmed Sample output moves to primer trimming to remove primer sequeunces error prone nucleotides, and UMIs (Cutadapt)

rule cutadapt_primer_removal:

```
input:
    r1="index_adapter_trimming/step1_{sample}_R1_001.fastq.gz",
    r2="index_adapter_trimming/step1_{sample}_R2_001.fastq.gz"
output:
    r1="cutadapt_primer_removal/primer_removal_{sample}_R1_001.fastq.gz",
```

```

r2="cutadapt_primer_removal/primer_removal_{sample}_R2_001.fastq.gz"
threads: 4
log="logs/cutadapt_step2/step2_{sample}.log"
shell:
    ""
    mkdir -p Index_adapters_trimming
    cutadapt \
        -j {threads} \
        -u {Cutadapt[R1_primer_Length]} \
        -U {Cutadapt[R2_primer_Length]} \
        --report=minimal \
        -o {output.r1} -p {output.r2} \
        {input.r1} {input.r2} > {log} 2>&1
    ""

#BB_merge removes overhang by performing a direct overlap on primer trimmed samples (BB_merge)
#Quality Trimming (fastp)
# Run through Divisive Amplicon Denoising Algorithm 2 to obtain ASV Fasta (Dada2)
### Following ASV.fa generation, this file will be input into shapemapper2 with the Trimmed_fastq's
#shapemapper2
#RNP_MaP
#Binding Profile generation

```

- Objective:**
- Continue working on snakemake pipeline
 - Update workflow schematic
 - Integrate R studio code to be able to integrate with the pipeline
- Future:**
- Determine how to create an input function so snakemake can determine the sample path during the DAG phase.
 - Create current DAG layout
 - Integrate DMSO selectivity, and sample insert into shapemapper2 and RNP_MaP into pipeline. Integrate shapemapper2 and RNP_MaP as well.
- Other Important Notes:**
- In the R code, the library `optparse`, an R equivalent to `argparse`, was used to make the code compatible to the snakemake file. Arguments are used to input the specified files indicated in the Snakemake file. Additional code has also been added to construct a fasta file inside the R script, removing the necessity to create one in a new rule. ASV in the fasta file received an ASV name followed by the number in the sequence. This will ease the burden of individually naming each line manually. The only downside is that each ASV has an arbitrary name. Biostrings was used to perform these tasks. Next, the reverse complement of each sequence was taken, allowing for direct input into shapemapper2. The fasta file is output into the rule folder. The only problem that goes unsolved is the manual creation of the primer fasta file.
 - BB_merge, Fastp quality trimming, and dada2 have been input into the snakemake file.

Updated UNIX Code:	Updated R Studio Code:
<pre>#RNP_MaP Snakemake Pipeline #Created Febuary 2026 #@author: wjarret and chat GPT 5.2 ##Config. file configfile: "config_RNP_MaP.yaml" fastp = config["fastp"] samples = config["samples"] Index_adapters = config["Index_adapters"] Cutadapt = config["Cutadapt"] seqkit = config["seqkit"] ###Workflow #Define samples ###Rules rule all: input: expand("primer_reinsertion/primer_reinsertion_ASV_{sample}.fa", sample=config["samples"]) rule copy_fastq: input: r1="/home/wjarret/scratchjw/RNP_MaP_Snakemake/data/samples/{sample}_R1_001.fastq.gz", r2="/home/wjarret/scratchjw/RNP_MaP_Snakemake/data/samples/{sample}_R2_001.fastq.gz" output: r1="raw/{sample}_R1_001.fastq.gz", r2="raw/{sample}_R2_001.fastq.gz" shell: """ mkdir -p raw cp {input.r1} {output.r1} cp {input.r2} {output.r2} """ #Adapter Trimming cutadapt (Index Primer sequence and transposase) rule cutadapt_step1: input: r1="raw/{sample}_R1_001.fastq.gz", r2="raw/{sample}_R2_001.fastq.gz" output: r1="index_adapter_trimming/step1_{sample}_R1_001.fastq.gz", r2="index_adapter_trimming/step1_{sample}_R2_001.fastq.gz" threads: 8 params: adapter_NT = Index_adapters["Non_Transposase"], error = Cutadapt["error_rate"] log: "logs/cutadapt_step1/step1_{sample}.log" shell: """ mkdir -p Index_adapters_trimming mkdir -logs cutadapt \ -j {threads} \ -a file:{params.adapter_NT} \ -A file:{params.adapter_NT} \ -e {params.error} \ --report-minimal \ -o {output.r1} -p {output.r2} \ {input.r1} {input.r2} > {log} 2>&1 """ rule cutadapt_step2: input: r1="index_adapter_trimming/step1_{sample}_R1_001.fastq.gz", r2="index_adapter_trimming/step1_{sample}_R2_001.fastq.gz" output: r1="index_adapter_trimming/step2_{sample}_R1_001.fastq.gz", r2="index_adapter_trimming/step2_{sample}_R2_001.fastq.gz" threads: 8 params: adapter_NT = Index_adapters["Non_Transposase"], error = Cutadapt["error_rate"] log: "logs/cutadapt_step2/step2_{sample}.log" shell: """ mkdir -p Index_adapters_trimming mkdir -logs cutadapt \ -j {threads} \ -a file:{params.adapter_NT} \ -A file:{params.adapter_NT} \ -e {params.error} \ --report-minimal \ -o {output.r1} -p {output.r2} \ {input.r1} {input.r2} > {log} 2>&1 """</pre>	<pre>option_list <- list(make_option(c("--r1"), type="character", help="Forward FASTQ file"), make_option(c("--r2"), type="character", help="Reverse FASTQ file"), make_option(c("--out"), type="character", help="Output FASTA file")) opt <- parse_args(OptionParser(option_list=option_list)) r1_file <- opt\$r1 r2_file <- opt\$r2 output_file <- opt\$out filt_dir <- file.path(tempdir(), "filtered") dir.create(filt_dir, showWarnings = FALSE) # For a single sample, just define filtered file names directly filtF <- file.path(filt_dir, "F_filt.fastq.gz") filtR <- file.path(filt_dir, "R_filt.fastq.gz") print(filtF) print(filtR) #Filtered dir should still be empty right before filtering. This is because dada2 is operating in append mode. This has led to a lot of issues #and hopefully it runs through smoothly # # cat("\nContents of filtered immediately before filterAndTrim\n") # print(list.files(filt_dir, full.names = TRUE)) # Filter and trim (removes reads with N because maxN=0) #out <- filterAndTrim(fnFs, filtFs, fnRs, filtRs, maxN = 0, compress = TRUE, multithread = TRUE) out <- filterAndTrim(r1_file, filtF, r2_file, filtR, maxN = 0, compress = TRUE, multithread = TRUE) print(out) #Error learning algorithm (Will take some times so be sure to activate multithread and add at least 8 cpus-per-node) errF <- learnErrors(filtF, multithread = TRUE) errR <- learnErrors(filtR, multithread = TRUE) # get folder where FASTA will be written out_dir <- dirname(output_file) # On a cluster (no display), write plot to PDF instead of screen pdf(file = file.path(out_dir, "U1_error_plot_F.pdf")) plotErrors(errF, nominalQ = TRUE) dev.off() pdf(file = file.path(out_dir, "U1_error_plot_R.pdf")) plotErrors(errR, nominalQ = TRUE) dev.off() # Denoise sequences dadaFs <- dada(filtF, err = errF, multithread = TRUE) dadaRs <- dada(filtR, err = errR, multithread = TRUE) # Merge paired reads mergers <- mergePairs(dadaFs, filtF, dadaRs, filtR, verbose = TRUE) # Construct sequence table seqtab <- makeSequenceTable(mergers) print(dim(seqtab)) # Inspect distribution of sequence lengths len_tab <- table(nchar(getSequences(seqtab))) print(len_tab) # seqtab: samples as rows, ASVs as columns asv_seqs <- colnames(seqtab) row_lengths <- nchar(getSequences(seqtab))</pre>

```

r1="index_adapter_trimming/step1_{sample}_R1_001.fastq.gz",
r2="index_adapter_trimming/step1_{sample}_R2_001.fastq.gz"
output:
r1="index_adapter_trimming/step2_{sample}_R1_001.fastq.gz",
r2="index_adapter_trimming/step2_{sample}_R2_001.fastq.gz"
threads: 8
params:
  adapter_T = Index_adapters["Transposase"],
  error = Cutadapt["error_rate"]
log: "logs/cutadapt_step2/{sample}.log"
shell:
  ""
  mkdir -p index_adapters_trimming
  cutadapt \
    -j {threads} \
    -a file:{params.adapter_T} \
    -A file:{params.adapter_T} \
    -e {params.error} \
    --report=minimal \
    -o {output.r1} -p {output.r2} \
    {input.r1} {input.r2} > {log} 2>&1
  ""

#UMI extraction ** Might be worthwhile to add the ability for the user to directly input UMI length **
(fastp):
rule umi_extraction:
  input:
    r1="index_adapter_trimming/step2_{sample}_R1_001.fastq.gz",
    r2="index_adapter_trimming/step2_{sample}_R2_001.fastq.gz"
  output:
    r1="umi_removed/{sample}_R1.fastq.gz",
    r2="umi_removed/{sample}_R2.fastq.gz"
  threads: 2
  params:
    umi_len = fastp["umi_len"]
  log: "logs/fastp_umi/{sample}.log"
  shell:
    ""
    fastp \
      --thread {threads} \
      --in1 {input.r1} \
      --in2 {input.r2} \
      --out1 {output.r1} \
      --out2 {output.r2} \
      --umi \
      --umi_loc per_read \
      --umi_len {params.umi_len} \
      --umi_prefix UMI \
      -j umi_removed/{wildcards.sample}.json \
      -h umi_removed/{wildcards.sample}.html \
      > {log} 2>&1
    ""

### After this primer trimming, samples can either be moved directly into the variant calling or directly
into shapemapper2

### Variant calling (only DMSO samples)

#Trimmed Sample output moves to primer trimming to remove primer sequences error prone
nucleotides, and UMIs (Cutadapt)
rule cutadapt_primer_removal:
  input:
    r1="index_adapter_trimming/step1_{sample}_R1_001.fastq.gz",
    r2="index_adapter_trimming/step1_{sample}_R2_001.fastq.gz"
  output:
    r1="cutadapt_primer_removal/primer_removal_{sample}_R1_001.fastq.gz",
    r2="cutadapt_primer_removal/primer_removal_{sample}_R2_001.fastq.gz"
  threads: 4
  params:
    primer_1 = Cutadapt["R1_primer_Length"],
    primer_2 = Cutadapt["R2_primer_Length"]
  log: "logs/cutadapt_step2/step2_{sample}.log"
  shell:
    ""
    mkdir -p index_adapters_trimming
    cutadapt \
      -j {threads} \
      -u {params.primer_1} \
      -U {params.primer_2} \
      --report=minimal \
      -o {output.r1} -p {output.r2} \
      {input.r1} {input.r2} > {log} 2>&1
    ""

#BB_merge removes overhang by performing a direct overlap on primer trimmed samples (BB_merge)
rule BB_Merge:
  input:
    r1="index_adapter_trimming/step1_{sample}_R1_001.fastq.gz",
    r2="index_adapter_trimming/step1_{sample}_R2_001.fastq.gz"
  output:
    r1="BB_Merge/BB_Merge{sample}_R1_001.fastq.gz",
    r2="BB_Merge/BB_Merge{sample}_R2_001.fastq.gz"
  log: "logs/BB_Merge/BBMerge_{sample}.log"
  shell:
    ""
    bbmerge.sh \
      in1={input.r1} \
      in2={input.r2} \
      out1={output.r1} \
      out2={output.r2} \
      tbo=f
    ""

#Quality Trimming (fastp)
rule Quality_trimming_fastp:
  input:
    r1="BB_Merge/BB_Merge{sample}_R1_001.fastq.gz",
    r2="BB_Merge/BB_Merge{sample}_R2_001.fastq.gz"
  output:
    r1="Quality_trimming_fastp/Quality_trimming_fastp_{sample}_F_R1_001.fastq.gz",
    r2="Quality_trimming_fastp/Quality_trimming_fastp_{sample}_R_R2_001.fastq.gz"
  log: "logs/Quality_trimming_fastp/Quality_trimming_fastp_{sample}.log"
  threads: 4
  params:
    quality_score = fastp["quality_score_threshold"]
  shell:
    ""
    fastp \

```

```

len_tab <- table(nchar(gsub("N", "", DNA_sequences)))
print(len_tab)

```

```

# seqtab: samples as rows, ASVs as columns
asv_seqs <- colnames(seqtab)
asv_lengths <- nchar(asv_seqs)

```

```

# Write to Excel (saved in the working dir where the job runs)
out_xlsx <- file.path(out_dir, "sequence_table_with_sequences_U1.xlsx")
write.xlsx(mergers, out_xlsx, rowNames = FALSE)
cat("Wrote Excel file to:", out_xlsx, "\n")

```

```

#Construct Fasta file
sequences <- mergers$sequence

```

```

#Make sequences column identifiable as DNA sequences
#Biostrings allows us to do that

```

```

DNA_sequences<- DNABStringSet(sequences)

```

```

#Names each ASV
names(DNA_sequences) <- paste0('ASV', 1:length(DNA_sequences))
DNA_sequences <- reverseComplement(DNA_sequences)
print(DNA_sequences)

```

```

#Outputs fasta to directory
writeXStringSet(x = DNA_sequences, filepath = output_file)

```

```

-i {input.r1} \
-l {input.r2} \
-o {output.r1} \
-O {output.r2} \
--correction \
-q {quality_score} \
--n_base_limit 0 \
--thread {threads}
"""

# Run through Divisive Amplicon Denoising Algorithm 2 to obtain ASV Fasta (Dada2)
#In order for this to work, the Inputs need to be defines
rule dada2:
    input:
        r1="Quality_trimming_fastp/Quality_trimming_fastp_{sample}_R1_001.fastq.gz",
        r2="Quality_trimming_fastp/Quality_trimming_fastp_{sample}_R2_001.fastq.gz"
    output:
        fa="dada2/ASV_{sample}.fa"
    log:
        "logs/dada2/dada2_{sample}.log"
    threads: 4
    shell:
        """
        Rscript scripts/dada2_snakemake_02102026.R \
        --r1 {input.r1} \
        --r2 {input.r2} \
        --out {output.fa} > {log} 2>&1
        """

rule primer_reinsertion:
    input:
        "dada2/ASV_{sample}.fa"
    output:
        "primer_reinsertion_ASV_{sample}.fa"
    log:
        "logs/primer_reinsertion/primer_reinsertion_{sample}.log"
    params:
        adapter_f = seqkit["adapter_forward"],
        adapter_r = seqkit["adapter_reverse"]
    shell:
        """
        seqkit mutate --insertion 0:{adapter_f},-1:{adapter_r} input.fasta -o output.fasta
        """

### Following ASV.fa generation, this file will be input into shapemapper2 with the Trimmed_fastq's

#shapemapper2

#RNP_MaP

#Binding Profile generation

```

011 02(14-21)2026 Workflow Update and Pipeline version 3

Monday, February 23, 2026 10:06 AM

Objective:

- Finalize snakemake pipeline

Future:

- Run on snRNA samples

Other Important Notes:

- The final pipeline yielded good results on final testing. Issues with running was caused by the lack of envs folder containing all the environments that the script runs. Also, the formatting of the sbatch file must have the following in order to run.

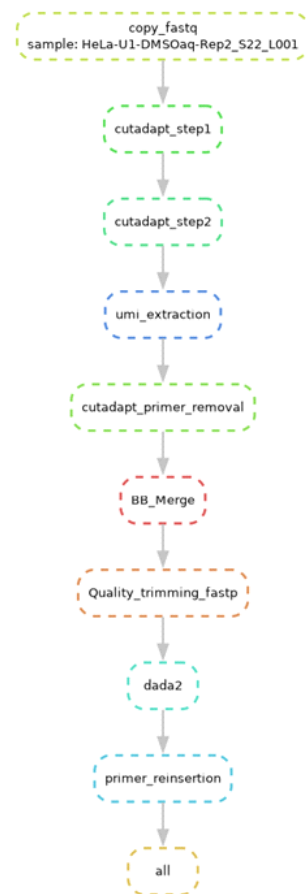
```
# Ensure micromamba is available
export MAMBA_ROOT_PREFIX=$HOME/micromamba
export PATH="$MAMBA_ROOT_PREFIX/bin:$PATH"
# Initialize micromamba in non-interactive bash
eval "$(micromamba shell hook --shell=bash)"
# Activate the prebuilt RNP-MaP environment
micromamba activate snakemake_rnp_map
```

From <https://greatlakes.arc-ts.umich.edu/pun/sys/dashboard/files/fs//home/wjarret/scratchiw/01292026_snRNA_RNP_MaP_adptr_Lig/U1_validation/snakemake.sbatch>

- When running, latency also became an issue, this required the '--latency-wait' flag in order to prevent the next rule from running prior to the input directory being ran.
- DMSOaq sample isolation has also been implemented in the code, allowing for parallel processing of all samples up to umi_extraction. Dada2 only requires DMSO, non-crosslinked, samples containing total RNA (aqueous). The following code has been added to first create a for loop that identifies all DMSOaq samples, printing out the total number of DMSOaq samples, followed by all samples, and then naming each sample.

```
(samples,) = glob_wildcards("data/samples/{sample}_R1_001.fastq.gz")
DMSO_SAMPLES = [s for s in samples if "DMSOaq" in s]
print(f"Total samples: {len(samples)}")
print(f"All samples: {samples}")
print(f"DMSO samples for variant calling: {len(DMSO_SAMPLES)}")
print(f"DMSO samples: {DMSO_SAMPLES}")
```

- It is important to note that shapemapper2 and RNP-MaP have yet to be added.



Most recent Version

```
#RNP_MaP Snakemake Pipeline
#Created February 2026
#@author: wjarret and chat GPT 5.2
###Config. file
configfile: "config_RNP_MaP.yaml"
fastp = config["fastp"]
samples = config["samples"]
Index_adapters = config["Index_adapters"]
Cutadapt = config["Cutadapt"]
```



```

seqkit = config["seqkit"]
###Workflow
#Define samples
###Rules
#function matches the given file pattern against the ones in the file system or directory and then infers the values for all
wildcards in the pattern
#A named tuple for each wildcard is returned. samples is a tuple that has only one item that is a list of values for each
wildcard returned.
(samples,) = glob_wildcards("data/samples/{sample}_R1_001.fastq.gz")
DMSO_SAMPLES = [s for s in samples if "DMSOaq" in s]
print(f"Total samples: {len(samples)}")
print(f"All samples: {samples}")
print(f"DMSO samples for variant calling: {len(DMSO_SAMPLES)}")
print(f"DMSO samples: {DMSO_SAMPLES}")
rule all:
    input:
        # Only DMSO samples go through full pipeline to primer_reinsertion
        expand("primer_reinsertion/primer_reinsertion_ASV_{sample}.fa", sample=DMSO_SAMPLES)
rule copy_fastq:
    input:
        r1="data/samples/{sample}_R1_001.fastq.gz",
        r2="data/samples/{sample}_R2_001.fastq.gz"
    output:
        r1="raw/{sample}_R1_001.fastq.gz",
        r2="raw/{sample}_R2_001.fastq.gz"
    shell:
        """
        mkdir -p raw
        mkdir -p logs
        cp {input.r1} {output.r1}
        cp {input.r2} {output.r2}
        """
#Adapter Trimming cutadapt (Index Primer sequence and transposase)
rule cutadapt_step1:
    input:
        r1="raw/{sample}_R1_001.fastq.gz",
        r2="raw/{sample}_R2_001.fastq.gz"
    output:
        r1="index_adapter_trimming/step1_{sample}_R1_001.fastq.gz",
        r2="index_adapter_trimming/step1_{sample}_R2_001.fastq.gz"
    threads: 8
    params:
        adapter_NT = Index_adapters["Non_Transposase"],
        error = Cutadapt["error_rate"]
    log: "logs/cutadapt_step1/step1_{sample}.log"
    conda: "envs/environment.yaml"
    shell:
        """
        mkdir -p index_adapter_trimming
        cutadapt \
        -j {threads} \
        -a file:{params.adapter\_NT} \
        -A file:{params.adapter\_NT} \
        -e {params.error} \
        --report=minimal \
        -o {output.r1} -p {output.r2} \
        {input.r1} {input.r2} > {log} 2>&1
        """

```

```

"""
rule cutadapt_step2:
    input:
        r1="index_adapter_trimming/step1_{sample}_R1_001.fastq.gz",
        r2="index_adapter_trimming/step1_{sample}_R2_001.fastq.gz"
    output:
        r1="index_adapter_trimming/step2_{sample}_R1_001.fastq.gz",
        r2="index_adapter_trimming/step2_{sample}_R2_001.fastq.gz"
    threads: 8
    conda: "envs/environment.yaml"
    params:
        adapter_T = Index_adapters["Transposase"],
        error = Cutadapt["error_rate"]
    log: "logs/cutadapt_step2/step2_{sample}.log"
    shell:
        """
        mkdir -p Index_adapter_trimming
        cutadapt \
            -j {threads} \
            -a file:{params.adapter\_T} \
            -A file:{params.adapter\_T} \
            -e {params.error} \
            --report=minimal \
            -o {output.r1} -p {output.r2} \
            {input.r1} {input.r2} > {log} 2>&1
        """

```

#UMI extraction

```

rule umi_extraction:
    input:
        r1="index_adapter_trimming/step2_{sample}_R1_001.fastq.gz",
        r2="index_adapter_trimming/step2_{sample}_R2_001.fastq.gz"
    output:
        r1="umi_removed/{sample}_R1_001.fastq.gz",
        r2="umi_removed/{sample}_R2_001.fastq.gz"
    threads: 2
    conda: "envs/environment.yaml"
    params:
        umi_len = fastp["umi_len"]
    log: "logs/fastp_umi/{sample}.log"
    shell:
        """
        fastp \
            --thread {threads} \
            --in1 {input.r1} \
            --in2 {input.r2} \
            --out1 {output.r1} \
            --out2 {output.r2} \
            --umi \
            --umi_loc per_read \
            --umi_len {params.umi_len} \
            --umi_prefix UMI \
            -j umi_removed/{wildcards.sample}.json \
            -h umi_removed/{wildcards.sample}.html \
            > {log} 2>&1
        """

```

After this primer trimming, samples can either be moved directly into the variant calling or directly into shapemapper2

Variant calling (only DMSO samples)

rule organize_DMSO:

input:

r1="umi_removed/{sample}_R1_001.fastq.gz",

r2="umi_removed/{sample}_R2_001.fastq.gz"

output:

r1="conditioned/DMSOaq/{sample}_R1_001.fastq.gz",

r2="conditioned/DMSOaq/{sample}_R2_001.fastq.gz"

shell:

"""

mkdir -p conditioned/DMSOaq

ln -sf \$(realpath {input.r1}) {output.r1}

ln -sf \$(realpath {input.r2}) {output.r2}

"""

#Trimmed Sample output moves to primer trimming to remove primer sequences error prone nucleotides, and UMIs (Cutadapt)

rule cutadapt_primer_removal:

input:

r1="conditioned/DMSOaq/{sample}_R1_001.fastq.gz",

r2="conditioned/DMSOaq/{sample}_R2_001.fastq.gz"

output:

r1="cutadapt_primer_removal/primer_removal_{sample}_R1_001.fastq.gz",

r2="cutadapt_primer_removal/primer_removal_{sample}_R2_001.fastq.gz"

threads: 4

conda: "envs/environment.yaml"

params:

primer_1 = Cutadapt["R1_primer_Length"],

primer_2 = Cutadapt["R2_primer_Length"],

min_length = Cutadapt["min_length"],

max_length = Cutadapt["max_length"]

log: "logs/cutadapt_primer_removal/primer_removal_{sample}.log"

shell:

"""

mkdir -p Index_adapters_trimming

cutadapt \

-j {threads} \

-u {params.primer_1} \

-U {params.primer_2} \

--minimum-length {params.min_length} \

--maximum-length {params.max_length} \

--report=minimal \

-o {output.r1} -p {output.r2} \

{input.r1} {input.r2} > {log} 2>&1

"""

#BB_merge removes overhang by performing a direct overlap on primer trimmed samples (BB_merge)

rule BB_Merge:

input:

r1="cutadapt_primer_removal/primer_removal_{sample}_R1_001.fastq.gz",

r2="cutadapt_primer_removal/primer_removal_{sample}_R2_001.fastq.gz"

output:

r1="BB_Merge/BB_Merge_{sample}_R1_001.fastq.gz",

r2="BB_Merge/BB_Merge_{sample}_R2_001.fastq.gz"

log: "logs/BB_Merge/BBMerge_{sample}.log"

shell:

"""

mkdir -p BB_Merge

/home/wjarret/micromamba/bbmap/bbmerge.sh \

```

        in1={input.r1} \
        in2={input.r2} \
        out1={output.r1} \
        out2={output.r2} \
        tbo=t \
        merge=false > log 2>&1
    """

#Quality Trimming (fastp)
rule Quality_trimming_fastp:
    input:
        r1="BB_Merge/BB_Merge_{sample}_R1_001.fastq.gz",
        r2="BB_Merge/BB_Merge_{sample}_R2_001.fastq.gz"
    output:
        r1="Quality_trimming_fastp/Quality_trimming_fastp_{sample}_F_R1_001.fastq.gz",
        r2="Quality_trimming_fastp/Quality_trimming_fastp_{sample}_R_R2_001.fastq.gz"
    log: "logs/Quality_trimming_fastp/Quality_trimming_fastp_{sample}.log"
    threads: 4
    conda: "envs/environment.yaml"
    params:
        quality_score = fastp["quality_score_threshold"]
    shell:
        """
        mkdir -p Quality_trimming_fastp
        fastp \
        -i {input.r1} \
        -I {input.r2} \
        -o {output.r1} \
        -O {output.r2} \
        --detect_adapter_for_pe \
        --correction \
        -q {params.quality_score} \
        --n_base_limit 0 \
        --thread {threads}
        """

# Run through Divisive Amplicon Denoising Algorithm 2 to obtain ASV Fasta (Dada2)
#In order for this to work, the Inputs need to be defines
rule dada2:
    input:
        r1="Quality_trimming_fastp/Quality_trimming_fastp_{sample}_F_R1_001.fastq.gz",
        r2="Quality_trimming_fastp/Quality_trimming_fastp_{sample}_R_R2_001.fastq.gz"
    output:
        fa="dada2/ASV_{sample}.fa"
    log:
        "logs/dada2/dada2_{sample}.log"
    threads: 4
    conda: "envs/environment.yaml"
    shell:
        """
        mkdir -p dada2
        Rscript scripts/dada2_snakemake.R \
        --r1 {input.r1} \
        --r2 {input.r2} \
        --out {output.fa} > {log} 2>&1
        """

rule primer_reinsertion:
    input:
        "dada2/ASV_{sample}.fa"

```

```

output:
  "primer_reinsertion/primer_reinsertion_ASV_{sample}.fa"
log:
  "logs/primer_reinsertion/primer_reinsertion_{sample}.log"
params:
  adapter_f = seqkit["adapter_forward"]
conda: "envs/environment.yaml"
shell:
  ""

  mkdir -p primer_reinsertion
  seqkit mutate --insertion 0:{params.adapter_f} {input} -o {output}
  ""

### Following ASV.fa generation, this file will be input into shapemapper2 with the Trimmed_fastq's
# rule generating_folderz:
#   input:
#     r1="umi_removed/{sample}_R1_001.fastq.gz",
#     r2="umi_removed/{sample}_R2_001.fastq.gz"
#   output:
#   log:
#   shell:
# #shapemapper2
# rule shapemapper2:
#   input:
#     r1="umi_removed/{sample}_R1_001.fastq.gz",
#     r2="umi_removed/{sample}_R2_001.fastq.gz"
#   output:
#     {sample}.map
#     {sample}.shape
#     {sample}_histograms.pdf
#     {sample}_mapped_depths.pdf
#     {sample}_per-amplicon_abundance.txt
#     {sample}_profile.txt
#     {sample}_profiles.pdf
#     {sample}_ribosketch_colors.txt
#     {sample}_varna_colors.txt
#     {sample}_mutation_counts.txt
#     {sample}_parsed.mut
#     {sample}_mutation_counts.txt
#     {sample}_parsed.mut
#   log: "logs/shapemapper2/{sample}.log"
#   shell:
#     ""
#     mkdir -p BB_Merge
#     /home/wjarret/micromamba/bbmap/bbmerge.sh \
#       in1={input.r1} \
#       in2={input.r2} \
#       out1={output.r1} \
#       out2={output.r2} \
#       tbo=f \
#       merge=false > log 2>&1
#     ""
#RNP_MaP
#Binding Profile generation

```

From <https://greatlakes.arc-ts.umich.edu/pun/sys/dashboard/files/fs//home/wjarret/scratchjw/01292026_snRNA_RNP_MaP_adptr_Lig/U1/RNP_MaP.smk>

012 02232026 Implementing pipeline on U1

Monday, February 23, 2026 10:10 AM

- Objective:
- Run on all snRNA samples
 - Validate with previously analyzed U1 data
- Future:
- Finish running the remainder of snRNAs

Important Notes:

Primer reinsertion step's output is missing the forward primer region sequence. A two-step pipe approach lead to the fixing of this problem, indicating that inserting simultaneously in a single command caused the absence of insertion.

```
"""" mkdir -p primer_reinsertion seqkit
mutate --insertion -0:{params.adapter_f}
{input} | \ seqkit mutate --
insertion -1:{params.adapter_r} - \ -o {output}
2> {log} """"
```

- Snakemake pipeline provided expected U1 output from previous data. This result validates that the pipeline is consistent with manually ran tests.
- After running output, SDA samples did not run through the U1 pipeline. To correct this, an additional function was added to separate out OOPSint samples as well as SDA. Then in a rule all function, the final output was called requesting the output of umi_removal SDA samples as it will not be input into the dada2 analysis portion of the pipeline



U1 snRNA Amplicon Sequencing Data Analysis

Date	02/23/26
Cell line / Target	HeLa U1 snRNA (Sm binding region)
Conditions	DMSOaq (S2), OOPSint (S1)
Purpose	Observe the binding profiles of each snRNA at the Sm binding region, analyzing profile structure, variation, and correlation between conditions.
Protocol	1. Input sequencing data into Snakemake pipeline 2. Analyze rule outputs 3. Process through ShapeMapper2 4. Process through RNP-MaP 5. Analyze using PCA a. Variation analysis b. Correlation analysis between profiles

Results

Configuration File Parameters

Samples	
U1_F_DMSO_aq	data/samples/HeLa-U1-DMSOaq_S2_L001_R1_001.fastq.gz
U1_R_DMSO_aq	data/samples/HeLa-U1-DMSOaq_S2_L001_R2_001.fastq.gz
U1_F_OOPS_int	data/samples/HeLa-U1-OOPSint_S1_L001_R1_001.fastq.gz
U1_R_OOPS_int	data/samples/HeLa-U1-OOPSint_S1_L001_R2_001.fastq.gz
Index Adapters	
Non_Transposase	data/ARL_NEBNext_adapters_No_transposease.fa
Transposase	data/ARL_NEBNext_adapters_transposease.fa
Cutadapt	
error_rate	0.2
R1_primer_length	0
R2_primer_length	15
min_length	100 bp
max_length	180 bp
fastp	
umi_len	7
quality_score_threshold	20
seqkit	
adapter_forward	ACTTACCTGGCAGGG
adapter_reverse	(not specified)

DMSOaq (S2)

Cutadapt Adapter Trimming — Transposase Sequences

Run Information	
Run status	WARN
Input	
Total input reads	602,393
Total input bases (bp)	~1.80 × 10 ⁸
Read Filtering	
Reads too short	0
Reads too long	0
Reads with too many N	0
Output reads (passed)	602,393
Pass rate (%)	100.00%
R1 Output	

Reads with adapters (R1)	1,069 (0.18%)
Quality-trimmed bases R1 (bp)	0
Output bases R1 (bp)	90,599,015
R2 Output	
Reads with adapters (R2)	8,797 (1.46%)
Quality-trimmed bases R2 (bp)	0
Output bases R2 (bp)	89,025,930

Note: WARN status results from a low adapter detection rate (~0.18% R1; ~1.46% R2). This is expected — samples were run on a 300-cycle kit, so read runoff into the adapter region is uncommon. No reads were lost.

Cutadapt Adapter Trimming — Index Sequences

Status	
Run status	WARN
Input	
Total input reads	602,393
Total input bases (bp)	~1.80 × 10 ⁸
Read Filtering	
Reads too short	0
Reads too long	0
Reads with too many N	0
Output reads (passed)	602,393
Pass rate (%)	100.00%
R1 Output	
Reads with adapters (R1)	666 (0.11%)
Quality-trimmed bases R1 (bp)	0
Output bases R1 (bp)	90,595,184
R2 Output	
Reads with adapters (R2)	19,733 (3.28%)
Quality-trimmed bases R2 (bp)	0
Output bases R2 (bp)	88,959,613

Note: WARN status again reflects low adapter rate on a 300-cycle kit. No reads were discarded. Both trimming steps confirm clean input with minimal adapter contamination.

fastp UMI Extraction

	R1	R2
Before Filtering		
Total reads	602,393	602,393
Total bases (bp)	90,595,184	88,959,613
Q20 bases (bp)	88,254,856 (97.4167%)	81,320,982 (91.4134%)
Q30 bases (bp)	87,370,208 (96.4402%)	79,728,862 (89.6237%)
Q40 bases (bp)	33 (3.64e-05%)	1 (1.12e-06%)
After Filtering		
Total reads	601,605	601,605
Total bases (bp)	86,231,366	84,595,150
Q20 bases (bp)	83,973,070 (97.3811%)	77,364,361 (91.4525%)
Q30 bases (bp)	83,146,706 (96.4228%)	76,012,559 (89.8545%)
Q40 bases (bp)	31 (3.59e-05%)	1 (1.18e-06%)

Filtering Result	
Reads passed filter	1,203,210
Pass rate (%)	99.87%
Failed — low quality	1,572
Failed — too many N	0
Failed — too short	4
Reads with adapter trimmed	15,022
Bases trimmed (adapters, bp)	96,046
Duplication rate (%)	1.33883%
Insert size peak (bp)	162

Note: UMI extraction removed 2 extra nucleotides from the 5' end beyond the target UMI. This occurs because the 5' end contains a 5 nt UMI from the Step 1 primer, while the configuration file specifies a 7 nt UMI added via 3' adapter ligation. The 2 extra nucleotides are reinserted during the primer reinsertion step. As these nucleotides are outside the region of interest, they should not introduce variability in ShapeMapper2 output.

Primer Removal / Size Exclusion

Run Information	
Run status	OK
Input	
Total input reads	601,605
Total input bases (bp)	170,826,516
Read Filtering	
Reads too short	3153
Reads too long	0
Reads with too many N	0
Output reads (passed)	598,452
Pass rate (%)	99.48%

Output	
Output bases R1 (bp)	85,989,305
Output bases R2 (bp)	75,377,530

Note: Only size exclusion was applied (100–180 bp thresholds); primer removal was not performed as samples are 3'-ligated. Originally, primer regions were removed to eliminate non-variable sequences introduced by gene-specific Step 1 PCR primers. The first 5 nt following the R1 primer region are also considered error-prone due to MaP RT and are excluded accordingly.

BBMerge Overhang Removal

Pair Outcomes	
Total pairs	598,954
Joined (overlapping)	569,317 (95.132%)
Ambiguous	0 (0.000%)
No solution (non-overlapping)	29,135(4.868%)
Too short	0 (0.000%)
Insert Size Statistics	
Average insert size (bp)	146.8
Standard deviation (bp)	3.7
Mode (bp)	147
Insert size range (bp)	100-241
10th percentile (bp)	148
25th percentile (bp)	147
50th percentile / median (bp)	147
75th percentile (bp)	147
90th percentile (bp)	147

Note: Inspection of output FASTQ files confirms overhanging regions were properly removed by BBMerge's trim -by-overlap function. The extremely tight insert size distribution (SD = 4.0 bp; mode = median = 162 bp) reflects a highly uniform, well -size-selected U1 amplicon library.

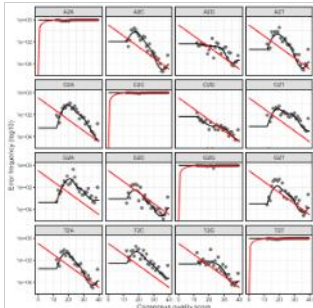
Quality Trimming and Correction by Overlap (fastp)

Metric	R1	R2
Before Filtering		
Total reads	569,748	569,748
Total bases (bp)	81,848,420	80,279,801
Q20 bases (bp)	80,317,074 (98.129%)	75,044,142 (93.4782%)
Q30 bases (bp)	79,710,457 (97.3879%)	73,989,044 (92.164%)
Q40 bases (bp)	15 (1.83e-05%)	0 (0.000%)
After Filtering		
Total reads	558,563	558,563
Total bases (bp)	80,241,424	78,701,468
Q20 bases (bp)	78,973,280 (98.4196%)	74,247,701 (94.3409%)
Q30 bases (bp)	78,443,086 (97.7588%)	73,301,488 (93.1387%)
Q40 bases (bp)	15 (1.87e-05%)	0 (0.000%)

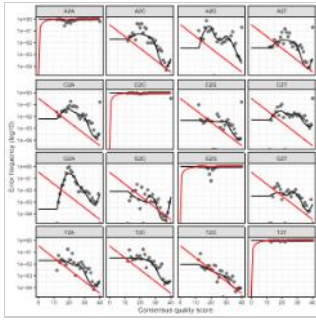
Filtering Result	
Reads passed filter	1,114,026
Pass rate (%)	98.04%
Failed — low quality	24608
Failed — too many N	0
Failed — too short	0
Reads with adapter trimmed	0
Bases trimmed (adapters, bp)	0
Overlap Correction	
Reads corrected by overlap	56,631
Bases corrected by overlap (bp)	73,688
Library Quality	
Duplication rate (%)	82.0522%
Insert size peak (bp)	147

DADA2 Denoising and Primer Reinsertion

- Forward Error Plot



- Reverse Error Plot



sequence_table_with_sequences_U1 (4)(AutoRecovered)

Note: All primers were successfully reinserted following DADA2 denoising.

OOPSint (S1)

After the OOPSint sample was integrated into the pipeline, the following outputs were obtained.

Cutadapt Adapter Trimming — Transposase and Index Sequences

Metric	Step 1 — Transposase	Step 2 — Index
Run Information		
Run status	WARN	WARN
Input		
Total input reads	985,692	985,692
Total input bases (bp)	293,740,913	293,694,646
Read Filtering		
Reads too short	0	0
Reads too long	0	0
Reads with too many N	0	0
Output reads (passed)	985,692	985,692
Pass rate (%)	100.00%	100.00%
R1 Output		
Reads with adapters (R1)	1,670 (0.17%)	1,430 (0.15%)
Quality-trimmed bases R1 (bp)	0	0
Output bases R1 (bp)	148,035,024	148,025,985
R2 Output		
Reads with adapters (R2)	8,870 (0.90%)	51,575 (5.23%)
Quality-trimmed bases R2 (bp)	0	0
Output bases R2 (bp)	145,659,622	145,496,177

Note: Both trimming steps returned WARN status, consistent with low adapter detection (~0.17% R1 Step 1; ~0.14% R1 Step 2; ~0.90% R2 Step 1; ~5.23% R2 Step 2). This is not unexpected for OOPSint samples and does not indicate a critical failure. No reads were lost to length or N-content filters, confirming good input read quality.

fastp UMI Extraction

	R1	R2
Before Filtering		
Total reads	985,692	985,692
Total bases (bp)	148,025,985	145,496,177
Q20 bases (bp)	144,409,260 (97.5567%)	138,186,910 (94.9763%)
Q30 bases (bp)	143,071,311 (96.6528%)	136,569,453 (93.8646%)
Q40 bases (bp)	21 (1.42e-05%)	2 (1.37e-06%)
After Filtering		
Total reads	983,860	983,860
Total bases (bp)	140,886,024	138,354,371
Q20 bases (bp)	137,407,928 (97.5313%)	131,472,342 (95.0258%)
Q30 bases (bp)	136,159,796 (96.6454%)	130,079,778 (94.0193%)
Q40 bases (bp)	10 (7.10e-06%)	2 (1.45e-06%)

Filtering Result	
Reads passed filter	1,967,720
Pass rate (%)	99.81%
Failed — low quality	3,656
Failed — too many N	0
Failed — too short	8
Reads with adapter trimmed	23,244
Bases trimmed (adapters, bp)	149,001
Duplication rate (%)	2.20464%
Insert size peak (bp)	162

Note: Input read quality is high: R1 Q30 ~96.7%, R2 Q30 ~93.9%. Post-filtering dropout was minimal — only ~1,832 read pairs lost (0.19%). Failures were predominantly low-quality reads (3,656) with negligible N or length attrition. Duplication rate of 2.2% indicates good library complexity and minimal PCR over-amplification, consistent with a well-prepared OOPSint sample. Insert size peak at 162 bp is consistent with expected OOPSint RNA fragment length and aligns with the DMSOaq result.

013 02242026 Implementing pipeline on all snRNA Samples

Wednesday, February 25, 2026 2:26 PM

- Objective:**
- Run all snRNA samples through snakemake pipeline, and shapemapper2
- Future:**
- Run all sequences through shapemapper2 pipeline

- Important Notes:**
- Samples require 3' removal of forward primer to reduce random variability of samples
 - Looking at U11, U12, U4-1,U4-2, U4atac,, U5A, U5B

Date	02/24/2026
Experiment	snRNA amplicon sequencing data analysis using Snakemake pipeline
Cell line	HeLa
Targets	U11.U12,U4,U4atac,U5A,U5B (Sm binding region)
Purpose	Observe the binding profiles of each snRNA at the Sm binding region, analyzing profile structure, variation, and correlation between snRNA
Protocol	1. Input sequencing data into Snakemake pipeline 2. Analyze rule outputs 3. Process through ShapeMapper2 4. Process through RNP-MaP 5. Analyze using PCA a. Variation analysis b. Correlation analysis between profiles
Notes	All snRNA were successfully processed through the snakemake pipeline. Multiple sequences were identified with transposase and index sequences. Umi extraction successful removed 5' and 3' end UMI, leaving a 2 nt overhang on the 3' end of R2 and the 5' end of R1. The Forward primer region was subsequently removed and any existing overhangs were trimmed by overlap. A quality and size filter was also applied, removing sequences that were longer than the expected insert size. It is important to recognize that 3' adapter ligation and provide longer products, but in each experiment, of the percentage of reads that were removed, only sequences that were too short were removed.

Results:

U11

Date recorded: 26 February 2026
Cell line: HeLa | Conditions: DMSOaq (S4) and OOPSint (S3)

Configuration File Parameters

The pipeline was configured to process paired-end HeLa U11 reads from two conditions: DMSO aqueous and OOPS interactome fractions. Only forward primer sequences (adapter_forward) were specified, with Cutadapt set to a permissive error rate of 0.2 and size selection between 80–150 bp. UMI extraction was configured for 7 bp UMIs with a quality score threshold of 20.

Samples	
U11_F_DMSO_aq	data/samples/HeLa-U11-DMSOaq_S4_L001_R1_001.fastq.gz
U11_R_DMSO_aq	data/samples/HeLa-U11-DMSOaq_S4_L001_R2_001.fastq.gz
U11_F_OOPS_int	data/samples/HeLa-U11-OOPSint_S3_L001_R1_001.fastq.gz
U11_R_OOPS_int	data/samples/HeLa-U11-OOPSint_S3_L001_R2_001.fastq.gz
Index Adapters	
Non_Transposase	data/ARL_NEBNext_adapters_No_transposease.fa
Transposase	data/ARL_NEBNext_adapters_transposease.fa
Cutadapt	
error_rate	0.2
R1_primer_length	0
R2_primer_length	17
min_length	80 bp
max_length	150 bp
fastp	
umi_len	7
quality_score_threshold	20
seqkit	
adapter_forward	AAAGGGCTTCTGTCGTG
adapter_reverse	(not specified)

Cutadapt Adapter Trimming — Transposase Sequences

Date recorded: 26 February 2026
Tool: Cutadapt | Cell line: HeLa | Sample: DMSOaq (S4) & OOPSint (S3) | Target: U11

Transposase adapter sequences were identified in 10.5% of DMSO read pairs and 10.0% of OOPS read pairs, with no reads lost to length or N-content filters. All input reads were retained after trimming, indicating clean adapter removal without excessive signal loss.

Metric	DMSOaq (S4)	OOPSint (S3)
Run Information		



Run status	WARN	WARN
Input		
Total input reads	327,919	442,741
Total input bases (bp)	90,381,650	~1.21 × 10 ⁸
R1 Output		
Reads with adapters (R1)	34,602	44,250
Adapter rate R1 (%)	10.55%	9.99%
Quality-trimmed bases R1 (bp)	0	0
Output bases R1 (bp)	44,999,277	60,361,786
R2 Output		
Reads with adapters (R2)	9,654	10,733
Adapter rate R2 (%)	2.94%	2.42%
Quality-trimmed bases R2 (bp)	0	0
Output bases R2 (bp)	45,106,842	60,557,689

Cutadapt Adapter Trimming — Index Sequences

Date recorded: 26 February 2026

Tool: Cutadapt | Cell line: HeLa | Sample: DMSOaq (S4) & OOPSint (S3) | Target: U11

Index adapter trimming completed successfully for both conditions (status OK). Adapters were detected in 7–10% of forward reads and ~8.5% of reverse reads per condition. No reads were discarded at this stage.

Metric	DMSOaq (S4)	OOPSint (S3)
Run Information		
Run status	OK	OK
Input		
Total input reads	327,919	442,741
Total input bases (bp)	90,106,119	~1.21 × 10 ⁸
R1 Output		
Reads with adapters (R1)	22,998	30,651
Adapter rate R1 (%)	7.01%	6.92%
Quality-trimmed bases R1 (bp)	0	0
Output bases R1 (bp)	44,846,628	60,166,430
R2 Output		
Reads with adapters (R2)	27,794	33,742
Adapter rate R2 (%)	8.48%	7.62%
Quality-trimmed bases R2 (bp)	0	0
Output bases R2 (bp)	44,948,137	60,358,381

fastp UMI Extraction

Date recorded: 26 February 2026

Tool: fastp | Cell line: HeLa | Sample: DMSOaq (S4) | Target: U11

UMI extraction proceeded well for both conditions. The DMSO sample retained 652,192 reads (99.5% pass rate) with a low duplication rate of 1.1% and insert size peak at 132 bp. The OOPS sample showed a slightly higher failure rate due to low quality (31,590 reads failed), retaining 853,212 reads with a 4.0% duplication rate and the same 132 bp insert peak, consistent with expected library complexity differences between conditions.

DMSOaq (S4)

Metric	R1	R2
Before Filtering		
Total reads	327,919	327,919
Total bases (bp)	44,846,628	44,948,137
Q20 bases (bp)	43,650,873 (97.3337%)	41,935,701 (93.298%)
Q30 bases (bp)	43,321,329 (96.5989%)	41,375,055 (92.0507%)
Q40 bases (bp)	148 (0.00033%)	4,355 (0.00969%)
After Filtering		
Total reads	326,096	326,096
Total bases (bp)	40,466,334	40,480,159
Q20 bases (bp)	39,407,368 (97.3831%)	37,824,370 (93.4393%)
Q30 bases (bp)	39,139,318 (96.7207%)	37,431,314 (92.4683%)
Q40 bases (bp)	147 (0.000363%)	4,354 (0.01076%)

Filtering Metric	DMSOaq (S4)
Filtering Result	
Reads passed filter	652,192
Pass rate (%)	99.44%
Failed — low quality	3,592
Failed — too many N	0
Failed — too short	54
Reads with adapter trimmed	619,442
Bases trimmed (adapters, bp)	4,044,647
Duplication rate (%)	1.13595%
Insert size peak (bp)	132

OOPSint (S3)

Metric	R1	R2
Before Filtering		
Total reads	442,741	442,741
Total bases (bp)	60,166,430	60,358,381
Q20 bases (bp)	58,107,411 (96.5778%)	55,616,111 (92.1431%)
Q30 bases (bp)	57,680,381 (95.868%)	54,806,305 (90.8015%)
Q40 bases (bp)	41 (0.0000681%)	311 (0.000515%)
After Filtering		
Total reads	426,606	426,606
Total bases (bp)	53,950,456	53,977,748
Q20 bases (bp)	52,532,179 (97.3711%)	50,219,514 (93.0374%)
Q30 bases (bp)	52,192,257 (96.7411%)	49,641,919 (91.9674%)
Q40 bases (bp)	41 (0.000076%)	310 (0.000574%)

Filtering Metric	OOPSint (S3)
Filtering Result	
Reads passed filter	853,212
Pass rate (%)	96.36%
Failed — low quality	31,590
Failed — too many N	0
Failed — too short	680
Reads with adapter trimmed	818,164
Bases trimmed (adapters, bp)	5,372,865
Duplication rate (%)	4.00979%
Insert size peak (bp)	132

Primer Removal / Size Exclusion

Date recorded: 26 February 2026

Tool: Cutadapt| Cell line: HeLa | Sample: DMSOaq (S4) | Target: U11

Of 326,096 DMSO reads entering this stage, 298,452 passed (91.5%). The 27,644 reads removed were too short after primer trimming, consistent with expected removal of reads lacking full insert sequence. No reads were lost to length excess or N-content. Note: OOPS data for this step was not provided.

Metric	DMSOaq (S4)
Run Information	
Run status	OK
Input	
Total input reads	326,096
Total input bases (bp)	80,946,493
Read Filtering	
Reads too short	27,644 (8.48%)
Reads too long	0
Reads with too many N	0
Output reads (passed)	298,452
Pass rate (%)	91.52%
Output	
Reads with adapters (R1)	0
Quality-trimmed R1 (bp)	0
Output bases R1 (bp)	38,128,157

BBMerge Overhang Removal

Date recorded: 26 February 2026

Tool: BBMerge | Cell line: HeLa | Sample: DMSOaq (S4) | Target: U11

BBMerge was run in overlap-detection mode (tbo=true, merge=false): reads are identified as overlapping but written out unmerged. Of 298,452 pairs, 286,032 (95.8%) were successfully identified as overlapping. No pairs were ambiguous or too short. The insert size distribution is tight (mode and median 115 bp, SD 13.1 bp), consistent with a well-size-selected amplicon library. Only 4.2% of pairs could not be merged, likely due to low-overlap or divergent sequences.

Metric	DMSOaq (S4)
Pair Outcomes	
Total pairs	298,452
Joined (overlapping)	286,032 (95.84%)
Ambiguous	0 (0.00%)
No solution (non-overlapping)	12,420 (4.16%)
Too short	0 (0.00%)
Insert Size Statistics	

Average insert size (bp)	111
Standard deviation (bp)	13.1
Mode (bp)	115
Insert size range (bp)	80 – 247
10th percentile (bp)	84
25th percentile (bp)	114
50th percentile / median (bp)	115
75th percentile (bp)	115
90th percentile (bp)	116

Quality Trimming and Correction by Overlap

Date recorded: 26 February 2026

Tool: Fastp| Cell line: HeLa | Sample: DMSOaq (S4) | Target: U11

Post-merge quality trimming retained 563,602 reads (98.5% pass rate). Overlap-based correction improved 16,422 reads, fixing 20,492 bases — a modest but meaningful error correction step. The high duplication rate (87.8%) reflects expected PCR amplicon convergence after merging, with the insert size peak confirming the dominant 115 bp U11 amplicon.

Metric	R1	R2
Before Filtering		
Total reads	286,032	286,032
Total bases (bp)	31,652,675	31,587,498
Q20 bases (bp)	31,012,139 (97.9764%)	30,165,002 (95.4966%)
Q30 bases (bp)	30,879,051 (97.5559%)	29,916,374 (94.7095%)
Q40 bases (bp)	128 (0.000404%)	3,782 (0.01197%)
After Filtering		
Total reads	281,801	281,801
Total bases (bp)	31,183,111	31,119,278
Q20 bases (bp)	30,633,831 (98.2385%)	29,925,097 (96.1626%)
Q30 bases (bp)	30,514,609 (97.8562%)	29,708,366 (95.4661%)
Q40 bases (bp)	128 (0.00041%)	3,782 (0.01215%)

Filtering Metric	DMSOaq (S4)
Filtering Result	
Reads passed filter	563,602
Pass rate (%)	98.52%
Failed — low quality	8,462
Failed — too many N	0
Failed — too short	0
Reads with adapter trimmed	0
Bases trimmed (adapters, bp)	0
Reads corrected by overlap	16,422
Bases corrected by overlap (bp)	20,492
Duplication rate (%)	87.8199%
Insert size peak (bp)	115

Divisive Amplicon Denoising Algorithm (DADA2)

Date recorded: 26 February 2026

Tool: dada2 | Cell line: HeLa | Sample: DMSOaq (S4) | Target: U11

DADA2 denoising processed 281,801 reads for the DMSOaq sample. After error-rate learning and denoising, 257,316 paired reads (in 73 unique pairings) were successfully merged from 281,046 input pairs across 365 candidate pairings — a merge success rate of 91.5%. The read count entering and leaving the DADA2 step is identical (281,801), confirming the tool reports reads in/out prior to the merging step. The 73 unique amplicon pairings suggest a focused, amplicon-specific signal consistent with a U11 pulldown experiment.

Metric	DMSOaq (S4)
DADA2 Run Statistics	
Reads in	281,801
Reads out	281,801
Paired reads successfully merged	257,316
Unique pairings used	73
Total pairings considered	365
Total input to merging step	281,046
Merge success rate (%)	91.56%
Amplicon Length Distribution (merged reads)	
80 bp	6
81 bp	6
82 bp	2
83 bp	6

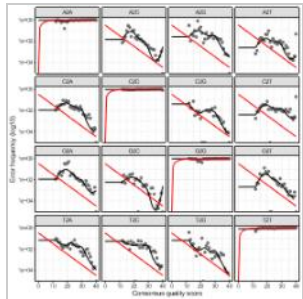
85 bp	1
87 bp	1
89 bp	2
90 bp	2
91 bp	1
97 bp	3
98 bp	3
100 bp	1
102 bp	2
104 bp	1
112 bp	1
114 bp	1
115 bp	9
118 bp	1
119 bp	1
121 bp	1
122 bp	1
125 bp	3
130 bp	2
131 bp	1
133 bp	1
141 bp	1
145 bp	1
164 bp	1
167 bp	2
172 bp	1
194 bp	1
202 bp	2
210 bp	1
217 bp	1
222 bp	1
229 bp	1
233 bp	1

- Forward Error Plot

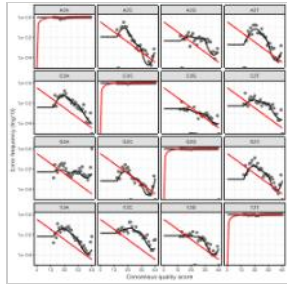


sequence_t
able_with...

- Forward Error Plot



- Reverse Error Plot



Notes:
Divisive Amplicon Denoising (DADA2) Error learning was performed on ~31 million bases from 281,801 reads. The forward and reverse error rate models are shown in the plots above. Observed error rates (black curves) closely follow the learned model and trend downward with increasing quality score, indicating well-calibrated error estimation. Of 281,046 read pairs processed, 257,316 (91.6%) were successfully merged into 73 unique sequence pairings. The dominant amplicon length is 115 bp (9 unique sequences), with a long tail of lower-abundance lengths reflecting minor variants or off-target products.

Primer Reinsertion

- All primers were successfully reinserted.

U12
Configuration File Parameters

samples:
U12_F_DMSO_aq: data/samples/HeLa-U12-DMSOaq_S6_L001_R1_001.fastq.gz
U12_R_DMSO_aq: data/samples/HeLa-U12-DMSOaq_S6_L001_R2_001.fastq.gz
U12_F_OOPS_int: data/samples/HeLa-U12-OOPSint_S5_L001_R1_001.fastq.gz
U12_R_OOPS_int: data/samples/HeLa-U12-OOPSint_S5_L001_R2_001.fastq.gz
Index_adapters:
Non_Transposase: data/ARL_NEBNext_adapters_No_transposase.fa
Transposase: data/ARL_NEBNext_adapters_transposase.fa
Cutadapt:
error_rate: 0.2
R1_primer_Length: 0
R2_primer_Length: 21
min_length: 100
max_length: 180
fastp:
umi_len: 7
quality_score_threshold: 20
seqkit:
adapter_forward: GCCTTAACTTATGAGTAAGG
adapter_reverse:

Cutadapt adapter trimming (Transposase Sequences):

Date recorded: 26 February 2026
Tool: Cutadapt | Cell line: HeLa | Sample: DMSOaq (S6) | Target: U12

Sample	Status	In Reads	In BP	Too Short	Too Long	Too Many N	Out Reads	w/ Adapters (R1)	QualTrim BP (R1)	Out BP (R1)	w/ Adapters (R2)	QualTrim BP (R2)	Out BP (R2)
DMSO (HeLa-U12-DMSOaq)	WARN	328,765	96,882,721	0	0	0	328765	2052	0	48,716,789	20333	0	48,050,758
OOPS (HeLa-U12-OOPSint)	WARN	409,106	121,155,298	0	0	0	409106	1545	0	60,865,695	9917	0	60,236,693

Cutadapt adapter trimming (Index Sequences):

Date recorded: 26 February 2026
Tool: Cutadapt | Cell line: HeLa | Sample: DMSOaq (S6) | Target: U12

Sample	Status	In Reads	In BP	Too Short	Too Long	Too Many N	Out Reads	w/ Adapters (R1)	QualTrim BP (R1)	Out BP (R1)	w/ Adapters (R2)	QualTrim BP (R2)	Out BP (R2)
DMSO (HeLa-U12-DMSOaq)	OK	328,765	96,767,547	0	0	0	328765	844	0	48,707,915	13718	0	47,990,704
OOPS (HeLa-U12-OOPSint)	WARN	409,106	121,102,388	0	0	0	409106	536	0	60,861,777	81840	0	59,941,190

FastP Umi Extraction:

Date recorded: 26 February 2026
Tool: Fastp | Cell line: HeLa | Sample: DMSOaq (S6) | Target: U12

Quality Trimming:

Sample	Read	Stage	Total Reads	Total Bases	Q20 Bases	Q20 %	Q30 Bases	Q30 %	Q40 Bases	Q40 %
DMSO	R1	Before	328,765	48,707,915	44,694,113	91.7594%	43,109,757	88.5067%	57	0.0001%
DMSO	R1	After	328,365	46,255,761	42,341,073	91.5369%	40,812,408	88.2321%	57	0.0001%
DMSO	R2	Before	328,765	47,990,704	42,378,443	88.3055%	40,663,697	84.7324%	17	0.0000%
DMSO	R2	After	328,365	45,533,356	40,217,893	88.3262%	38,681,866	84.9528%	17	0.0000%
OOPS	R1	Before	409,106	60,861,777	58,307,608	95.8033%	57,184,183	93.9575%	250	0.0004%
OOPS	R1	After	408,735	57,876,943	55,382,265	95.6897%	54,293,265	93.8081%	249	0.0004%
OOPS	R2	Before	409,106	59,941,190	57,269,108	95.5422%	56,444,300	94.1661%	380	0.0006%
OOPS	R2	After	408,735	56,951,035	54,393,913	95.5100%	53,646,995	94.1985%	379	0.0007%

Sample	Reads Passed	Failed: Low Quality	Failed: Too Many N	Failed: Too Short	Reads w/ Adapter Trimmed	Bases Trimmed	Duplicati on Rate	Insert Size Peak
DMSO	656,730	754	0	46	32,400	203,616	2.17%	148
OOPS	817,470	734	0	8	29,042	161,721	4.20%	148

Primer Removal/Size exclusion:

Date recorded: 26 February 2026
Tool: Cutadapt | Cell line: HeLa | Sample: DMSOaq (S6) | Target: U12

Sample	Status	In Reads	In BP	Too Short	Too Long	Too Many N	Out Reads	w/ Adapters (R1)	QualTrim BP (R1)	Out BP (R1)	w/ Adapters (R2)	QualTrim BP (R2)	Out BP (R2)
DMSO (HeLa-U12-DMSOaq)	OK	328,365	91,789,117	12,960	0	0	315,405	0	0	45,088,383	0	0	37,747,909

BBMerge overhang removal

Date recorded: 26 February 2026

Tool: BBMerge | Cell line: HeLa | Sample: DMSOaq (S6) | Target: U12

Merging Summary			
Category	Count	Percentage	Notes
Total Pairs	315,405	—	
Joined	293,301	92.992%	Successfully merged
Ambiguous	0	0.000%	
No Solution	22,104	7.008%	Could not be merged
Too Short	0	0.000%	
Insert Size Statistics			
Statistic	Value	Unit	Notes
Average Insert	127.3	bp	
Standard Deviation	5.1	bp	Very tight distribution
Mode	127	bp	
Insert Range	100 – 244	bp	
90th Percentile	127	bp	
75th Percentile	127	bp	
50th Percentile (Med)	127	bp	
25th Percentile	127	bp	
10th Percentile	127	bp	

Note: 92.99% of pairs successfully merged. The insert size distribution is extremely tight (SD = 5.1 bp) with all percentiles collapsing to 127 bp, indicating a highly uniform, well-size-selected library.

Quality Trimming and Correction by Overlap

Date recorded: 26 February 2026

Tool: fastp| Cell line: HeLa | Sample: DMSOaq (S6) | Target: U12

Read Quality Summary

Read	Stage	Total Reads	Total Bases	Q20 Bases	Q20 %	Q30 Bases	Q30 %	Q40 Bases	Q40 %
R1	Before	293,301	37,272,036	34,800,190	93.3681 %	33,919,194	91.0044 %	21	0.0001 %
R1	After	288,185	36,620,523	34,365,660	93.8426 %	33,539,621	91.5870 %	21	0.0001 %
R2	Before	293,301	35,097,587	31,338,509	89.2896 %	30,141,352	85.8787 %	14	0.0000 %
R2	After	288,185	34,485,146	31,015,309	89.9382 %	29,874,166	86.6291 %	14	0.0000 %

Filtering Results

Metric	Value
Reads Passed Filter	576,370
Failed: Low Quality	10,232
Failed: Too Many N	0
Failed: Too Short	0
Reads with Adapter Trimmed	0
Bases Trimmed (Adapters)	0
Reads Corrected by Overlap	64,402
Bases Corrected by Overlap	79,115
Duplication Rate	77.66%
Insert Size Peak	127 bp

Note: High duplication rate (77.66%) reflects expected PCR amplicon convergence post-merge. 64,402 reads were error-corrected by overlap analysis. No adapter trimming was required at this stage, confirming clean upstream processing.

Divisive Amplicon Denoising Algorithm

Date recorded: 26 February 2026

Tool: Dada2 | Cell line: HeLa | Sample: DMSOaq (S6) | Target: U12

Error Learning & Read Summary

Metric	Value	Unit	Notes
Reads In (R1)	288,185	reads	
Reads Out (R1)	288,185	reads	No reads lost (expected)
Bases for Error Learning (R1)	36,620,523	bp	
Bases for Error Learning (R2)	34,485,146	bp	

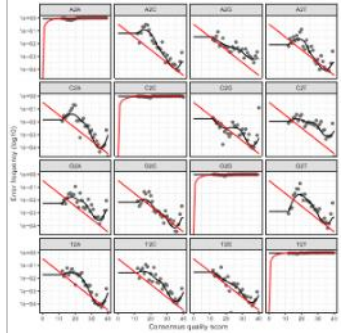
Unique Sequences (R1)	20,899	seqs	After denoising
Unique Sequences (R2)	22,441	seqs	After denoising
Paired-Reads Merged	277,216	reads	of 286,898 input
Merge Rate	96.63%		
Unique Pairings (Merged)	73	seqs	
Input Pairings	364		
Output DNASTringSet Length	73	seqs	

Amplicon Length Distribution

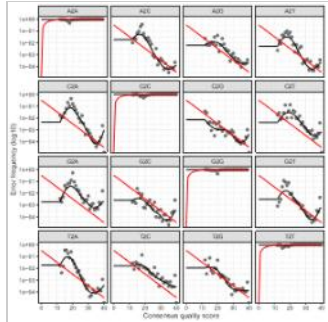
Length (bp)	Count	% of Total	Notes
100	1	1.37%	
102	4	5.48%	
107	2	2.74%	
108	1	1.37%	
109	1	1.37%	
110	1	1.37%	
111	1	1.37%	
113	1	1.37%	
114	1	1.37%	
118	1	1.37%	
121	2	2.74%	
127	21	28.77%	Dominant amplicon
129	1	1.37%	
136	1	1.37%	
137	4	5.48%	
139	2	2.74%	
140	6	8.22%	
141	6	8.22%	
150	5	6.85%	
165	1	1.37%	
171	2	2.74%	
172	1	1.37%	
184	1	1.37%	
191	1	1.37%	
211	1	1.37%	
213	1	1.37%	
219	1	1.37%	
228	1	1.37%	
243	1	1.37%	

Note: 96.63% of read pairs successfully merged into 73 unique pairings. The 127 bp amplicon is strongly dominant (21/73 unique pairings). Log-10 scale warnings during error plotting are cosmetic and do not affect denoising.

- Forward Error Plot



- Reverse Error Plot



Primer Reinsertion

- All primers were successfully reinserted.

U4 (U4-1, U4-2)

Configuration File Parameters

```
samples:
  U4_F_DMSO_aq: data/samples/HeLa-U4-DMSOaq_S8_L001_R1_001.fastq.gz
  U4_R_DMSO_aq: data/samples/HeLa-U4-DMSOaq_S8_L001_R2_001.fastq.gz
  U4_F_OOPS_int: data/samples/HeLa-U4-OOPSint_S7_L001_R1_001.fastq.gz
  U4_R_OOPS_int: data/samples/HeLa-U4-OOPSint_S7_L001_R2_001.fastq.gz
Index_adapters:
  Non_Transposase: data/ARL_NEBNext_adapters_No_transposease.fa
  Transposase: data/ARL_NEBNext_adapters_transposease.fa
Cutadapt:
  error_rate: 0.2
  R1_primer_Length: 0
  R2_primer_Length: 14
  min_length: 100
  max_length: 160
fastp:
  umi_len: 7
  quality_score_threshold: 20
seqkit:
  adapter_forward: CTTTGCAGTGGC
  adapter_reverse:
```

Cutadapt adapter trimming (Transposase Sequences):

Date recorded: 26 February 2026
Tool: Cutadapt | Cell line: HeLa | Sample: DMSOaq (S12) and OOPSint (11) | Target: U4
Summary Statistics

Metric	HeLa-U4-OOPSint (S7)	HeLa-U4-DMSOaq (S8)
Run Information		
Source file	HeLa-U4-OOPSint_S7_L001.log	HeLa-U4-DMSOaq_S8_L001.log
Run status	WARN	WARN
Input Reads		
Total input reads	527,092	424,553
Total input bases (bp)	155,555,479	124,625,186
Read 1 (R1) Output		
Reads with adapters (R1)	20,289	17,926
Adapter rate R1 (%)	3.85%	4.22%
Quality-trimmed bases R1 (bp)	0	0
Output bases R1 (bp)	77,656,610	62,261,292
Read 2 (R2) Output		
Reads with adapters (R2)	27,034	18,198
Adapter rate R2 (%)	5.13%	4.29%
Quality-trimmed bases R2 (bp)	0	0
Output bases R2 (bp)	77,624,424	62,158,259
Total Output		
Total output bases (R1 + R2, bp)	155,281,034	124,419,551
Base retention (%)	99.82%	99.83%

Notes:
Both samples completed trimming with WARN status. Adapter contamination rates were low for both samples (3.5 & 5.1% R1; 4.3 & 6.4% R2). No quality-based trimming was performed. Read counts and base counts are consistent with typical HeLa paired-end sequencing runs.

Cutadapt adapter trimming (Index Sequences):

Date recorded: 26 February 2026
Tool: Cutadapt | Cell line: HeLa | Sample: DMSOaq (S12) and OOPSint (11) | Target: U4
Summary Statistics

Metric	HeLa-U4-OOPSint (S7)	HeLa-U4-DMSOaq (S8)
Run Information		
Source file	HeLa-U4-OOPSint_S7_L001 (1).log	HeLa-U4-DMSOaq_S8_L001 (1).log
Run status	WARN	WARN
Input Reads		
Total input reads	527,092	424,553
Total input bases (bp)	155,281,034	124,419,551
Read 1 (R1) Output		
Reads with adapters (R1)	6,649	4,611
Adapter rate R1 (%)	1.26%	1.09%
Quality-trimmed bases R1 (bp)	0	0
Output bases R1 (bp)	77,620,904	62,227,409
Read 2 (R2) Output		

Reads with adapters (R2)	194,115	159,880
Adapter rate R2 (%)	36.83%	37.66%
Quality-trimmed bases R2 (bp)	0	0
Output bases R2 (bp)	76,695,243	61,416,167
Total Output		
Total output bases (R1 + R2, bp)	154,316,147	123,643,576
Base retention (%)	99.38%	99.38%

Notes:

Both samples completed trimming with WARN status. A notable asymmetry is observed between R1 and R2 adapter rates: R1 adapter contamination was very low (1.3% for OOPSint; 1.1% for DMSOaq), whereas R2 adapter rates were substantially higher (36.8% for OOPSint; 37.7% for DMSOaq). This R1/R2 discrepancy likely reflects the library preparation protocol (e.g. short insert sizes or ligation artefacts). No quality-based trimming was performed. Overall base retention was 99.4% (OOPSint) and 98.6% (DMSOaq).

FastP Umi Extraction:

Date recorded: 26 February 2026

Tool: fastp | Cell line: HeLa | Sample: DMSOaq (S12) and OOPSint (11) | Target: U4

Summary Statistics

Metric	HeLa-U4-OOPSint (S7)	HeLa-U4-DMSOaq (S8)
Run Information		
Source file	HeLa-U4-OOPSint_S7_L001 (1).log	HeLa-U4-DMSOaq_S8_L001 (1).log
Run status	WARN	WARN
Input Reads		
Total input reads	527,092	424,553
Total input bases (bp)	155,281,034	124,419,551
Read 1 (R1) Output		
Reads with adapters (R1)	6,649	4,611
Adapter rate R1 (%)	1.26%	1.09%
Quality-trimmed bases R1 (bp)	0	0
Output bases R1 (bp)	77,620,904	62,227,409
Read 2 (R2) Output		
Reads with adapters (R2)	194,115	159,880
Adapter rate R2 (%)	36.83%	37.66%
Quality-trimmed bases R2 (bp)	0	0
Output bases R2 (bp)	76,695,243	61,416,167
Total Output		
Total output bases (R1 + R2, bp)	154,316,147	123,643,576
Base retention (%)	99.38%	99.38%

Notes:

Both samples completed trimming with WARN status.A notable asymmetry is observed between R1 and R2 adapter rates: R1 adapter contamination was very low (1.3% for OOPSint and 1.1% for DMSOaq), whereas R2 adapter rates were substantially higher (36.8% for OOPSint; 37.7% for DMSOaq). This R1/R2 discrepancy likely reflects the library preparation protocol (e.g. short insert sizes or ligation artefacts). No quality-based trimming was performed. Overall base retention was 99.4% (OOPSint) and 98.6% (DMSOaq).

Primer Removal/Size exclusion:

Date recorded: 26 February 2026

Tool: Cutadapt | Cell line: HeLa | Sample: DMSOaq (S12) and OOPSint (11) | Target: U4

Summary Statistics

Metric	Value
Run Information	
Source file	HeLa-U4-DMSOaq_S8_L001 (3).log
Run status	OK
Input Reads	
Total input reads	424,390
Total input bases (bp)	117,103,739
Read Filtering	
Reads too short	21,639
Reads too long	0
Reads with too many N	0
Output reads (passed)	402,751
Reads removed (%)	5.10%
Reads passing filter (%)	94.90%
R1 Output	
Reads with adapters (R1)	0
Quality-trimmed bases R1 (bp)	0
Output bases R1 (bp)	56,966,240
R2 Output	
Reads with adapters (R2)	0
Quality-trimmed bases R2 (bp)	0

Output bases R2 (bp)	50,403,558
Total Output	
Total output bases (R1 + R2, bp)	107,369,798
Base retention (%)	91.69%

Notes:

Run status is OK (no WARN). Notably, 21,639 reads (5.10%) were removed as too short — a marked departure from previous Cutadapt runs where no reads were filtered. No reads failed due to length being too long or too many N bases. No adapter contamination was detected in either R1 or R2 (w/adapters = 0), and no quality-based trimming was performed. Overall base retention was 91.69%. The input read count (424,390) matches the output read count from the preceding fastp UMI removal step for this sample, confirming correct file chaining.

BBMerge overhang removal

Date recorded: 26 February 2026
 Tool: BBMerge| Cell line: HeLa | Sample: DMSOaq (S12) and OOPSint (11) | Target: U4

Summary Statistics

Pair Outcomes	
Total pairs	402,751
Joined (overlapping)	362,480 (90.001%)
Ambiguous	0 (0.000%)
No solution (non-overlapping)	40,271 (9.999%)
Too short	0 (0.000%)
Insert Size Statistics	
Average insert size (bp)	127.9
Standard deviation (bp)	6.6
Mode (bp)	128
Insert size range (bp)	100 – 254
10th percentile (bp)	128
25th percentile (bp)	128
50th percentile / median (bp)	128
75th percentile (bp)	128
90th percentile (bp)	129

Notes:

BBMerge was run in overlap-detection mode only (merge=false): reads are identified as overlapping but written out as separate unmerged files. Of 402,751 pairs, 362,480 (90.0%) were successfully identified as overlapping. No pairs were ambiguous and none were too short. The insert size distribution is extremely tight: the mode, median, 25th and 75th percentiles all fall at 128 bp, with a standard deviation of only 6.6 bp. This is highly consistent with the insert size peak of 142 bp estimated by fastp in the preceding step (the ~14 bp difference likely reflects UMI and primer sequence that have since been removed).

Quality Trimming and Correction by Overlap (fastp)

Date recorded: 26 February 2026
 Tool: fastp| Cell line: HeLa | Sample: DMSOaq (S12) and OOPSint (11) | Target: U4

Metric	R1	R2
Adapter Detection		
Adapter detected	None detected	None detected
Before Filtering		
Total reads	362,480	362,480
Total bases (bp)	46,263,492	45,357,182
Q20 bases (bp)	41,638,451 (90.0028%)	44,255,665 (97.5715%)
Q30 bases (bp)	40,133,868 (86.7506%)	43,921,405 (96.8345%)
Q40 bases (bp)	1,520 (0.003286%)	1 (0.0000022%)
After Filtering		
Total reads	361,521	361,521
Total bases (bp)	46,140,986	45,237,738
Q20 bases (bp)	41,802,705 (90.5978%)	44,181,666 (97.6655%)
Q30 bases (bp)	40,306,823 (87.3558%)	43,854,262 (96.9418%)
Q40 bases (bp)	1,520 (0.003294%)	1 (0.00000221%)

Filtering / Library Metric	Value
Filtering Result	
Reads passed filter	723,042
Pass rate (%)	99.74%
Failed — low quality	1,918
Failed — too many N	0
Failed — too short	0
Reads with adapter trimmed	0

Bases trimmed (adapters, bp)	0
Overlap Correction	
Reads corrected by overlap	186,045
Correction rate (%)	51.33%
Bases corrected by overlap (bp)	251,482
Library Quality	
Duplication rate (%)	79.1958%
Insert size peak (bp)	128

Note:

No adapter sequences were detected by fastp auto-detection in either R1 or R2. Quality trimming with a Q20 threshold retained 723,042 reads (99.7% pass rate) with only 1,918 reads failing due to low quality. Notably, overlap-based correction improved 186,045 reads (25.8% of all reads), fixing 251,482 bases — a substantial correction signal consistent with the tight, overlapping insert size of 128 bp. The high duplication rate (79.2%) reflects expected PCR amplicon convergence for a U4 pulldown library at this stage of processing.

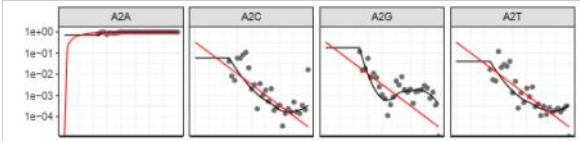
Divisive Amplicon Denoising Algorithm (DADA2)

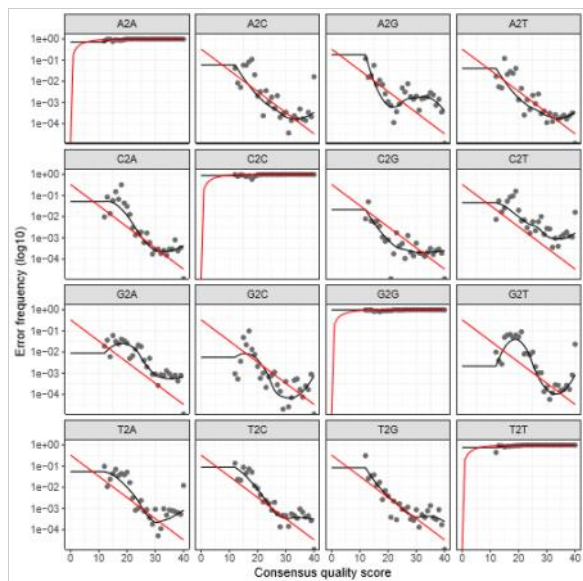
Date recorded: 26 February 2026
Tool: Dada2| Cell line: HeLa | Sample: DMSOaq (S12) and OOPSint (11) | Target: U4

DADA2 Run Statistics	
Reads in	361,521
Reads out	361,521
Unique R1 sequences (error learning)	30,807
Unique R2 sequences (error learning)	20,758
Paired reads successfully merged	339,842
Unique pairings used	63
Total pairings considered	541
Total input to merging step	359,547
Merge success rate (%)	94.52%
Output file	dada2/sequence_table_with_sequences_U1.xlsx

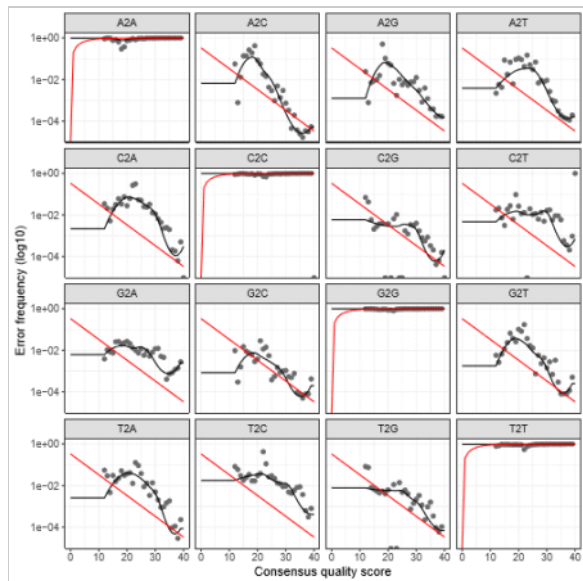
Amplicon Length (bp)	Number of Unique Pairings (ASVs)	Notes
101 bp	1	
103 bp	2	
104 bp	2	
106 bp	1	
111 bp	1	
118 bp	1	
128 bp	30	Most Dominant Length
130 bp	1	
132 bp	1	
134 bp	1	
137 bp	1	
138 bp	1	
139 bp	1	
140 bp	1	
148 bp	1	
168 bp	3	
169 bp	1	
174 bp	1	
181 bp	1	
198 bp	2	
202 bp	1	
203 bp	2	
209 bp	1	
214 bp	2	
222 bp	1	
246 bp	1	
250 bp	1	

- Forward Error Plot





- Reverse Error Plot



Note:

DADA2 processed 361,521 reads for the DMSOaq (S8) sample. After error-rate learning (30,807 unique R1 sequences; 20,758 unique R2 sequences), 339,842 paired reads (in 63 unique pairings) were successfully merged from 359,547 input pairs across 541 candidate pairings — a merge success rate of 94.52%. The dominant amplicon length is 128 bp (30 of 63 unique pairings), consistent with the insert size peak confirmed at every upstream step. A total of 63 ASVs were written to the output Excel file.

Primer Reinsertion

- All primers were successfully reinserted.

U5 (U5A, U5B)

Configuration File Parameters

Samples	
U5_F_DMSO_aq	data/samples/HeLa-U5-DMSOaq_S2_L001_R10_001.fastq.gz
U5_R_DMSO_aq	data/samples/HeLa-U5-DMSOaq_S2_L001_R10_001.fastq.gz
U5_F_OOPS_int	data/samples/HeLa-U5-OOPSint_S1_L001_R9_001.fastq.gz
U5_R_OOPS_int	data/samples/HeLa-U5-OOPSint_S1_L001_R9_001.fastq.gz
Index Adapters	
Non_Transposase	data/ARL_NEBNext_adapters_No_transposease.fa
Transposase	data/ARL_NEBNext_adapters_transposease.fa
Cutadapt	
error_rate	0.2
R1_primer_length	0

R2_primer_length	14
min_length	80 bp
max_length	130 bp
fastp	
umi_len	7
quality_score_threshold	20
seqkit	
adapter_forward	ACTCTGGTTTCTCT
adapter_reverse	(not specified)

Cutadapt adapter trimming (Transposase Sequences):

Date recorded: 26 February 2026

Tool: Cutadapt | Cell line: HeLa | Sample: DMSOaq (S12) and OOPSint (11) | Target: U5

Metric	DMSOaq (S12)	OOPSint (S11)
Run Information		
Run status	OK	WARN
Input		
Total input reads	775,736	591,380
Total input bases (bp)	194,434,655	149,768,121
R1 Output		
Reads with adapters (R1)	50,135 (6.46%)	61,553 (10.41%)
Quality-trimmed bases R1 (bp)	0	0
Output bases R1 (bp)	96,836,081	74,592,840
R2 Output		
Reads with adapters (R2)	63,034 (8.13%)	15,265 (2.58%)
Quality-trimmed bases R2 (bp)	0	0
Output bases R2 (bp)	96,857,661	74,841,775
Total Output		
Total output bases (R1 + R2, bp)	193,693,742	149,434,615
Base retention (%)	99.62%	99.78%

Notes:

DMSOaq (S12) completed with OK status. Index adapter contamination was detected in 6.46% of R1 reads and 8.13% of R2 reads.

OOPSint (S11) completed with WARN status, consistent with a low adapter detection rate (~10.41% R1; ~2.58% R2). This is not unexpected for OOPSint samples. The asymmetry between R1 and R2 adapter rates (10.41% vs 2.58%) may reflect differences in read orientation relative to the ligation adapter.

Cutadapt adapter trimming (Index Sequences):

Date recorded: 26 February 2026

Tool: Cutadapt | Cell line: HeLa | Sample: DMSOaq (S12) and OOPSint (11) | Target: U5

	DMSOaq (S12)	OOPSint (S11)
Run Information		
Run status	OK	OK
Input		
Total input reads	775,736	591,380
Total input bases (bp)	194,434,655	149,434,615
Read Filtering		
Reads too short	0	0
Reads too long	0	0
Reads with too many N	0	0
Output reads (passed)	775,736	591,380
Pass rate (%)	100.00%	100.00%
R1 Output		
Reads with adapters (R1)	50,135 (6.46%)	36,742 (6.21%)
Quality-trimmed bases R1 (bp)	0	0
Output bases R1 (bp)	96,836,081	74,360,271
R2 Output		
Reads with adapters (R2)	63,034 (8.13%)	47,205 (7.98%)
Quality-trimmed bases R2 (bp)	0	0
Output bases R2 (bp)	96,857,661	74,491,787

Notes:

Both samples completed with OK status. No reads were lost to length or N-content filters. Adapter contamination rates were modest and symmetric between R1 and R2 for both conditions.

FastP Umi Extraction:

Date recorded: 26 February 2026

Tool: Fastp| Cell line: HeLa | Sample: DMSOaq (S11) & OOPSint (S12) | Target: U5

Both samples show high read quality and very low duplication rates, with a shared insert size peak of 115 bp. A notably high adapter trimming rate was observed in both conditions (>97% of reads contained adapter sequence), consistent with short-insert libraries where reads run through the adapter.

DMSOaq (S12)

	R1	R2
Before Filtering		
Total reads	775,736	775,736
Total bases (bp)	96,836,081	96,857,661
Q20 bases (bp)	93,672,879 (96.7334%)	94,634,822 (97.705%)
Q30 bases (bp)	92,977,563 (96.0154%)	94,170,253 (97.2254%)
Q40 bases (bp)	1,196 (0.001235%)	345,900 (0.357122%)
After Filtering		
Total reads	774,875	774,875
Total bases (bp)	86,568,011	86,392,371
Q20 bases (bp)	83,729,078 (96.7206%)	84,460,404 (97.7637%)
Q30 bases (bp)	83,156,735 (96.0594%)	84,125,695 (97.3763%)
Q40 bases (bp)	1,135 (0.001311%)	345,757 (0.400217%)

Filtering Result	
Reads passed filter	1,549,750
Pass rate (%)	99.89%
Failed — low quality	1,718
Failed — too many N	0
Failed — too short	4
Reads with adapter trimmed	1,500,468
Adapter trimming rate (%)	96.82%
Bases trimmed (adapters, bp)	9,779,950
Duplication rate (%)	0.818191%
Insert size peak (bp)	115
Processing time	6 seconds

OOPSint (S11)

	R1	R2
Before Filtering		
Total reads	591,380	591,380
Total bases (bp)	74,360,271	74,491,787
Q20 bases (bp)	72,606,932 (97.6421%)	70,815,908 (95.0654%)
Q30 bases (bp)	72,247,004 (97.1581%)	70,180,681 (94.2126%)
Q40 bases (bp)	1,407 (0.001892%)	148,104 (0.198819%)
After Filtering		
Total reads	590,907	590,907
Total bases (bp)	66,519,925	66,480,309
Q20 bases (bp)	64,942,825 (97.6291%)	63,247,713 (95.1375%)
Q30 bases (bp)	64,656,925 (97.1993%)	62,784,268 (94.4404%)
Q40 bases (bp)	1,345 (0.002022%)	148,017 (0.222648%)

Filtering Result	
Reads passed filter	1,181,814
Pass rate (%)	99.92%
Failed — low quality	934
Failed — too many N	0
Failed — too short	12
Reads with adapter trimmed	1,151,884
Adapter trimming rate (%)	97.47%
Bases trimmed (adapters, bp)	7,535,525
Duplication rate (%)	0.279854%
Insert size peak (bp)	115
Processing time	13 seconds

Notes:

Both samples retained >99.9% of reads after fastp filtering. The very high adapter trimming rates (97.0% DMSO; 97.5% OOPS) are notable and indicate the majority of read pairs have inserts shorter than the read length, causing reads to sequence into the adapter. This is consistent with the 115 bp insert size peak, which falls within a typical short-insert range for this library type.

Duplication rates are very low for both samples (0.82% DMSO; 0.28% OOPS), indicating excellent library complexity. The OOPSint sample has an even lower duplication rate than DMSOaq, consistent with enrichment for RNA-protein crosslinked fragments. Both conditions share an insert size peak of 115 bp.

Q40 base counts in R2 are unusually high compared to R1 for both samples (e.g. DMSO: R2 345,900 Q40 bases vs R1 1,196). This asymmetry likely reflects a sequencing-chemistry-specific effect on the reverse read and is not indicative of data quality issues.

Primer Removal/Size exclusion:

Date recorded: 26 February 2026

Run Information	
Run status	OK
Input	
Total input reads	774,875
Total input bases (bp)	172,960,382
Read Filtering	
Reads too short	93,904 (12.12%)
Reads too long	9,379 (1.21%)
Reads with too many N	0
Total reads removed	103,283 (13.33%)
Output reads (passed)	671,592
Pass rate (%)	86.67%
Output	
Reads with adapters trimmed (R1)	0
Quality-trimmed bases R1 (bp)	0
Output bases R1 (bp)	77,326,968
Reads with adapters trimmed (R2)	0
Quality-trimmed bases R2 (bp)	0
Output bases R2 (bp)	67,916,390
Total output bases (R1 + R2, bp)	145,243,358
Base retention (%)	83.97%

Notes:

Run completed with OK status. Of 774,875 input reads, 671,592 passed (86.67%), with 103,283 reads removed in total. Reads were lost via two routes: 93,904 reads (12.12%) were too short and 9,379 reads (1.21%) were too long. No reads were removed due to N-content.

The presence of 9,379 reads exceeding the maximum length threshold is notable — this was not observed in the equivalent U1 or U4 primer removal steps (where too_long = 0). This may indicate a greater proportion of chimeric or non-specific amplification products in the U5 DMSOaq library, or a tighter upper size threshold configured for U5. The input read count (774,875) correctly matches the output from the preceding fastp UMI extraction step, confirming correct file chaining.

BBMerge overhang removal

Date recorded: 26 February 2026

Tool: BBMerge v39.10 (tbo=true, merge=false — overlap detection only, reads written unmerged)
Cell line: HeLa | Sample: DMSOaq (S12) | Target: U5

Summary Statistics

Pair Outcomes	
Total pairs	671,592
Joined (overlapping)	660,568(98.359%)
Ambiguous	0 (0.000%)
No solution (non-overlapping)	11024 (1.641%)
Too short	0 (0.000%)
Insert Size Statistics	
Average insert size (bp)	101.0
Standard deviation (bp)	3.4
Mode (bp)	101
Insert size range (bp)	80– 138
10th percentile (bp)	100
25th percentile (bp)	100
50th percentile / median (bp)	101
75th percentile (bp)	101
90th percentile (bp)	105

Notes:

BBMerge was run in overlap-detection mode with merging set to false. Reads are identified as overlapping but written out as separate unmerged files. Run completed with ~98.4% of reads joined by overlap, leaving ~1.6% with no solution. No pairs were found to be Ambiguous or too short. The insert size average of 101 bp follows what is anticipated after cutadapt primer removal, with the insert size after FastP UMI extraction being 115 bp with the primer size removal being 14 bp. An insert size distribution was very tight, with a standard deviation of 3.4 bp. The very narrow insert size distribution is expected for U5. The average, mode, 50th and 75th percentiles were all 101 bp.

Quality Trimming and Correction by Overlap

Date recorded: 26 February 2026

Tool: fastp | Cell line: HeLa | Sample: DMSOaq (S12) | Target: U5

Summary Statistics — DMSOaq (S12)

	R1	R2
Adapter Detection		
Adapter detected	None detected	None detected
Before Filtering		
Total reads	660,568	660,568

Total bases (bp)	66,738,535	66,723,082
Q20 bases (bp)	65,025,292 (97.4329%)	65,571,430 (98.274%)
Q30 bases (bp)	64,713,467 (96.9657%)	65,336,860 (97.9224%)
Q40 bases (bp)	461 (6.91e-04%)	300,754 (0.451%)
After Filtering		
Total reads	658,714	658,714
Total bases (bp)	66,550,947	66,535,535
Q20 bases (bp)	64,903,581 (97.5247%)	65,450,861 (98.3698%)
Q30 bases (bp)	64,595,608 (97.0619%)	65,220,688 (98.0238%)
Q40 bases (bp)	487 (7.32e-04%)	300,750 (0.452%)

Filtering Result	
Reads passed filter	1,317,428
Pass rate (%)	99.72%
Failed — low quality	3,708
Failed — too many N	0
Failed — too short	0
Reads with adapter trimmed	0
Bases trimmed (adapters, bp)	0
Overlap Correction	
Reads corrected by overlap	13,615
Correction rate (%)	2.06%
Bases corrected by overlap (bp)	14,842
Library Quality	
Duplication rate (%)	93.3383%
Insert size peak (bp)	101
Processing time	27 seconds

Notes

No adapter sequences were detected by fastp auto-detection in either R1 or R2, consistent with upstream adapter removal being complete. Quality trimming retained 1,317,428 reads (99.72% pass rate), with only 3,708 reads failing due to low quality. No reads failed N-content or length filters.

Overlap correction improved 13,615 reads (2.06% of R1 reads), fixing 14,842 bases. This is a lower correction rate than observed in the U1 DMSOaq equivalent (~9.9%), likely reflecting the shorter insert size (101 bp vs 162 bp) providing less overlap signal per read pair.

The duplication rate of 93.3% is very high, substantially exceeding the U1 DMSOaq equivalent (~81.4%) and the U4 DMSOaq equivalent (~79.2%). This is expected at this stage of amplicon processing and reflects convergence of reads onto a small number of distinct U5 amplicon sequences. The insert size peak of 101 bp is consistent with the size-selected U5 amplicon library.

As observed in the fastp UMI step, R2 Q40 base counts remain markedly higher than R1 (300,750 vs 487 after filtering), consistent with a sequencing-chemistry effect on the reverse read rather than a quality issue.

Divisive Amplicon Denoising Algorithm (DADA2)

Date recorded: 26 February 2026
Tool: Dada2 | Cell line: HeLa | Sample: DMSOaq (S12) | Target: U5

Run Statistics

DADA2 Run Statistics	
Reads in	658,714
Reads out	658,714
R1 total bases used for error learning (bp)	66,550,947
R2 total bases used for error learning (bp)	66,535,535
Unique R1 sequences (error learning)	28,626
Unique R2 sequences (error learning)	25,991
Paired reads successfully merged	617,132
Unique pairings used	95
Total pairings considered	828
Total input to merging step	657,136
Merge success rate (%)	93.91%
Output file	dada2/sequence_table_with_sequences_U1.xlsx

Amplicon Length Distribution

95 total ASVs across 22 distinct amplicon lengths. Dominant length: 101 bp (33 ASVs). Secondary peak: 105 bp (20 ASVs).

Amplicon Length (bp)	Number of Unique Pairings (ASVs)
80 bp	1
82 bp	1
83 bp	2
86 bp	2
88 bp	1
89 bp	1
90 bp	1
94 bp	2
98 bp	2

100 bp	6
101 bp	33
102 bp	6
105 bp	20
106 bp	1
108 bp	1
109 bp	1
110 bp	1
111 bp	1
112 bp	2
113 bp	1
115 bp	2
116 bp	1

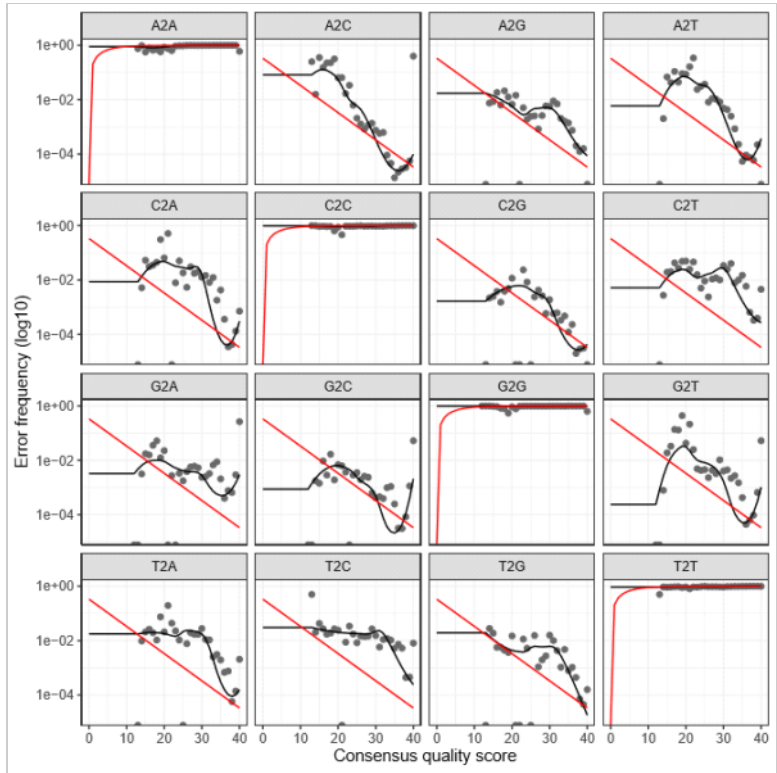
Notes

DADA2 processed 658,714 reads for the DMSOaq (S12) sample. After error-rate learning (28,626 unique R1 sequences; 25,991 unique R2 sequences), 617,132 paired reads (in 95 unique pairings) were successfully merged from 657,136 input pairs across 828 candidate pairings — a merge success rate of 93.91%.

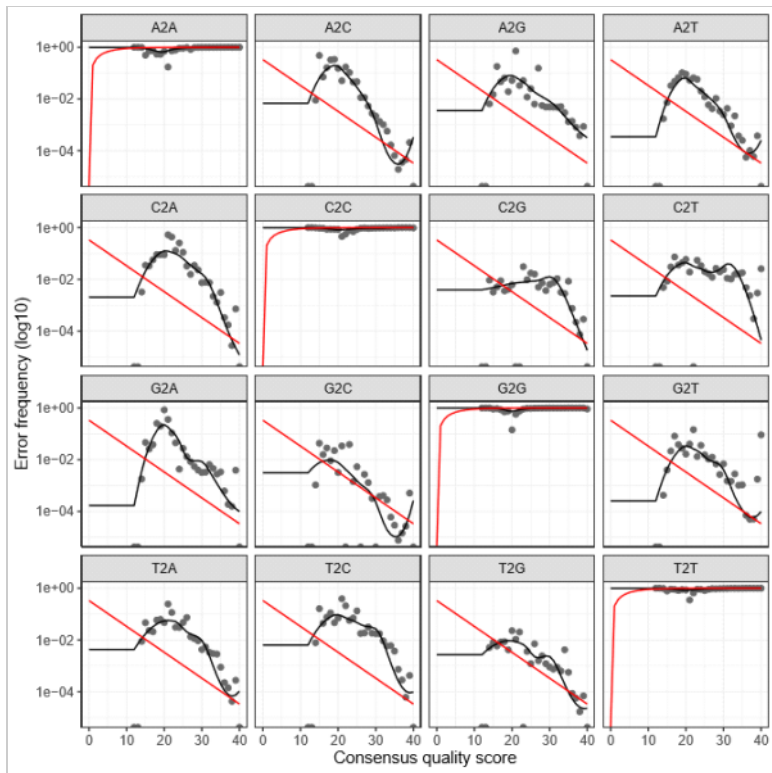
The dominant amplicon length is 101 bp (33 of 95 ASVs), consistent with the insert size peak tracked throughout the U5 pipeline. A secondary cluster at 105 bp (20 ASVs) is notable and likely represents a distinct U5 amplicon variant or isoform. The 95 unique pairings is substantially more than the U1 (63) and U4 (63) equivalents, suggesting greater sequence diversity in the U5 Sm binding region or a higher prevalence of non-specific amplification products.



sequence_t
 able_with...
 - U5 Reverse Error Plot



- U5 Forward Error Plot



Primer Reinsertion

- All primers were successfully reinserted.

U4atac

Configuration File Parameters

Samples	
U4atac_F_DMSO_aq	data/samples/HeLa-U4atac-DMSOaq_S2_L001_R10_001.fastq.gz
U4atac_R_DMSO_aq	data/samples/HeLa-U4atac-DMSOaq_S2_L001_R10_001.fastq.gz
U4atac_F_OOPS_int	data/samples/HeLa-U4atac-OOPSint_S1_L001_R9_001.fastq.gz
U4atac_R_OOPS_int	data/samples/HeLa-U4atac-OOPSint_S1_L001_R9_001.fastq.gz
Index Adapters	
Non_Transposase	data/ARL_NEBNext_adapters_No_transposease.fa
Transposase	data/ARL_NEBNext_adapters_transposease.fa
Cutadapt	
error_rate	0.2
R1_primer_length	0
R2_primer_length	16
min_length	90 bp
max_length	150 bp
fastp	
umi_len	7
quality_score_threshold	20
seqkit	
adapter_forward	CCATCCTTTCTTGGG
adapter_reverse	(not specified)

U4atac snRNA — Cutadapt Adapter Trimming (Transposase Sequences)

Date recorded: 26 February 2026

Tool: Cutadapt (paired-end mode) | Cell line: HeLa | Target: U4atac | Samples: DMSOaq (S10), OOPSint (S9)

Metric	DMSOaq (S10)	OOPSint (S9)
Run Information		
Run status	WARN	WARN
Input		
Total input reads	465,161	290,564
Total input bases (bp)	130,441,392	80,290,390
R1 Output		
Reads with adapters (R1)	46,994 (10.10%)	29,259 (10.07%)
Quality-trimmed bases R1 (bp)	0	0

Output bases R1 (bp)	65,021,399	39,987,819
R2 Output		
Reads with adapters (R2)	11,038 (2.37%)	7,335 (2.52%)
Quality-trimmed bases R2 (bp)	0	0
Output bases R2 (bp)	65,159,907	40,101,668
Total Output		
Total output bases (R1 + R2, bp)	130,181,306	80,089,487
Base retention (%)	99.80%	99.75%

Cutadapt Adapter Trimming — Index Sequences

Date recorded: 26 February 2026

Tool: Cutadapt (paired-end mode) | Cell line: HeLa | Target: U4atac | Samples: DMSOaq (S10), OOPSint (S9)

Both samples completed with OK status. No reads were lost to length or N-content filters. A reversal of the adapter rate asymmetry observed in the transposase step is present here: R2 rates exceed R1 for both conditions, consistent with index adapter orientation.

Metric	DMSOaq (S10)	OOPSint (S9)
Run Information		
Run status	OK	OK
Input		
Total input reads	465,161	290,564
Total input bases (bp)	130,181,306	80,089,487
R1 Output		
Reads with adapters (R1)	28,143 (6.05%)	17,469 (6.01%)
Quality-trimmed bases R1 (bp)	0	0
Output bases R1 (bp)	64,877,609	39,895,307
R2 Output		
Reads with adapters (R2)	37,910 (8.15%)	24,449 (8.41%)
Quality-trimmed bases R2 (bp)	0	0
Output bases R2 (bp)	64,961,045	39,968,692

fastp UMI Extraction

Date recorded: 26 February 2026

Tool: fastp (paired-end mode) | Cell line: HeLa | Target: U4atac | Samples: DMSOaq (S10), OOPSint (S9)

DMSOaq (S10)

Metric	R1	R2
Before Filtering		
Total reads	465,161	465,161
Total bases (bp)	64,877,609	64,961,045
Q20 bases (bp)	63,321,419 (97.6013%)	61,887,236 (95.2682%)
Q30 bases (bp)	62,873,362 (96.9107%)	61,283,872 (94.3394%)
Q40 bases (bp)	5,650 (0.00871%)	207 (3.19e-04%)
After Filtering		
Total reads	464,805	464,805
Total bases (bp)	58,832,297	58,794,205
Q20 bases (bp)	57,424,211 (97.6066%)	56,107,756 (95.4308%)
Q30 bases (bp)	57,062,742 (96.9922%)	55,667,353 (94.6817%)
Q40 bases (bp)	5,648 (9.60e-03%)	207 (3.52e-04%)

Filtering / Library Metric	DMSOaq (S10)
Filtering Result	
Reads passed filter	929,610
Pass rate (%)	99.92%
Failed — low quality	680
Failed — too many N	0
Failed — too short	32
Reads with adapter trimmed	874,696
Adapter trimming rate (%)	94.09%
Bases trimmed (adapters, bp)	5,656,735
Duplication rate (%)	0.965257%
Insert size peak (bp)	129
Processing time	3 seconds

Notes: Both conditions show high read quality with a shared insert size peak of 129 bp. Adapter trimming rates are very high (>94%), consistent with short-insert amplicon libraries. Duplication rates are very low (DMSOaq: 0.965257%; OOPSint: 2.33993%), confirming excellent library complexity.

OOPSint (S9)

Metric	R1	R2
Before Filtering		

Total reads	290,564	290,564
Total bases (bp)	39,895,307	39,968,692
Q20 bases (bp)	38,614,655 (96.79%)	38,181,227 (95.5278%)
Q30 bases (bp)	38,295,634 (95.9903%)	37,849,207 (94.6971%)
Q40 bases (bp)	2,606 (6.53e-03%)	19 (4.75e-05%)
After Filtering		
Total reads	285,483	285,483
Total bases (bp)	36,053,960	36,019,256
Q20 bases (bp)	35,034,424 (97.1722%)	34,603,377 (96.0691%)
Q30 bases (bp)	34,770,893 (96.4413%)	34,357,731 (95.3871%)
Q40 bases (bp)	2,604 (7.22e-03%)	19 (5.27e-05%)

Filtering / Library Metric	OOPSint (S9)
Filtering Result	
Reads passed filter	570,966
Pass rate (%)	98.25%
Failed — low quality	9,806
Failed — too many N	0
Failed — too short	356
Reads with adapter trimmed	527,912
Adapter trimming rate (%)	92.46%
Bases trimmed (adapters, bp)	3,422,511
Duplication rate (%)	2.33993%
Insert size peak (bp)	129
Processing time	7 seconds

Notes:

Both samples retained >99% of reads after fastp filtering. The very high adapter trimming rates (DMSOaq: 94.09%; OOPSint: 92.46%) are consistent with short-insert amplicon libraries where most read pairs sequence into the adapter, as observed in the U5 pipeline. Both conditions share an insert size peak of 129 bp.

The OOPSint sample showed modestly higher failure rates than DMSOaq: 9,806 low-quality reads (vs 680) and 356 too-short reads (vs 32). The OOPSint duplication rate (2.33993%) is slightly higher than DMSOaq (0.965257%), though both values indicate excellent library complexity at this stage.

Primer Removal / Size Exclusion (Cutadapt)

Date recorded: 26 February 2026

Cell line: HeLa | Target: U4atac | Sample: DMSOaq (S10) | Cutadapt

Run completed with OK status. Of 464,805 input reads, 435,013 passed (93.59%). All 29,792 removed reads failed the minimum length threshold (6.41%); no reads exceeded the maximum length, in contrast to the U5 DMSOaq sample where 9,379 reads were too long.

Run Information	
Run status	OK
Input	
Total input reads	464,805
Total input bases (bp)	117,626,502
Read Filtering	
Reads too short	29,792 (6.41%)
Reads too long	0
Reads with too many N	0
Output reads (passed)	435,013
Pass rate (%)	93.59%
Output	
Output bases R1 (bp)	56,178,090
Output bases R2 (bp)	49,182,314
Total output bases (bp)	105,360,404

BBMerge Overhang Removal

Date recorded: 26 February 2026

Cell line: HeLa | Target: U4atac | Sample: DMSOaq (S10) | BBMerge

97.0% of pairs overlapped and were trimmed by BBMerge. The insert distribution is tight (SD = 8.9 bp) and centred precisely at 113 bp across all percentiles from the 25th to the 75th, consistent with a well-size-selected U4atac amplicon.

Pair Outcomes	
Total pairs	435,013
Joined (overlapping)	422,147 (97.042%)
Ambiguous	0 (0.000%)
No solution (non-overlapping)	12,866 (2.958%)
Too short	0 (0.000%)
Insert Size Statistics	
Average insert size (bp)	113.7
Standard deviation (bp)	8.9
Mode (bp)	113
Insert size range (bp)	90 – 250
10th percentile (bp)	111
25th percentile (bp)	112

50th percentile / median (bp)	113
75th percentile (bp)	113
90th percentile (bp)	114

Quality Trimming and Correction by Overlap (fastp)

Date recorded: 26 February 2026
Cell line: HeLa | Target: U4atac | Sample: DMSOaq (S10) | fastp

No adapter sequences detected. 836,186 reads passed (99.04%), with 8,108 failing low-quality filtering. Overlap correction improved 11,282 reads (2.67%), fixing 13,181 bases. Duplication rate of 89.841% and insert size peak of 113 bp are consistent with upstream steps.

Adapter Detection		
Adapter detected	None detected	None detected
Before Filtering		
Total reads	422,147	422,147
Total bases (bp)	47,794,602	47,662,586
Q20 bases (bp)	46,964,693 (98.2636%)	45,985,091 (96.4805%)
Q30 bases (bp)	46,753,537 (97.8218%)	45,661,155 (95.8008%)
Q40 bases (bp)	5,326 (0.01114%)	198 (4.15e-04%)
After Filtering		
Total reads	418,093	418,093
Total bases (bp)	47,335,417	47,205,054
Q20 bases (bp)	46,595,372 (98.4366%)	45,740,796 (96.8981%)
Q30 bases (bp)	46,396,701 (98.0169%)	45,445,065 (96.2716%)
Q40 bases (bp)	5,320 (0.01124%)	198 (4.19e-04%)

Filtering Result	
Reads passed filter	836,186
Pass rate (%)	99.04%
Failed — low quality	8,108
Failed — too many N	0
Failed — too short	0
Reads with adapter trimmed	0
Bases trimmed (adapters, bp)	0
Overlap Correction	
Reads corrected by overlap	11,282
Correction rate (%)	2.67%
Bases corrected by overlap (bp)	13,181
Library Quality	
Duplication rate (%)	89.841%
Insert size peak (bp)	113
Processing time	31 seconds

Divisive Amplicon Denoising Algorithm (DADA2)

Date recorded: 26 February 2026
Cell line: HeLa | Target: U4atac | Sample: DMSOaq (S10) | Dada2

395,718 paired reads (94.74% merge rate) were resolved into 149 unique pairings from 417,703 input pairs across 559 candidate pairings. The dominant amplicon length is 113 bp (22 ASVs), consistent with the insert size peak tracked throughout the pipeline. With 149 unique pairings, U4atac shows substantially greater ASV diversity than U1 (63), U4 (63), and U5 (95).

DADA2 Run Statistics	
Reads in	418,093
Reads out	418,093
R1 bases for error learning (bp)	47,335,417
R2 bases for error learning (bp)	47,205,054
Unique R1 sequences (error learning)	24,613
Unique R2 sequences (error learning)	28,448
Paired reads successfully merged	395,718
Unique pairings used	149
Total pairings considered	559
Total input to merging step	417,703
Merge success rate (%)	94.74%
Output file	dada2/sequence_table_with_sequences_U1.xlsx

Amplicon Length Distribution

149 total ASVs across 64 distinct amplicon lengths. Dominant cluster: 113 bp (22 ASVs). Secondary clusters: 189 bp (12 ASVs), 188 bp (10 ASVs), 186 bp (7 ASVs). The bimodal distribution — a primary peak at 113 bp and a broad secondary distribution between ~180–210 bp — may reflect two distinct amplicon populations within the U4atac library.

Amplicon Length (bp)	Number of Unique Pairings (ASVs)	Notes
95 bp	1	
99 bp	2	
103 bp	1	
105 bp	1	
106 bp	1	
107 bp	3	

108 bp	1	
109 bp	3	
112 bp	1	
113 bp	22	Most Abundant
114 bp	1	
115 bp	2	
116 bp	1	
126 bp	3	
130 bp	1	
132 bp	2	
134 bp	2	
139 bp	1	
140 bp	1	
141 bp	1	
143 bp	1	
146 bp	1	
148 bp	1	
157 bp	1	
158 bp	1	
161 bp	1	
162 bp	2	
163 bp	1	
165 bp	1	
166 bp	1	
167 bp	1	
169 bp	1	
172 bp	2	
174 bp	2	
176 bp	1	
178 bp	1	
179 bp	1	
180 bp	2	
181 bp	1	
182 bp	1	
184 bp	4	
185 bp	2	
186 bp	7	
187 bp	2	
188 bp	10	
189 bp	12	
190 bp	6	
193 bp	1	
197 bp	3	
198 bp	3	
199 bp	1	
202 bp	1	
203 bp	2	
210 bp	4	
213 bp	2	
223 bp	1	
224 bp	1	
225 bp	3	
227 bp	1	
233 bp	3	
238 bp	2	
246 bp	1	
248 bp	1	
250 bp	1	

014 02252026-03092026 RNP-MaP, Shapemapper2, and PCA Analysis on all snRNA Data from snakemake pipeline

Thursday, February 26, 2026 2:07 PM

Objective:

- Run all snRNA samples through shapemapper2
- Analyze U1 PCA, correlation matrix, Co-variance Matrix

Future:

-

Important Notes:

Date	02/25/2026
Experiment	snRNA amplicon sequencing data analysis using Shapemapper2
Cell line	HeLa
Targets	U1, U11, U12, U4, U4atac, U5A, U5B (Sm binding region)
Purpose	Observe the binding profiles of each snRNA at the Sm binding region, analyzing profile structure, variation, and correlation between snRNA
Protocol	1. Process through ShapeMapper2 2. Process through RNP-MaP 3. Analyze using PCA a. Variation analysis b. Correlation analysis between profiles
Notes	

Results

U1 snRNA — Cutadapt (Index) Trimming QC Log

Date recorded: 12 March 2026

Cell line: HeLa | Target: U1 | Samples: DMSOaq (S12), OOPSint (S10) | Tool: Cutadapt

1. Cutadapt Adapter Trimming — Index Sequences

Both samples completed with WARN status (no reads were lost; WARN typically reflects low adapter detection rates). No reads were lost to length or N-content filters. Adapter detection rates are notably lower than observed in other snRNA targets (R1: ~0.95–0.63%; R2: ~1.89–1.35%), which may reflect differences in library composition or sequencing depth for U1.



Metric	DMSOaq (S12)	OOPSint (S10)
Run Information		
Run status	WARN	WARN
Input		
Total input reads	97,287	128,969
Total input bases (bp)	28,421,400	38,043,133
Read Filtering		
Reads too short	0	0
Reads too long	0	0
Reads with too many N	0	0
Output reads (passed)	97,287	128,969
Pass rate (%)	100.00%	100.00%
R1 Output		
Reads with adapters (R1)	923 (0.95%)	818 (0.63%)
Quality-trimmed bases R1 (bp)	0	0
Output bases R1 (bp)	14,243,397	19,066,166
R2 Output		
Reads with adapters (R2)	1,836 (1.89%)	1,742 (1.35%)
Quality-trimmed bases R2 (bp)	0	0
Output bases R2 (bp)	14,163,759	18,964,703

Notes

Both samples passed all read filters (0 reads lost to length or N-content). The WARN status flags are not indicative of data loss.

Adapter rates in this index-trimming step are substantially lower for U1 compared to U4atac and U5 samples, where R2 adapter rates reached 8–10%. This asymmetry (R2 > R1) is still present but less pronounced here.

DMSOaq (S12) had 97,287 input reads; OOPSint (S10) had 128,969 input reads. OOPSint had a higher read count at this stage, consistent with sequencing depth variation across samples.

Action: Proceed to fastp UMI extraction.

2. fastp UMI Extraction

Both conditions show high read quality with a shared insert size peak of 164 bp. Adapter trimming rates are modest (DMSOaq: 5.67%; OOPSint: 3.71%), lower than the short-insert amplicons seen in U4atac/U5 libraries ($\geq 92\%$). Duplication rates are very low (DMSOaq: 0.815%; OOPSint: 1.155%), confirming excellent library complexity. UMI length was 5 nt (per_read).

DMSOaq (S12)

Metric	R1	R2
Before Filtering		

Total reads	97,287	97,287
Total bases (bp)	14,237,167	14,145,937
Q20 bases (bp)	13,986,631 (98.2403%)	12,841,623 (90.7796%)
Q30 bases (bp)	13,899,072 (97.6253%)	12,609,011 (89.1352%)
Q40 bases (bp)	1 (7.02e-06%)	0 (0%)
After Filtering		
Total reads	97,212	97,212
Total bases (bp)	13,716,051	13,621,265
Q20 bases (bp)	13,480,093 (98.2797%)	12,408,667 (91.0978%)
Q30 bases (bp)	13,403,395 (97.7205%)	12,220,263 (89.7146%)
Q40 bases (bp)	1 (7.29e-06%)	0 (0%)

Filtering / Library Metric	Value
Filtering Result	
Reads passed filter	194,424
Pass rate (%)	99.92%
Failed — low quality	150
Failed — too many N	0
Failed — too short	0
Reads with adapter trimmed	11,042
Adapter trimming rate (%)	5.67%
Bases trimmed (adapters, bp)	51,699
Duplication rate (%)	0.815114%
Insert size peak (bp)	164
Processing time	1 second

Command:

```
fastp --thread 2 --in1 index_adapter_trimming/step2_HeLa-U1-DMSOaq-Rep1_S12_L001_R1_001.fastq.gz \
--in2 index_adapter_trimming/step2_HeLa-U1-DMSOaq-Rep1_S12_L001_R2_001.fastq.gz \
--out1 umi_removed/HeLa-U1-DMSOaq-Rep1_S12_L001_R1_001.fastq.gz \
--out2 umi_removed/HeLa-U1-DMSOaq-Rep1_S12_L001_R2_001.fastq.gz \
--umi --umi_loc per_read --umi_len 5 --umi_prefix UMI
```

OOPSint (S10)

Metric	R1	R2
Before Filtering		
Total reads	128,969	128,969
Total bases (bp)	19,042,361	18,931,507
Q20 bases (bp)	18,691,223 (98.156%)	18,180,532 (96.0332%)
Q30 bases (bp)	18,562,836 (97.4818%)	18,010,456 (95.1348%)

Q40 bases (bp)	0 (0%)	7 (3.70e-05%)
After Filtering		
Total reads	128,875	128,875
Total bases (bp)	18,365,350	18,252,373
Q20 bases (bp)	18,034,456 (98.1983%)	17,553,541 (96.1713%)
Q30 bases (bp)	17,922,343 (97.5878%)	17,420,269 (95.4411%)
Q40 bases (bp)	0 (0%)	7 (3.84e-05%)

Filtering / Library Metric	Value
Filtering Result	
Reads passed filter	257,750
Pass rate (%)	99.93%
Failed — low quality	188
Failed — too many N	0
Failed — too short	0
Reads with adapter trimmed	9,576
Adapter trimming rate (%)	3.71%
Bases trimmed (adapters, bp)	43,672
Duplication rate (%)	1.15532%
Insert size peak (bp)	164
Processing time	1 second

Command:

```
fastp --thread 2 --in1 index_adapter_trimming/step2_HeLa-U1-OOPSint-Rep1_S10_L001_R1_001.fastq.gz \
--in2 index_adapter_trimming/step2_HeLa-U1-OOPSint-Rep1_S10_L001_R2_001.fastq.gz \
--out1 umi_removed/HeLa-U1-OOPSint-Rep1_S10_L001_R1_001.fastq.gz \
--out2 umi_removed/HeLa-U1-OOPSint-Rep1_S10_L001_R2_001.fastq.gz \
--umi --umi_loc per_read --umi_len 5 --umi_prefix UMI
```

Notes

Both samples retained $\geq 99.9\%$ of reads after fastp filtering. Read quality is high across both conditions, with R1 Q30 rates exceeding 97% before and after filtering for both samples.

R2 Q30 rates are notably lower than R1 (DMSOaq: 89.1% before / 89.7% after; OOPSint: 95.1% before / 95.4% after), consistent with typical R2 quality decline in paired-end sequencing. The DMSOaq R2 Q30 drop is more pronounced and may warrant monitoring in downstream steps.

The insert size peak of 164 bp is larger than the 129 bp observed for U4atac, reflecting longer U1 amplicon inserts. Adapter trimming rates (3–6%) are substantially lower than in U4atac/U5 ($\geq 92\%$), indicating these U1 libraries are not short-insert amplicon-dominated.

The OOPSint sample showed slightly higher duplication (1.155%) than DMSOaq (0.815%), though both values confirm excellent library complexity at this stage. Note that UMI length here is 5 nt vs. 7 nt used for U4atac — verify this is intentional for U1.

Action: Proceed to primer removal / size exclusion.

015 03172026 PCA Analysis on all snRNA Data from snakemake pipeline

Tuesday, March 17, 2026 4:56 PM

Objective:

- Run PCA in python

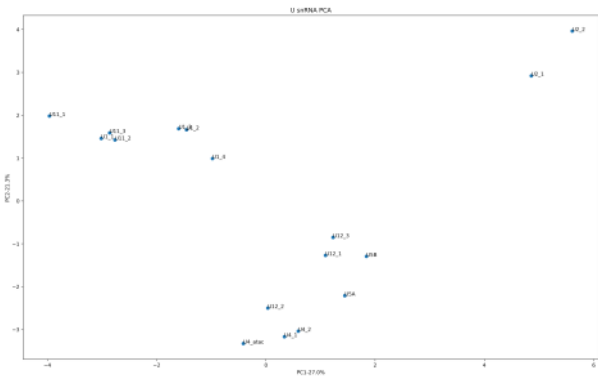
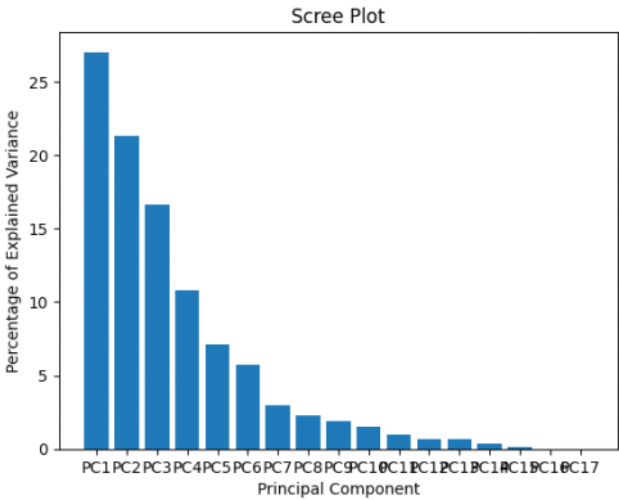
Future:

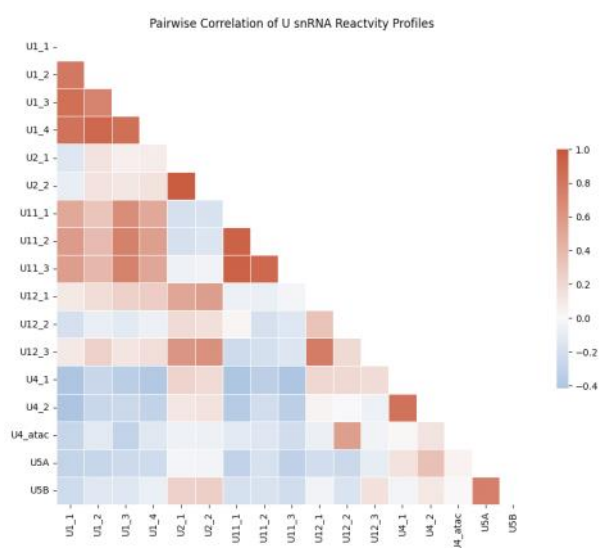
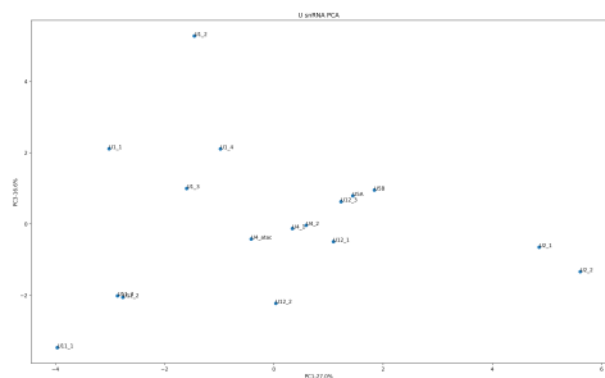
- Figure Out Clustering
- Figure out how to plot data in 3D space html
- Code:



03152026_
PCA_snRNA

Date	02/24/2026
Experiment	snRNA amplicon sequencing data analysis using Snakemake pipeline
Cell line	HeLa
Targets	U11.U12,U4,U4atac,U5A,U5B (Sm binding region)
Purpose	Observe the binding profiles of each snRNA at the Sm binding region, analyzing profile structure, variation, and correlation between snRNA
Protocol	1. Upload Data 2. Slice specific window for SM binding site (.iloc) 3. Flatten the multidimensional data into a 1-D array (.flatten) 4. Scale and Transpose data using sklearn preprocessing 5. Plot scree plot displaying percentage of explained variance 6. Plot PC space explaining majority of variance 7. Plot correlation and co-variance matrix
Notes	- Majority of variance is explained by PC1, PC2, and PC3 - Variation can be seen between replicates, need to determine if this is biological or technical variance. More data will be needed. - U11 replicates show greater variability than other snRNAs, particularly along PC3. U12 replicates also show some spread, though less extreme. Given that samples were collected across multiple dates, whether this variation reflects batch effects or true biological differences in RNP-MaP reactivity profiles require further investigation From < https://colab.research.google.com/drive/16HjHIZocB41cD-agA_UnvqkyMaqO1SIE#scrollTo=WYmviZ1KskYd >





016 03232026 RNP-MaP Step 1 -2

Monday, March 23, 2026 5:34 PM

Date	03/23/2026																																
Experiment	Step 1 and step 2																																
Cell line	HeLa																																
Targets	U1, U2,U11.U12,U4,U4atac,U5A/B (Sm binding region)																																
Purpose	Prepare samples for Illumina sequencing																																
Protocol	Materials: - Step 1: - o 2X Q5 hot start master mix - o 10 uM gene-specific forward and reverse MaP primers. - o cDNA template (1ng/uL) - Step 2: - o Purified Step 1 product - o In-house index primers (i7,i5) - Equipment: - o Thermocycler - o Magnetic stand Protocol: 1. Perform step 1: In 25 uL, mix 1-3 uL cDNA, 1.25 uL of each primer,12.5 uL of Q5 Hot-start Master Mix, and 7 uL of water 2. Incubate for specified thermocycler times 3. Clean up with 1.2X beads 4. Take concentration via Qubit 5. Image sample using Tape Station 6. Perform step 2: In 50 uL, mix 11 uL cDNA, 5 uL of each primer,25 uL of Q5 Hot-start Master Mix, and 4 uL of water 7. Incubate for specified thermocycler times 8. Clean up with 1.2X beads 9. Take concentration via Qubit 10. Image sample using Tape Station																																
Notes	Step 1 Notes <table><tr><th>Sample</th><th>F primer</th><th>R Primer</th><th>cDNA added (uL)</th></tr><tr><td>U1</td><td>36</td><td>401</td><td>3</td></tr><tr><td>U2</td><td>933</td><td>401</td><td>3</td></tr><tr><td>U11</td><td>942</td><td>401</td><td>3</td></tr><tr><td>U12</td><td>945</td><td>401</td><td>3</td></tr><tr><td>U4</td><td>936</td><td>401</td><td>3</td></tr><tr><td>U4atac</td><td>948</td><td>401</td><td>3</td></tr><tr><td>U5</td><td>939</td><td>401</td><td>3</td></tr></table> - Samples were resuspended in 15 uL - 3 uL used for quibit - 1 uL used for D1000 Tape	Sample	F primer	R Primer	cDNA added (uL)	U1	36	401	3	U2	933	401	3	U11	942	401	3	U12	945	401	3	U4	936	401	3	U4atac	948	401	3	U5	939	401	3
Sample	F primer	R Primer	cDNA added (uL)																														
U1	36	401	3																														
U2	933	401	3																														
U11	942	401	3																														
U12	945	401	3																														
U4	936	401	3																														
U4atac	948	401	3																														
U5	939	401	3																														

Step 1 Qubit concentrations

Sample	Concentration (ng/uL)
U1 D	3.378
U1 S	3.027
U2 D	3.451
U2 S	1.225
U11 D	0.453
U11 S	0.169
U12 D	0.461
U12 S	0.105
U4 D	1.906
U4 S	0.432
U4atac D	0.301
U4atac S	0.335
U5 D	0.816
U5 S	0.278

D1000 Tapestation



snRNA_ada
pter_Liga...



U2_snRNA_
adapter_L...

Step 2 Notes

Sample	F primer	R primer	cDNA added	Concentration (ng/uL)	Notes
U1 D	81	42	11	11.433	
U1 S	82	43	11	2.083	
U2 D	83	44	11	1.9	Redone
U2 S	84	45	11	2.984	Redone
U11 D	85	46	11	5.01	
U11 S	86	47	11	0.29	
U12 D	87	48	11	0.989	
U12 S	88	49	11	0.329	
U4 D	89	50	11	10.871	
U4 S	75	35	11	8.63	

U4atac D	76	36	11	3.002	
U4atac S	77	37	11	1.068	
U5 D	78	38	11	7.118	
U5 S	79	39	11	3.454	



snRNA_ad
apter_Lig...

Library Pool

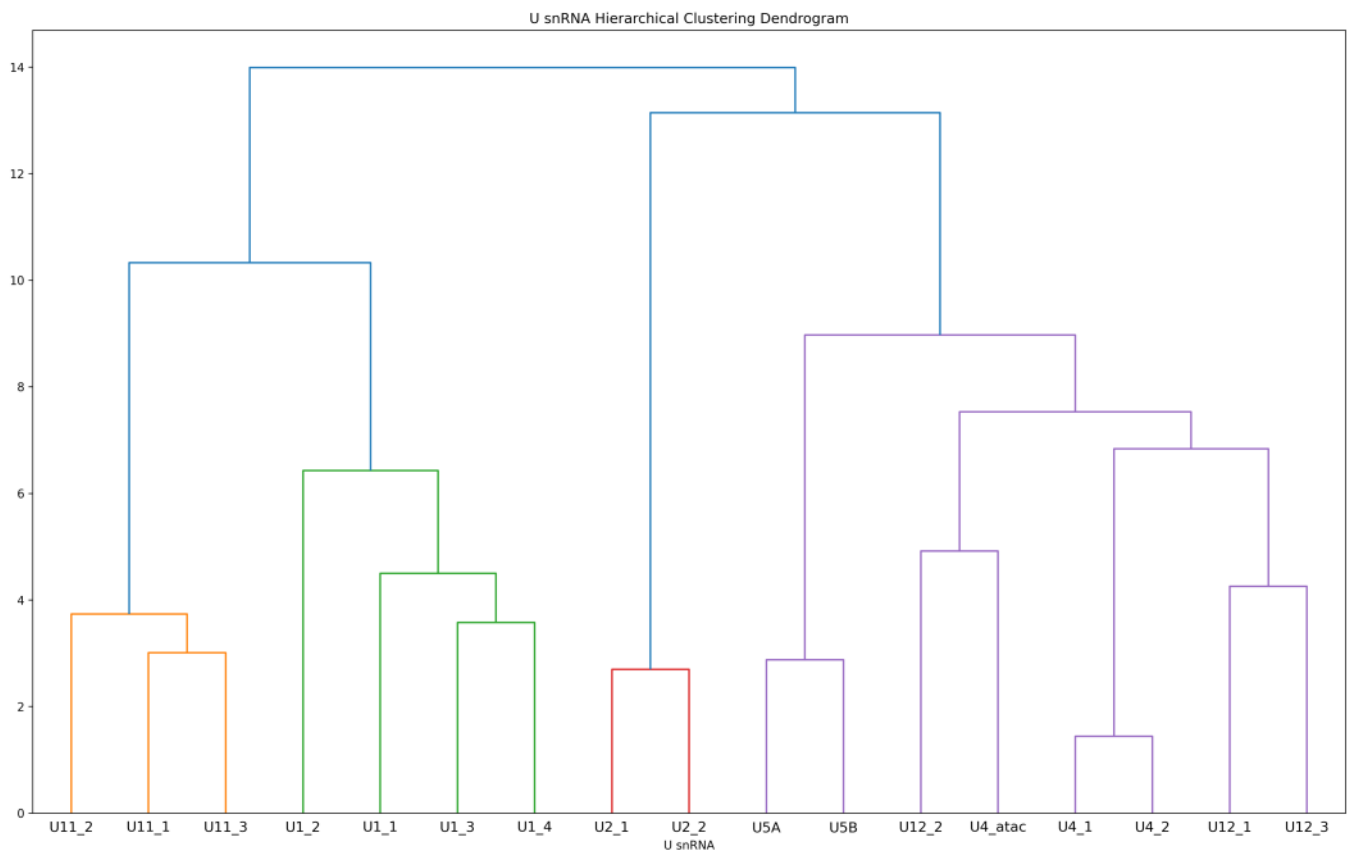
Sample	[ng/ul]	% on Tapestation	avg length	[adjusted]	nM	[in pool]	ul to pool	uL H2O to target	[target]	
U1 D	11.433	100	164	11.433	107.2	0.7142857143	0.13	4.77	10	nM
U1 S	2.083	100	164	2.083	19.5	0.7142857143	0.73		target volume	
U2 D	1.9	93	188	1.767	14.5	0.7142857143	0.99		20	uL
U2 S	2.894	100	188	2.894	23.7	0.7142857143	0.60			
U11 D	5.01	100	135	5.01	57.1	0.7142857143	0.25			
U11 S	0.29	100	135	0.29	3.3	0.7142857143	4.32	Fill in the green values for each library to mix		
U12 D	0.989	100	150	0.989	10.1	0.7142857143	1.41	Specify target pool concentration and volume		
U12 S	0.3289	100	150	0.3289	3.4	0.7142857143	4.24			
U4 D	10.871	100	185	10.871	90.4	0.7142857143	0.16	1538	average weight of a DNA nt pair	
U4 S	8.63	96	185	8.2848	68.9	0.7142857143	0.21			
U4 atac D	3.002	96	130	2.88192	34.1	0.7142857143	0.42			
U4 atac S	1.069	90	130	0.9621	11.4	0.7142857143	1.26			

	U5 D	7.118	100	117	7.118	93.6	0.7142 857143	0.15			
	U5 S	3.454	88	117	3.0395 2	40.0	0.7142 857143	0.36			

017 03252026 Hierarchical Clustering and Dendrogram Plot

Wednesday, March 25, 2026 5:30 PM

Date	03/25/2026
Experiment	Hierarchical Clustering and Dendrogram formation
Cell line	HeLa
Targets	All U snRNA samples (Sm binding region)
Purpose	Using Hierarchical Clustering to compare biological similarity of snRNP functionality to dendrogram formation
Protocol	#google Colab
Notes	Data shows that there may be some structural similarities based off how the data was sepereated. Furhter investigation will be needed



Looking at this result, I wonder if I can draw a line between the differences seen in the Sm site interaction type, the RNA structure, and the RNA function. I want to know if it 's possible to determine if RNA structure with similar structure, which is thought to have the same function, can also get entangled in that this relationship can be further highlighted by similarities in how the protein interact with it.

The sequence and the structure provide similar RNP-MaP signal which we can interpret as similarity in

function.

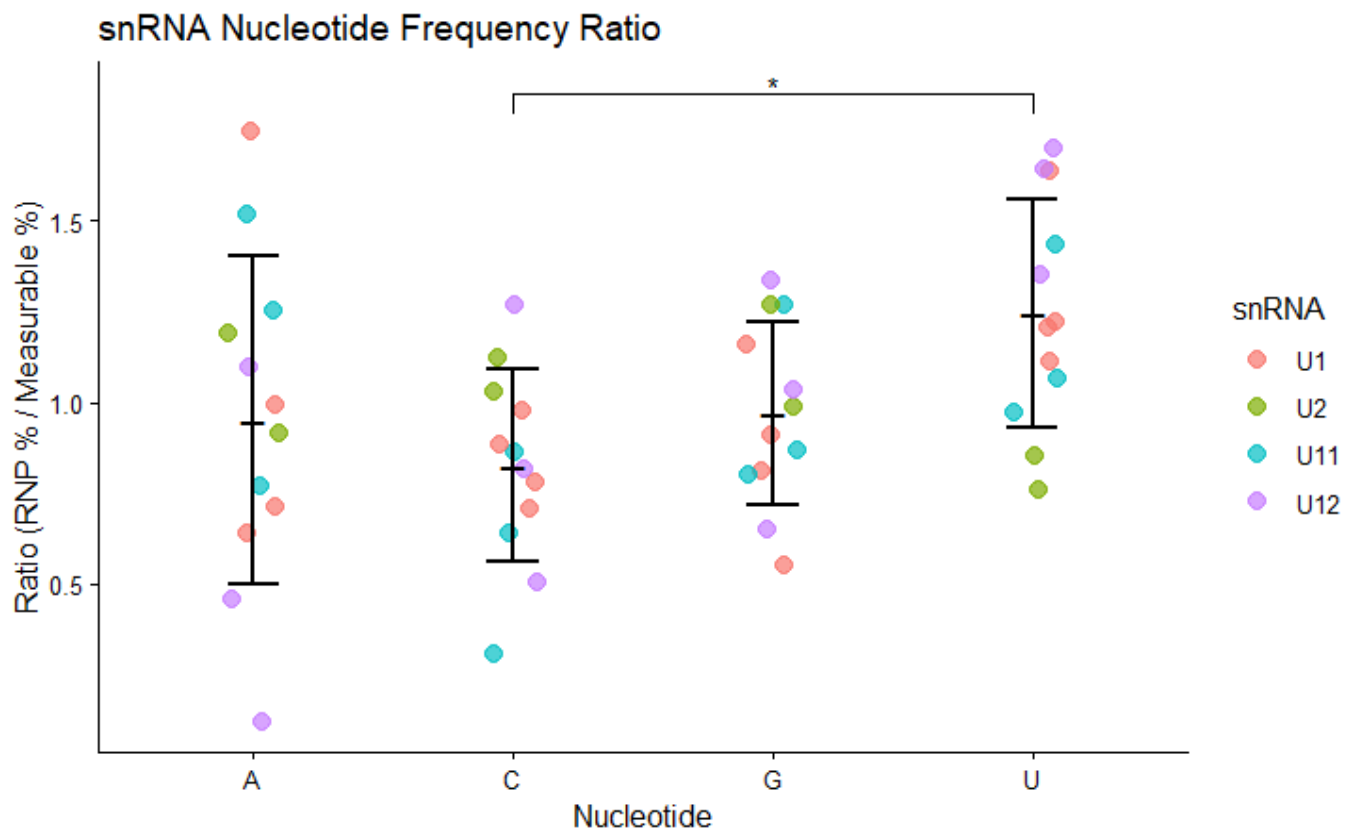
Hypothesis: RNA Secondary Structure conservation corresponds to RNP-MaP profile structure

018 03302026 Recreation of the snRNA Nucleotide Frequency Ratio

Monday, March 30, 2026

5:38 PM

Date	03/30/2026
Experiment	Nucleotide Frequency Ratio
Cell line	HeLa
Targets	All U snRNA samples (Sm binding region)
Purpose	Compare individual nucleotide rates
Protocol	#R script  snRNA_nuc leotide_r...
Notes	Kruskal-Wallis test was used to compare samples (ranked sum) with a Dunn's multiple comparisons test with a Bonferroni correction

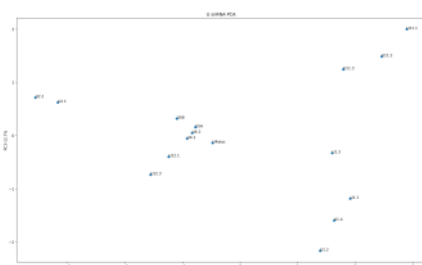
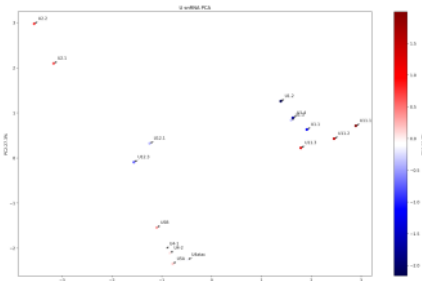
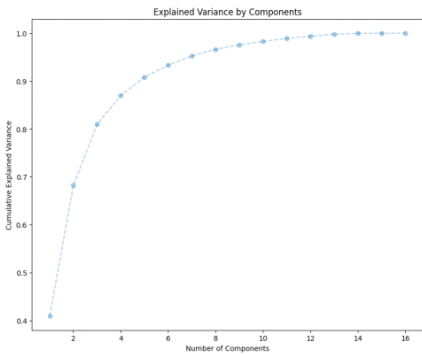
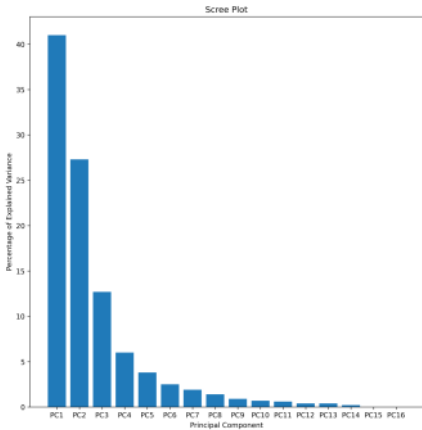


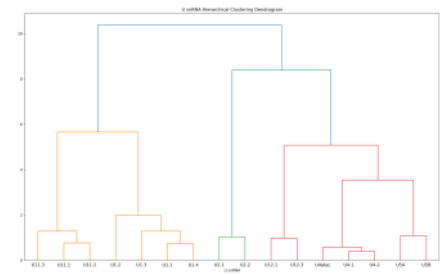
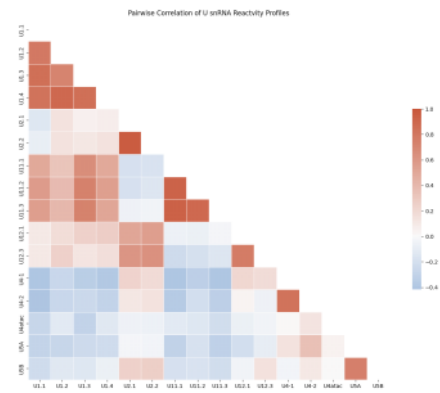
For the month of April, I have made many revisions to code that have been posted on my GitHub. This includes performing different forms of dimensionality reduction, PCA and correlation analysis. These results will be posted below.

Version control can be found on github

Date	04/2026
Experiment	Nucleotide Frequency Ratio
Cell line	HeLa
Targets	All U snRNA samples (Sm binding region)
Purpose	Determine relationships between snRNA signals
Protocol	Overview Scientific question: Can we determine groupings and separations of snRNA based on their nucleotides? Samples: All U snRNA sequencing data to date. Experimental design: Perform dimensionality reduction using Principal Component Analysis and Hierarchical Clustering. Expected outcome: This experiment will separate all U snRNA along different PCs, showing us similar groupings based around biological function (homologs) and structure. Steps: 1. Load in all datasets 2. Splice out desired nucleotide window frame 3. Run Principal Component Analysis (PCA) on all data and define inclusive principal components for Hierarchical Clustering 4. Analyze output with respect to biological significance Jupyter Notebook script Github: PCA and Hierarchical Clustering Analysis Analyzing how identifiability of the data
Methods	Data Preparation: All U snRNA sequencing datasets were used for this experiment (RNP-MaP 2024). Feature consists of normalized nucleotides within the defined window frame. Principal Component Analysis (PCA): PCA was performed on the nucleotide frequency matrix to identify principal components capturing maximum variance. Scree plot analysis determined the number of informative components for downstream clustering analysis. PCA input was standardized by centering (zero mean) without scaling, since nucleotide frequencies are already bounded between 0 and 1. Hierarchical Clustering: Ward linkage hierarchical clustering was applied to the PCA-reduced feature space, using Euclidean distance as the metric. The resulting dendrogram was visualized to identify natural groupings and assess the clustering structure relative to known snRNA biological classifications.
Results	Principal Component Analysis Variance Structure Scree plot analysis reveals that the nucleotide frequency data is dominated by a single, highly informative principal component. PC1 explains approximately 40% of the total variance, substantially more than any other component. PC2 explains ~28% of variance, and PC3 explains ~12%, with these three components together accounting for approximately 80% of the total variation. The cumulative explained variance plot shows diminishing returns after PC3, with the curve plateauing at approximately 95% explained variance by PC6. This data indicates that the differences among U snRNA species are relatively low-dimensional, meaning they are primarily captured by three dominant directions of variation. PCA: snRNA Separation The U snRNA PCA plot(PC1 vs. PC2) reveals clear separation of snRNA species in the principal component space. U2 replicates (U2.1, U2.2, U2.3) cluster tightly together in the upper left quadrant (positive PC2, negative PC1), indicating similarity in profiles. U1 replicates and U4 variants occupy distinct regions, with U1 samples distributed across negative PC1 values and U4 samples showing wider variance along PC3. Overall, it appears that U1:U11, U2, and all other snRNAs fall into individual groups respectively The secondary U snRNA PCA plot (PC1 vs. PC3) provides complementary separation, with clearer segregation of minor spliceosome components (U4atac, U11, U12) from major spliceosome snRNAs. U11.1 and U11.2 occupy the upper right region with high positive PC2 values, suggesting distinctive nucleotide frequency patterns characteristic of the minor spliceosome. This plot confirms that nucleotide composition is informative for discriminating among diverse U snRNA species. Pairwise Correlation of snRNA Reactivity Profiles The pairwise Spearman correlation matrix (ranked based) of U snRNA profiles demonstrates strong positive correlations within snRNA subgroups. U1 replicates and U11 show high correlations (0.8–1.0 range, red colors), consistent with shared structural and functional roles in the spliceosomes. Similarly, U2 and U12 also show similarities and U4/U5-family samples cluster into correlation blocks. Hierarchical Clustering Dendrogram Hierarchical clustering with Ward linkage on the PCA-reduced nucleotide frequency data produces a dendrogram with striking biological coherence. The dendrogram reveals four major clades: • U1 and U11: U1.1 and U1.3 form a tight subclade, with U1.2 and U1 slightly more distant but within the same major clade. • U2: U2.1, U2.2, U2.3 cluster together reflecting a consistent snRNA signal. Considering it lies within its own clade, it indicates that U2 may contain a unique profile. • U4/U5/U12: All snRNA fall in the same clade despite differences in non Sm sequence and structure the snRNA. Discussion and Future Directions These results demonstrate the ability of RNP-MaP to serve as a tool for predicting snRNA-Sm complex interactions. Within replicates, each sample outputs similar profiles, demonstrating consistency and the potential to identify specific snRNA signals from the vast amount of RNP-MaP data. When examining the pairwise correlation analysis, we observe similarities in profile structure among homologous snRNAs in both the major and minor spliceosome. This correlation is most evident when comparing (U2, U12) or (U1, U11) pairs. It is important to note that these features are separated in the PCA analysis. The key difference between the two analyses—PCA and correlation matrix—is that the correlation matrix shows the monotonic relationship between samples (i.e., do they move in the same direction?), while PCA incorporates magnitude, which can spread apart samples that have proportionally similar patterns. Additionally, sequence differences may directly affect snRNA profiles, separating profiles that could otherwise be similar but are compressed or stretched due to sequence variation. K Means Clustering paired with Dynamic time warping with barycenter averaging could be applied to account for these distortions, providing a more accurate understanding of the underlying snRNA binding patterns and revealing each snRNA signal. Performing dimensionality reduction with the SM sites and Non-Sm sites could also provide insight whether SM sites can stand out from non-Sm sites.
Notes	

Results:





Date	05/2026
Experiment	Multivariate Analysis of snRNA Data
CellLine	HeLa
Targets	All U snRNA samples (5m binding region)
Purpose	Data Quality Assessment
Experimental Design	Approach: Perform Shapiro-Wilk normality testing and Q-Q plot visualization on all samples. Given non-normality, apply non-parametric variance analysis (Kruskal-Wallis test) to assess whether snRNA groups differ significantly. Follow with Dunn's post-hoc test and analyze ranking of measured variance across samples. GitHub: Exploring how identifiability of the data

Methods:

Shapiro-Wilk Normality Test: Applied to each snRNA sample's Normalized_Score vector to test the null hypothesis that data are normally distributed. W statistic (Q=1 range) and p-value computed, p < 0.05 indicates rejection of normality.

Q-Q Plot Analysis: Quantile-Quantile plots generated for visual assessment of departures from normality. Comparison of observed quantiles vs. theoretical normal quantiles; deviations indicate non-normality.

Kruskal-Wallis Test: Non-parametric test for comparing distributions across groups. Applied to assess whether snRNA samples show significant differences in their overall reactivity distributions. H statistic and p-value computed, p < 0.05 indicates significant group differences.

Dunn's Post-Hoc Test: Pairwise comparisons with Benjamini-Hochberg FDR correction to identify specific snRNA pairs that differ significantly. Results presented as a matrix of TRUE/FALSE indicating significant pairwise differences.

Ranked Plot Analysis: Measured variance computed for each sample's Norm_Score profile. Cumulative frequency analysis performed to rank samples by variance magnitude.

Results:

Normality Analysis:

Shapiro-Wilk Test Results:

The Shapiro-Wilk normality test was performed on all 16 snRNA samples. All samples rejected the null hypothesis of normality. W statistics ranged from 0.483 (U4-1) to 0.792 (U2B), with all p-values $< 1 \times 10^{-10}$, indicating severe departure from normality across the entire dataset.

Notable findings: U4-1 showed the most extreme non-normality ($W = 0.483$, $p < 2.71 \times 10^{-10}$), while U2B showed the least extreme ($W = 0.792$, $p = 1.88 \times 10^{-10}$). However, even the 'best' sample deviates significantly from normality, establishing that non-normality is a fundamental property of RNP-MAP reactivity data across all snRNA types.

Q-Q Plot Analysis:

Q-Q plots were generated for all 16 samples and display consistent patterns of non-normality. The data exhibits an S-shaped curvature, departing from the theoretical normal line. One important characteristic is the left tail: The data is zero inflated as a result of the RNP-MAP pipeline transformation. When fit to the RANSAC model, the RNP-MAP data that fell below the line, containing negative residuals, was transformed to be zero as the negative values hold no real biological information.

"Note: Negative data would mean that the SDA resulted in less mutations which does not align with experimental outcomes as SDA only increases mutation rates at individual sites. This would also be a statistical anomaly given random change to multiple sites is expected. This is purely a result of the RANSAC model's data fitting and the threshold requirement we are using."

Variance Analysis (Non-Parametric)

Kruskal-Wallis Test Results

Test Statistics: W statistic = 23.764, p-value = 0.0692

The Kruskal-Wallis test assessed whether snRNA samples differ significantly in their overall distribution properties. With $p = 0.0692$, the result approaches but does not reach traditional significance at $\alpha = 0.05$. This marginal p-value suggests that while snRNA samples show some variance in their distributions, no definitive statistical evidence supports significant group differences when controlling for Type I error at the conventional 0.05 threshold.

Dunn's Post-Hoc Pairwise Comparisons

Pairwise comparisons between all 16 snRNA samples (120 unique pairs) with Benjamini-Hochberg FDR correction revealed no statistically significant differences between any pair. The post-hoc comparison matrix shows all FALSE entries, indicating that after FDR correction, no individual pairwise comparisons reach significance at $\alpha = 0.05$.

This conservative result reflects the stringent FDR correction applied to multiple comparisons and is consistent with the marginally non-significant Kruskal-Wallis omnibus test. While visual inspection of variance magnitudes shows some sample-to-sample variation, the formal statistical tests do not detect pairwise differences that survive multiple-comparison correction.

Ranked Variance Analysis

Despite non-significant group differences, individual samples show substantial variation in measured variance (SD of Norm_Score within each sample).

Highest Variance: U2-2 (1.132), U2-1 (0.821), U11-1 (0.710)—major spliceosome snRNAs showing heterogeneous reactivity across the binding region

Intermediate Variance: U1-2 (0.677), U2B (0.254), U4 (0.183)—wide range reflects different snRNA structures

Lowest Variance: U4-1 (0.079), U4-2 (0.068), U4atoc (0.114)—minor spliceosome snRNAs showing uniform reactivity, possibly reflecting more constrained structure or function

Cumulative Frequency Analysis of Pearson Correlations

A cumulative distribution analysis was performed on all pairwise Pearson correlations of snRNA reactivity profiles. Updated statistics (05/20/2026):

- > 1.975% of all pairwise correlations fall below the lowest highlighted pair.
- > 99.98% of all pairwise correlations fall above the highest highlighted pair.

The distribution reveals that the vast majority (99.98%) of snRNA pairwise comparisons show high correlation in their reactivity profiles, indicating that snRNA 5m binding patterns are broadly similar across species despite sequence differences. Only the most extreme outliers (bottom 1.975%) show notably low correlations, suggesting that most snRNAs share conserved 5m binding topologies.

- Discussion:**
- Universal Non-Normality:** All 16 snRNA samples exhibit severe departure from normality ($p < 10^{-10}$), confirming that zero-inflation and positive skew are fundamental properties of RNP-MAP reactivity data, not experimental artifacts.
 - No Statistically Significant Distributional Differences:** The Kruskal-Wallis test ($p = 0.069$) and subsequent Dunn's post-hoc tests (all FALSE) provide no evidence that snRNA species differ significantly in their distribution properties after FDR correction. This suggests that the underlying distributional mechanisms are conserved across snRNA species.
 - Variable Within-Sample Variance:** Despite equivalent distributional properties, individual samples show 10-fold variation in measured variance (0.079 to 1.132), with U2 variants and U11.1 showing high variance, while U4 variants show low variance. This may reflect structural differences in 5m binding site accessibility.
 - High Pairwise Correlation:** The cumulative distribution of Pearson correlations indicates that 99.98% of snRNA pairwise comparisons are highly correlated, suggesting that 5m binding topology is conserved across snRNA species despite sequence variation.

Biological Interpretation

The lack of significant distributional differences across snRNA species despite differences in measured variance suggests that snRNA 5m protein interactions are highly conserved in their overall structure and specificity. The variations in within-sample variance may reflect secondary structure accessibility rather than fundamental differences in 5m binding mechanisms. The high pairwise correlations support a model where all snRNA species adopt similar core 5m binding topologies, with variations in local structure modulating accessibility and reactivity intensity.

The confirmation of universal non-normality validates the selection of non-parametric statistical methods for all downstream analyses and supports the use of machine learning approaches that make no distributional assumptions.

Future Directions

Normality Analysis

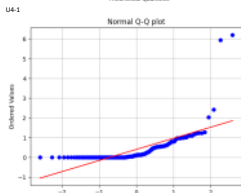
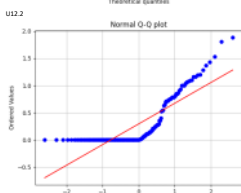
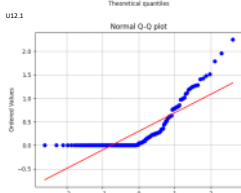
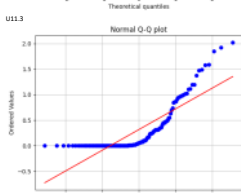
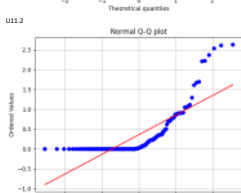
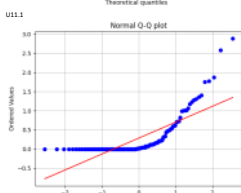
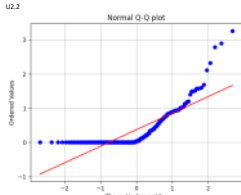
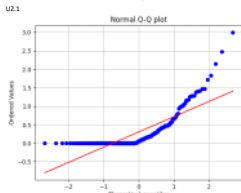
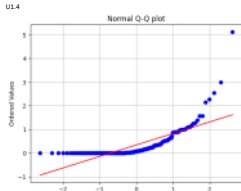
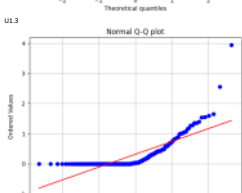
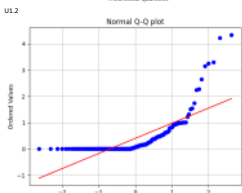
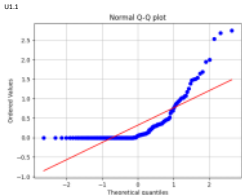
Shapiro-Wilks Test:

Index	Sample	W Statistic	P-value	Result
0	U1.1	0.546	2.69E-10	Deviates from normal
1	U1.2	0.577	2.69E-20	Deviates from normal
2	U1.3	0.634	1.44E-18	Deviates from normal
3	U1.4	0.575	1.14E-19	Deviates from normal
4	U2.1	0.667	5.16E-19	Deviates from normal
5	U2.2	0.607	1.77E-18	Deviates from normal
6	U11.1	0.61	6.50E-17	Deviates from normal
7	U11.2	0.65	4.97E-16	Deviates from normal
8	U11.3	0.707	6.71E-15	Deviates from normal
9	U12.1	0.694	2.85E-16	Deviates from normal
10	U12.2	0.711	7.76E-16	Deviates from normal
11	U4-1	0.483	2.71E-20	Deviates from normal
12	U4-2	0.772	1.35E-13	Deviates from normal
13	U4atoc	0.648	3.75E-18	Deviates from normal
14	USA	0.734	3.87E-13	Deviates from normal
15	USB	0.792	1.86E-11	Deviates from normal

Results: Shapiro-Wilks test was performed on all snRNA sequences. Shapiro-Wilks test rejects the null hypothesis for all samples, signifying the data significantly departs from normality.

Note: Dot-notation indicates replicate samples

Q-Q Plots:



Results: The results of the Q-Q plot shows similar results to the Shapiro-Wilks test. The data do not follow a normal distribution.

Variance Analysis (non-parametric)

Wanting to compare the variance of profiles, the data does not fit the assumptions required to perform a multivariate analysis of variance (ANOVA). I will perform a Dunn's test for Post-hoc test.

Kruskal-Wallis and Dunn's Post-hoc Test:

Output: H stat = 23.76352928475488 and P-value = 0.0692110961600476

Table:	1	2	3	4	5	6	7	8	9	10	11	12	13
1	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
2	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
3	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
4	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
5	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
6	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
7	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
8	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
9	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
10	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
11	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
12	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
13	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
14	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
15	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
16	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE

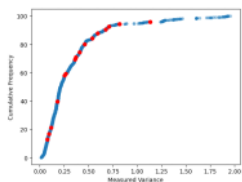
Result: Looking across all the snRNA sequences, there is no difference between the distributions of the samples.

Ranked Plots:

Ranked Variance plot

SM-Site = Red dot

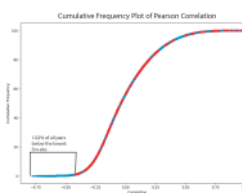
Non SM-site = Blue dot



Looking specifically at the 5m site sequences

Index	Sample	Measured Variance	Cumulative Frequency
0	U1.1	1.18142	0.459592
1	U1.2	1.18142	0.677085
2	U1.3	1.18142	0.408727
3	U1.4	1.14138	0.594195
4	U11.1	80.104	0.710297
5	U11.2	80.104	0.537362
6	U11.3	78.162	0.372123
7	U12.1	66.90	0.36106
8	U12.2	66.90	0.266357
9	U2.1	90.114	0.821001
10	U2.2	90.114	1.13163
11	U4-1	110.134	0.079377
12	U4-2	110.134	0.096185
13	U4atoc	97.121	0.114009
14	USA	80.104	0.182755
15	USB	80.104	0.25403

Cumulative distribution plot of Pearson correlations



Updated stats (05/20/2026): 1.975084067020941% of all pairs fall below the lowest highlighted pair 99.998484249905% of all pairs fall above the highest highlighted pair.

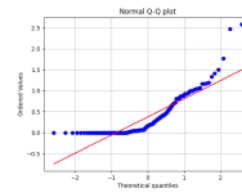
fit to the line, representing a normal distribution, indicating the data deviates from a normal

multivariate ANOVA (MANOVA). So, I will be performing a Kruskal-Wallis analysis to determine how the analysis.

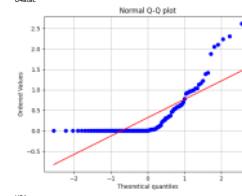
[illegible]

mples.

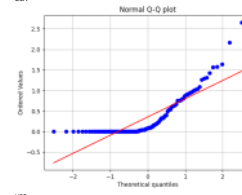
not highlighted pair



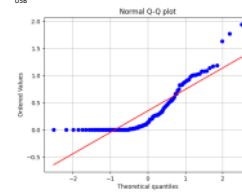
Ukraine



USA



US\$



021 05/2026 UMAP analysis on SM and Non-SM Sites

Saturday, May 9, 2026 6:56 PM

Date	05/2026
Experiment	Nucleotide Frequency Ratio
Cell line	HeLa
Targets	All U snRNA samples (Sm binding region)
Purpose	Compare individual nucleotide rates
Protocol	Overview Scientific question: Given the variance seen in previous U snRNA data, can we isolate the U snRNA data from other sequences? Samples: All U snRNA sequencing data to date. Experimental design: To set up this experiment, I will be isolating all possible 16 nt combinations of snRNA data by creating a sliding window. Expected outcome: This experiment will group together all U snRNA data, proving that the Sm binding region is unique and identifiable in a larger dataset. In this document, I will be creating a dataset consisting of 16 nucleotide chunks of snRNA RNP-MaP data. To do this, starting at the beginning of every sequence, for every 16 nucleotides, create its own data frame consisting of all RNP-MaP data, labeled with a unique name. After each window is created, dimensionality reduction will be used to assess variation. Steps: 1. Create function that uses a sliding window to get all possible 16 window combinations and label each one based on sequence identity. 2. Apply function across all datasets. 3. Run Principal Component Analysis (PCA) on all data and define inclusive principal components. 4. Due to increased data complexity, Uniform Manifold Approximation and Projection (UMAP) will be used. GitHub: Analyzing how identifiability of the data
Results	
Future Directions	

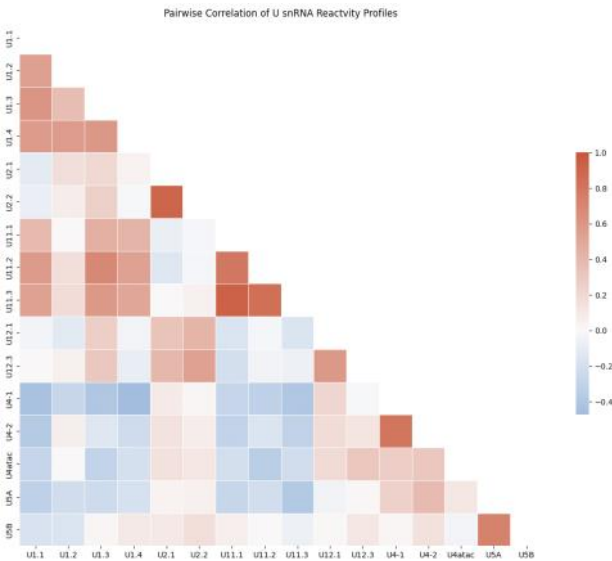
022 05/2026

Using DTW to observe effects of sequence differences

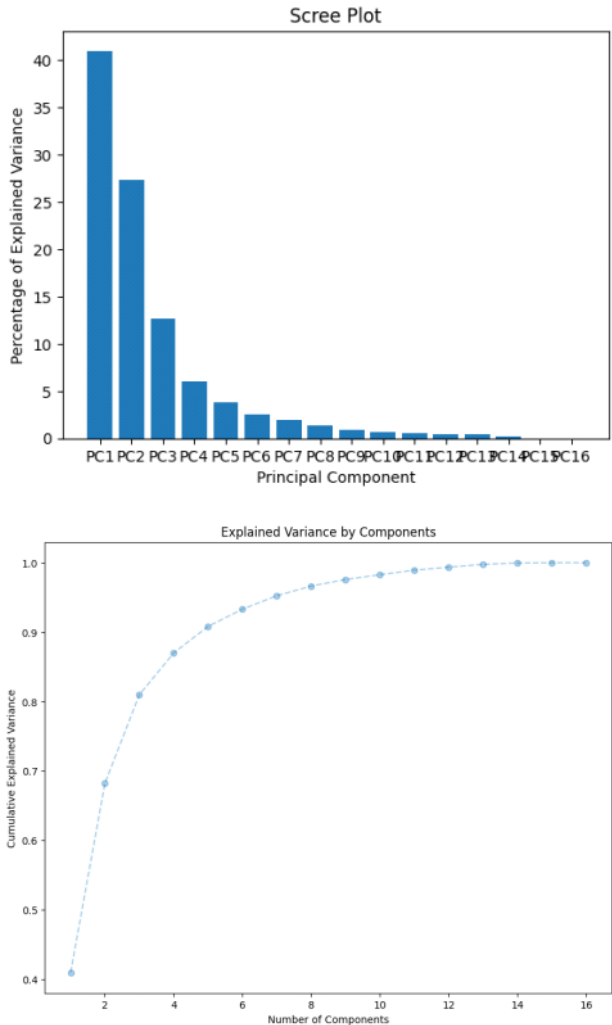
Friday, May 22, 2026 12:57 PM

Date	05/2026
Experiment	Using DTW to observe effects of sequence differences
Cell line	HeLa
Targets	All U snRNA samples (Sm binding region)
Purpose	Investigate the effects of sequence differences on the interactions of the lysine residues with different RNA. Determine whether Dynamic time warping can be used to account for these sequence differences that make the profiles appear more dissimilar despite visual similarities.
Protocol	Jupyter Notebook script GitHub: kmeans clustering and dynamic time warping 1. PCA 2. K-Means Clustering 3. Group clusters together 4. Perform DTW with DBA 5. Compare change in profiles via PCA and spearman correlation
Results	Spearman Correlation Analysis Spearman correlation analysis of the normalized snRNA profiles reveals correlation patterns consistent with previous experiments. The correlation heatmap shows expected clustering patterns within and between snRNA species, with stronger correlations observed for profiles within the same snRNA type. Principal Component Analysis Scree plot analysis shows that the first three principal components capture the majority of the variance in the data. PC1 explains approximately 41% of the variance, PC2 explains ~28%, and PC3 explains ~13%, together accounting for over 82% of total variation. The cumulative variance plot confirms that diminishing returns in explanatory power occur after PC3, suggesting that the underlying snRNA-lysine interaction landscape is primarily three-dimensional. The PCA biplot reveals three distinct clusters in the principal component space. PC1 appears to represent overall signal magnitude, while PC2 and PC3 capture shape-related variations. The color-coded clusters show clear separation, validating the K-means clustering solution (k=3) determined from prior hierarchical clustering analysis. DTW and DBA Before DTW application: Individual profiles within each cluster show substantial temporal variability and peak position shifts, despite maintaining similar overall shapes. This variability reflects both true biological differences and technical variations in how sequence differences manifest across different snRNA species. After DTW with DBA: Cluster 0, Cluster 1, and Cluster 2 profiles converged to smoother, more coherent consensus curves after temporal warping. The post-warped profiles demonstrate dramatically reduced noise and enhanced visualization of the underlying binding patterns. Cluster 0 and Cluster 1 show distinct multi-peak structures, while Cluster 2 exhibits a more complex profile with broader peaks. Critically, post-DTW PCA reveals that more variance is now concentrated in the first two principal components compared to the pre-DTW analysis. This redistribution indicates that DTW successfully aligned phase-shifted variations, allowing the PCA to better capture the intrinsic shape differences between clusters while suppressing artificial dissimilarity due to temporal misalignment. Discussion The DTW analysis successfully disentangled shape-based profile differences from sequence-driven variations. By aligning profiles to their cluster-specific centroids, DTW accounts for the hypothesis that sequence differences in U snRNAs may cause phase shifts in lysine binding residues interacting with the DNA

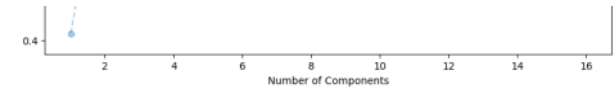
Spearman correlation analysis of different snRNA profiles show similar correlations to previous experiments



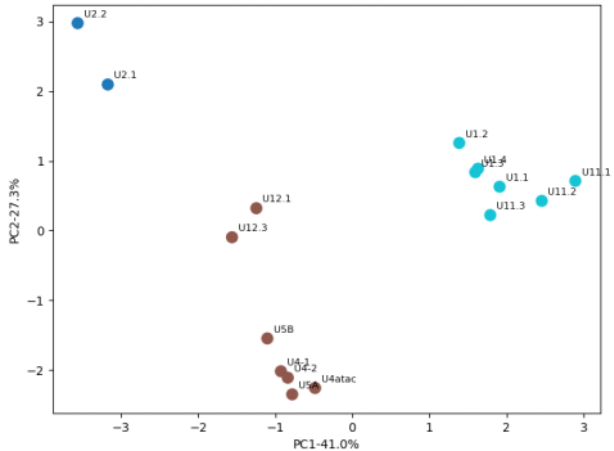
Scree Plot and explained variance by components plot shows that the majority of the variation is in the first 3 PC's.



	based profile differences from sequence-driven variations. By aligning profiles to their cluster-specific centroids, DTW accounts for the hypothesis that sequence differences in U snRNAs may cause phase shifts in where lysine residues interact with the RNA, while preserving the fundamental topology of Sm protein binding. The improved variance concentration in post-DTW PCA strongly supports the idea of the presence of similar binding profiles within SM Protein Binding regions.
Future Directions	Continue to create machine learning model Curate Train:Test Split datasets

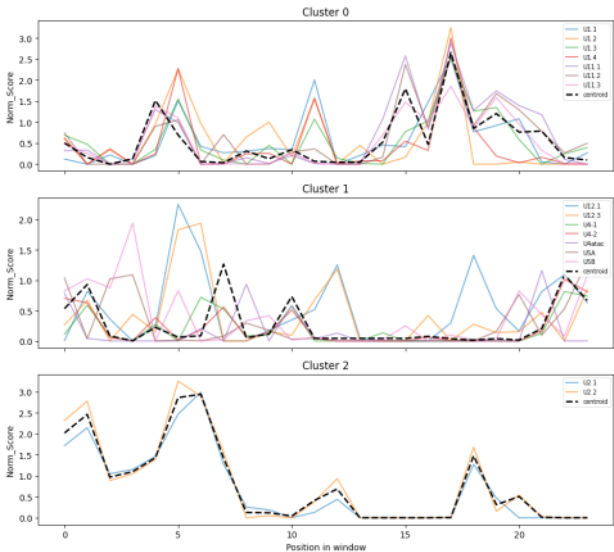


Pre DTW PCA: This PCA was standardized by subtracting the mean but not dividing by the standard deviation as the data was already normalized and scaled so that every feature is between 0 and 1. Each sample was then clustered using K means clustering, specifying 3 clusters based off previous hierarchical clustering data. The clusters are identifiable by color.

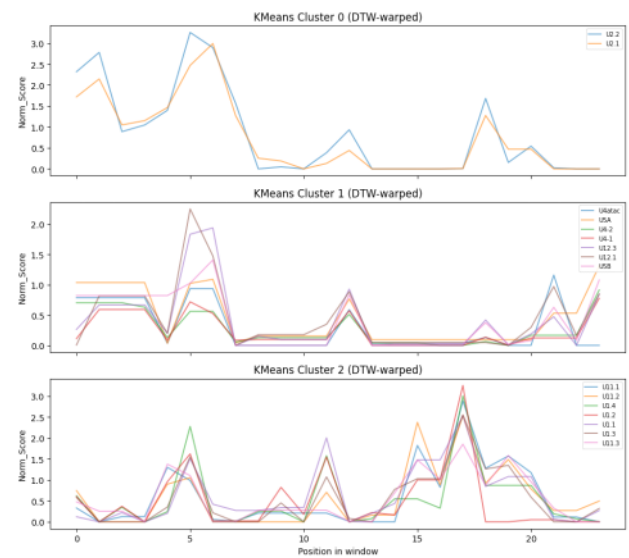


DTW was performed on each cluster warping each cluster based off of their centroid. DTW creates a distance matrix based off of the relationship between each sample and their centroid that was defined using Dynamic Time Warping Barycenter Averaging (DBA) to create time warped curves.

Before DTW with DBA:

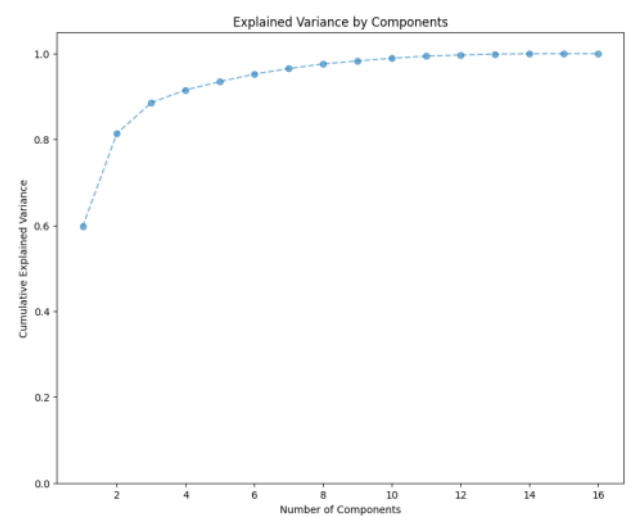
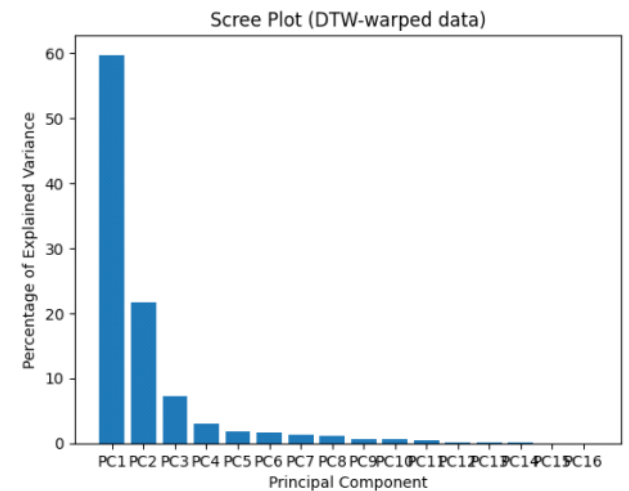


After DTW with DBA:

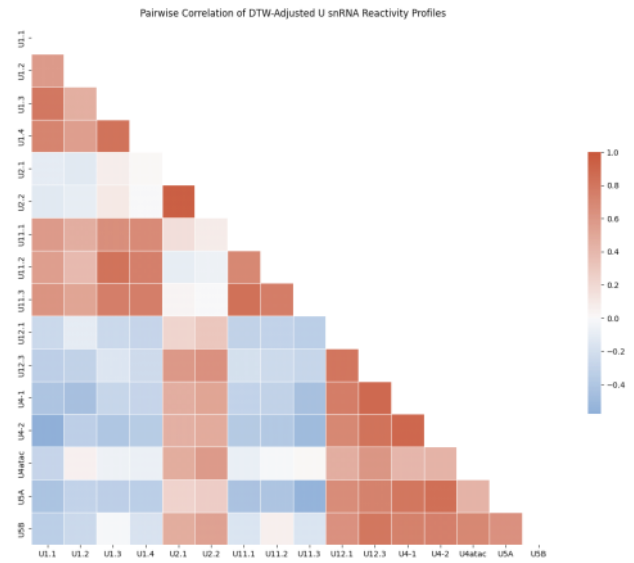
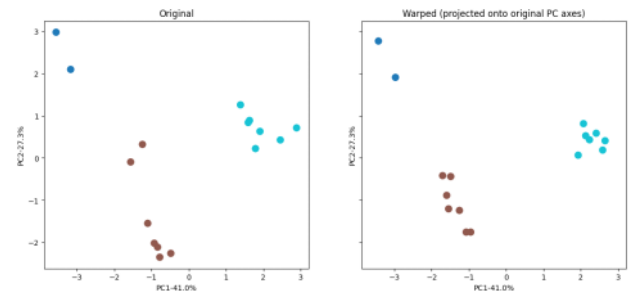


DTW adjusted data:

More variance is now explained in the first 2 PCs



Side by side comparisons of PCA data. The DTW data is now projected on the original PCA via change of basis. This projection allows for a more direct comparison between experiments.



Future Directions

- PCA analysis
- Hierarchical clustering
- Tabular Data set curating for ML model



HeLa-U4_D
MSO_aq_...



HeLa-U4_D
MSO_aq_...

U4-2
Shapemapper2:



HeLa-U4_D
MSO_aq_...



HeLa-U4_D
MSO_aq_...



HeLa-U4_D
MSO_aq_...

U4atac
Shapemapper2:



HeLa-U4at
ac_DMSO...



HeLa-U4ata
c_DMSO_...



HeLa-U4at
ac_DMSO...

U5A
Shapemapper2:



HeLa-U5_D
MSO_aq_...



HeLa-U5_D
MSO_aq_...



HeLa-U5_D
MSO_aq_...

U5B
Shapemapper2:



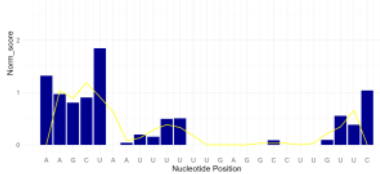
HeLa-U5_D
MSO_aq_...



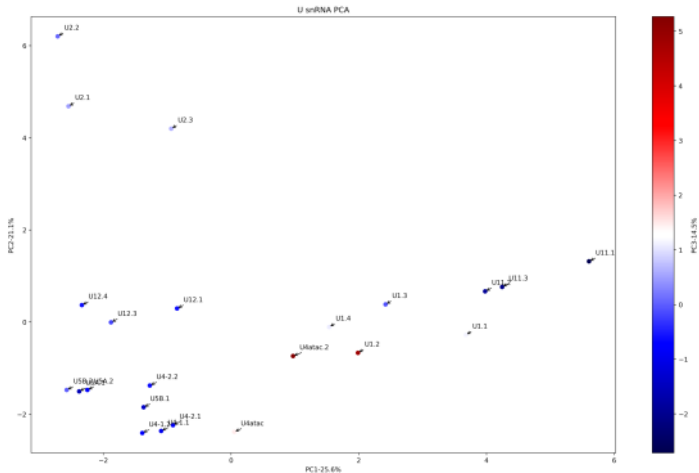
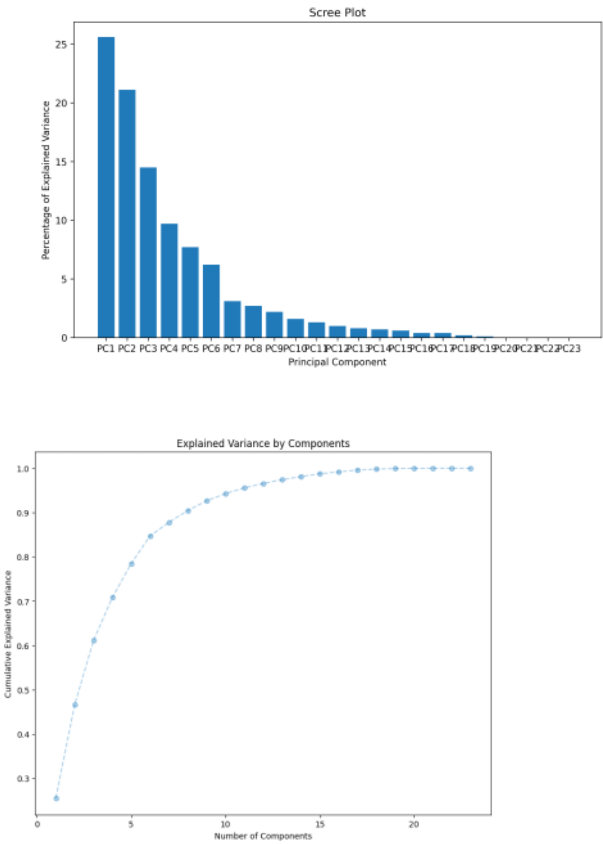
HeLa-U5_D
MSO_aq_...



HeLa-U5_D
MSO_aq_...



Date	06/03/2026
Experiment	PCA and Clustering Analysis of all snRNA data
Cell line	HeLa
Targets	All U snRNA samples (Sm binding region)
Purpose	Analyze old snRNA data with 04022026 sequencing data
Protocol	1. Upload Data 2. Slice specific window for SM binding site (.iloc) 3. Flatten the multidimensional data into a 1-D array (.flatten) 4. Scale and Transpose data using sklearn preprocessing 5. Plot scree plot displaying percentage of explained variance 6. Plot PC space explaining majority of variance 7. Plot correlation and co-variance matrix 8. Cluster together samples
Results	Replicates have been obtained for all samples. The PCA shows grouping of replicates with the exception of U4atac. When looking at the dendrogram, there is a separation between U5A.1. Considering we are looking at the first 8 principle components, which represents the majority of the variance, this could explain the visual differences.
Future Directions	Create superset of all snRNA data for ML model. Look at RNP-MaP outputs (NormwNeg) and how this impacts the data

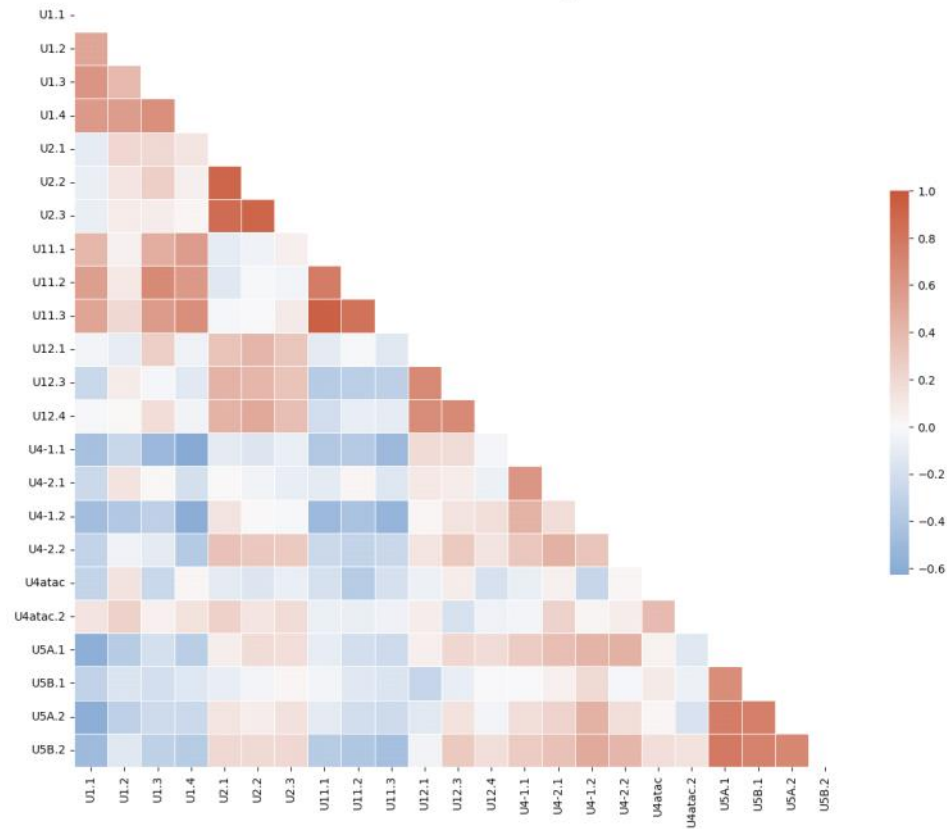


snRNA_PCA
_3D_inter...

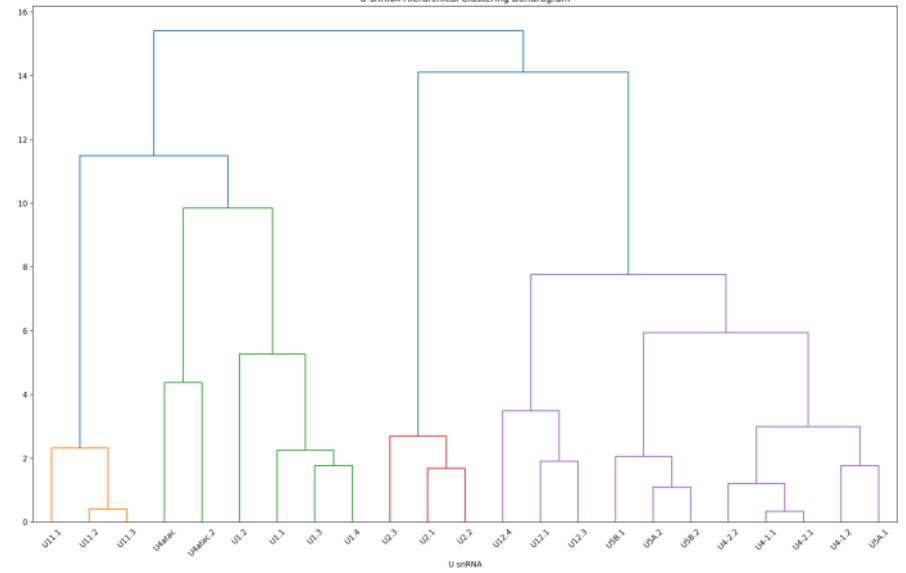


snRNA_PCA

Pairwise Correlation of U snRNA Reactivity Profiles

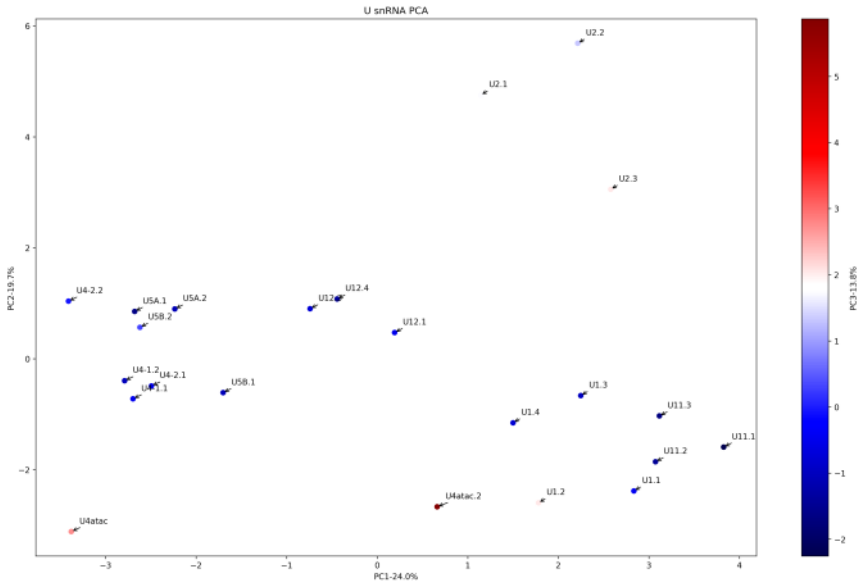
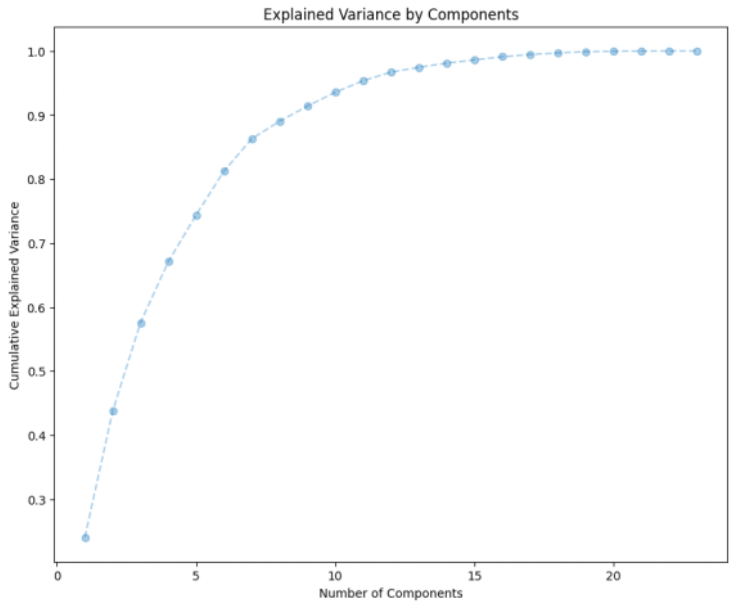
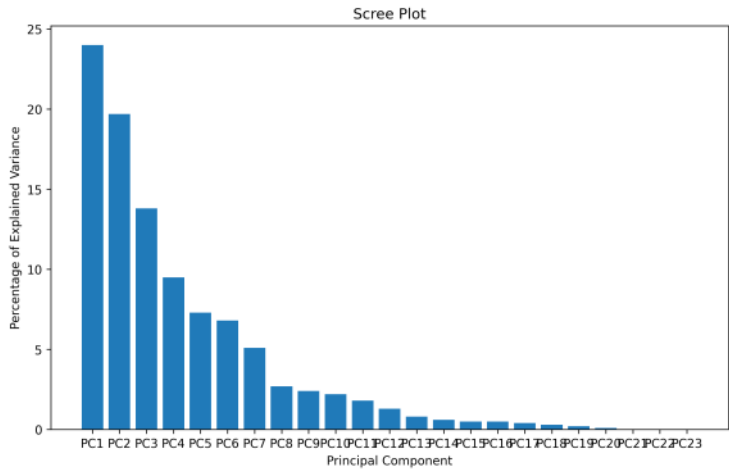


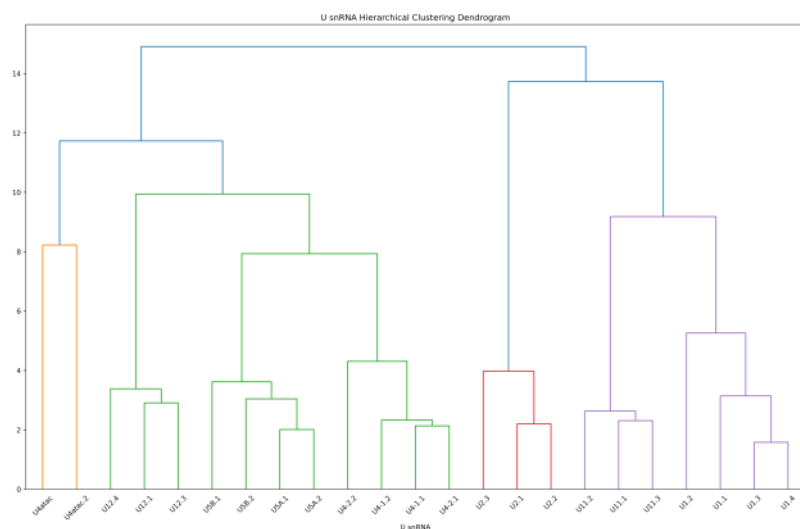
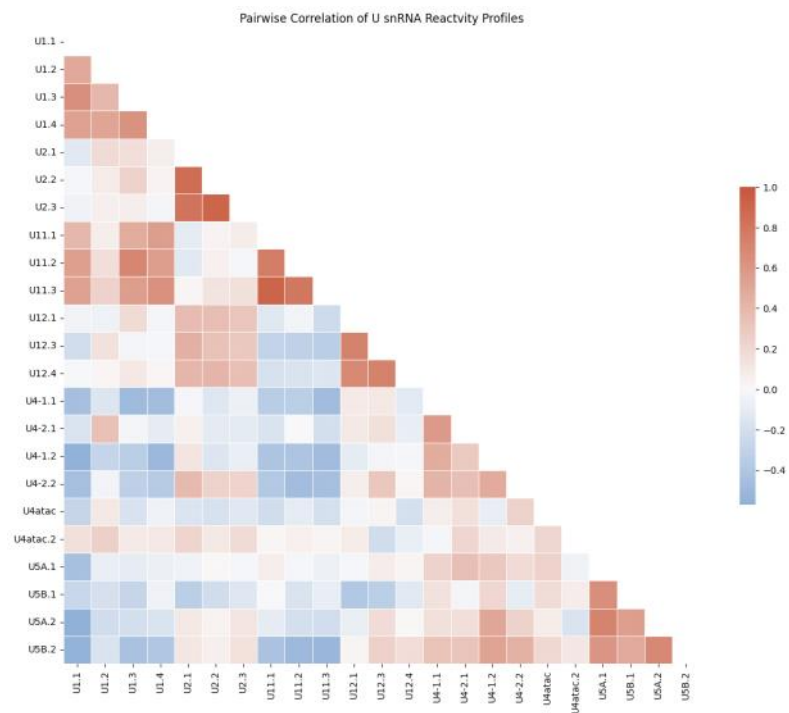
U snRNA Hierarchical Clustering Dendrogram



Date	06/04/2026
Experiment	PCA and Clustering Analysis of all snRNA data
Cell line	HeLa
Targets	All U snRNA samples (Sm binding region)
Purpose	Analyze old snRNA data with 04022026 sequencing data
Protocol	1. Upload Data 2. Slice specific window for SM binding site (.iloc) 3. Flatten the multidimensional data into a 1-D array (.flatten) 4. Normalize snRNA data(Logp1) 5. Scale and Transpose data using sklearn preprocessing 6. Plot scree plot displaying percentage of explained variance 7. Plot PC space explaining majority of variance 8. Plot correlation and co-variance matrix 9. Cluster together samples
Results	Overall, the normalization of snRNA data shifted the relationships of each sample to one another. That being said, it is important to note that the negative values are already normalized. Therefore, this analysis Only shows that log1p normalization of normalized data leads to non-normalized data
Future Directions	Perform analysis of data without normalization. Plot Q-Q plots and perform Shapiro-Wilks to analyze normality.

The following are outputs using the normalization score with negative values instead of Norm scores.

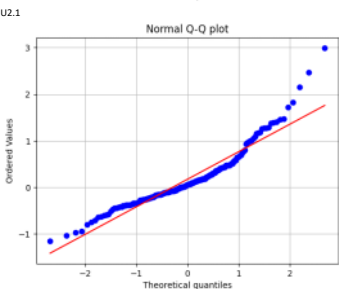
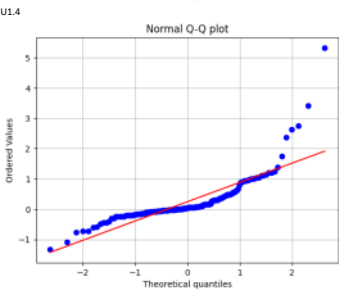
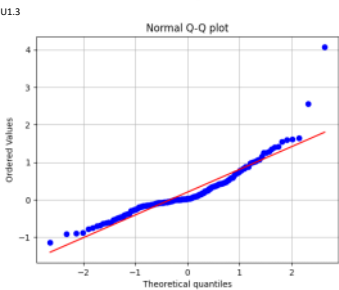
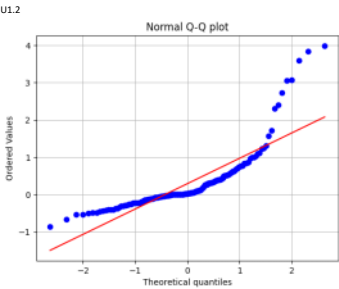
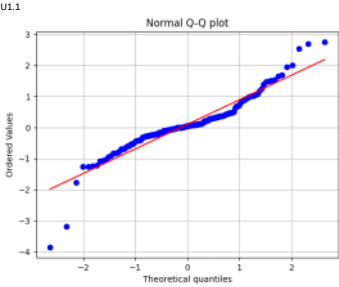




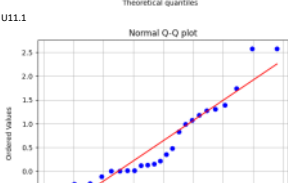
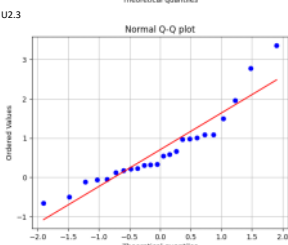
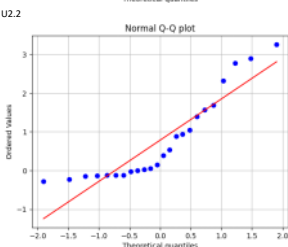
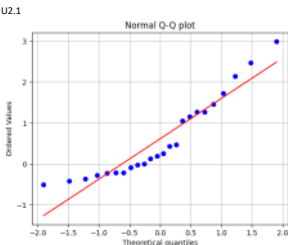
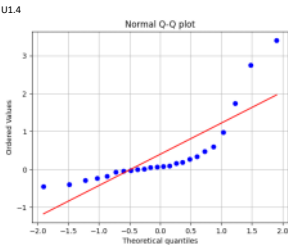
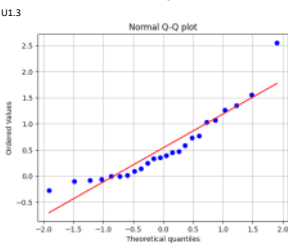
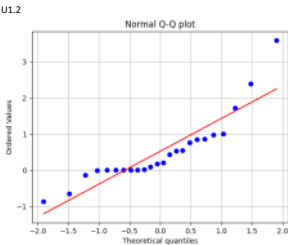
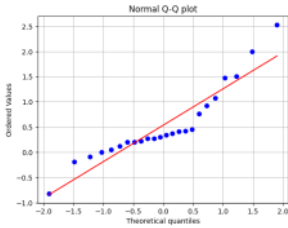
Date	06/05/2026
Experiment	Analysis of non-normalized snRNA data
Cell line	HeLa
Targets	All U snRNA samples (Sm binding region)
Purpose	Analyze old snRNA data with 04022026 sequencing data
Protocol	1. Upload Data 2. Slice specific window for SM binding site (.iloc) 3. Flatten the multidimensional data into a 1-D array (.flatten) 4. Normalize snRNA data(Logp1) 5. Scale and Transpose data using sklearn preprocessing 6. Plot scree plot displaying percentage of explained variance 7. Plot PC space explaining majority of variance 8. Plot correlation and co-variance matrix 9. Cluster together samples
Results	
Future Directions	

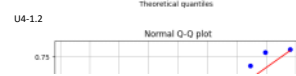
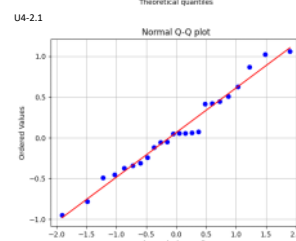
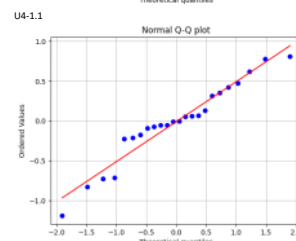
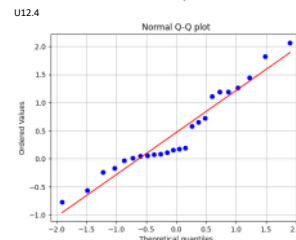
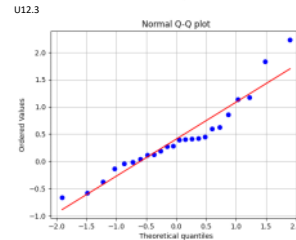
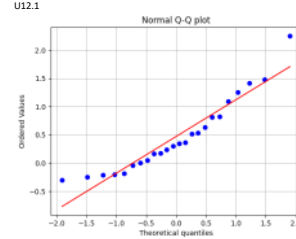
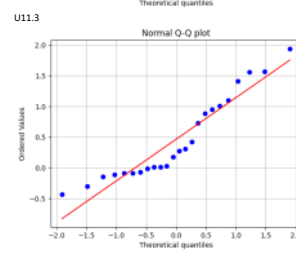
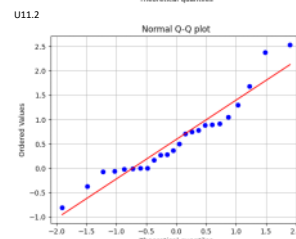
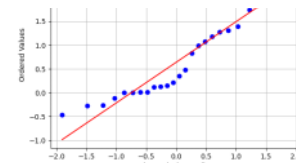
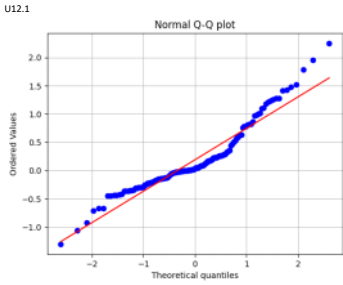
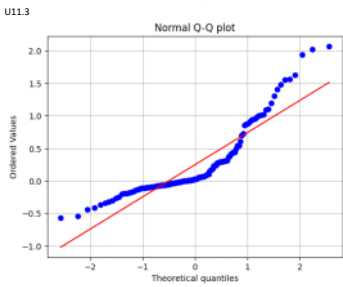
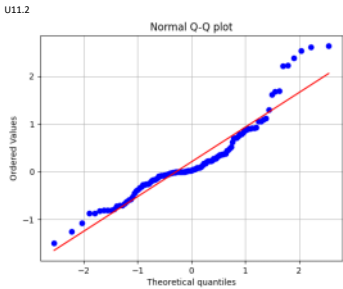
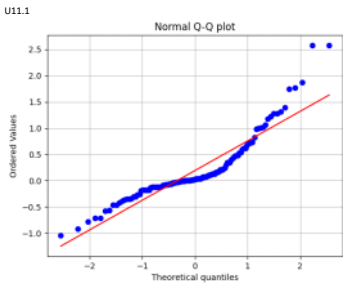
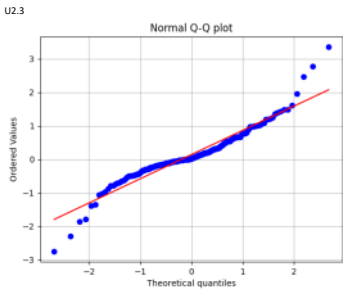
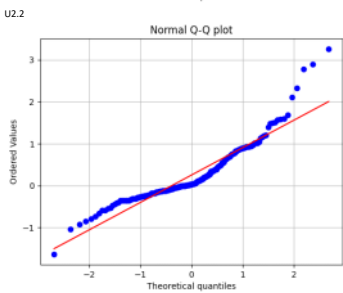
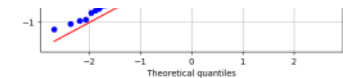
Full Length snRNA sequences QQ-Plot

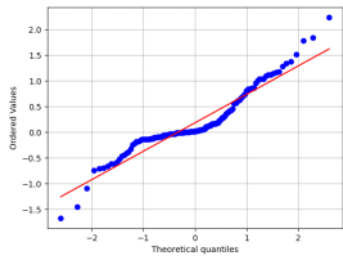
	U1.1	U1.2	U1.3	U1.4	U2.1	U2.2	U2.3	U11.1	U11.2	U11.3	U12.1	U12.3	U12.4	U4-1.1	U4-2.1	U4-1.2	U4-2.2	U4atac	U4atac.2	USA.1	USR.1	USA.2	USR.2
W statistic	9.052208e-01	7.293852e-01	8.739811e-01	7.266092e-01	9.122224e-01	8.984090e-01	9.298891e-01	8.649356e-01	9.094683e-01	8.331569e-01	9.247215e-01	9.177677e-01	0.978018	6.174305e-01	9.128401e-01	0.960022	0.983196	0.929788	0.967311	0.940393	0.934472	0.956729	0.961664
p-value	8.416684e-09	5.422542e-16	1.579787e-10	6.425087e-16	3.814960e-09	4.930401e-10	7.642938e-08	1.945690e-09	3.028831e-07	6.252445e-11	4.333622e-07	1.786882e-07	0.017800	1.179684e-17	1.442140e-07	0.000377	0.079316	0.000005	0.000312	0.000066	0.000027	0.000941	0.002273



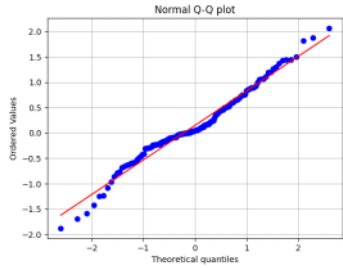
SM SITE QQ-PLOT



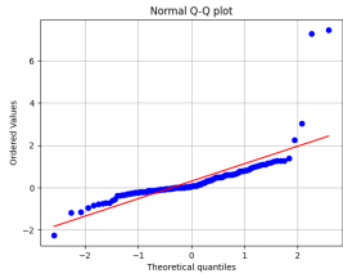




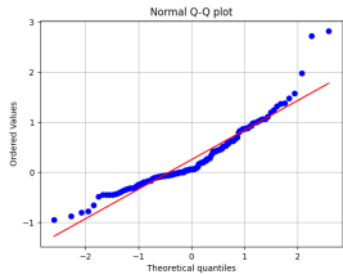
U12.4



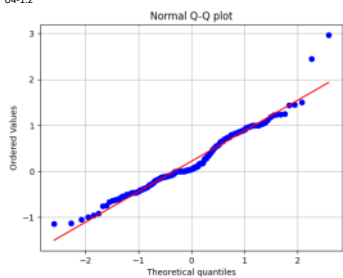
U4-1.1



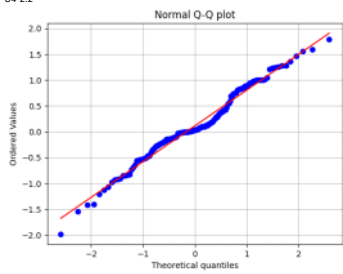
U4-2.1



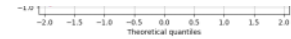
U4-1.2



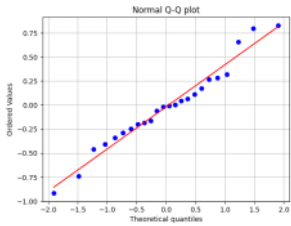
U4-2.2



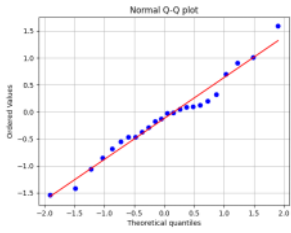
U4atac



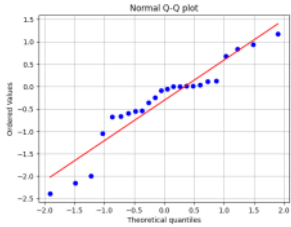
U4-1.2



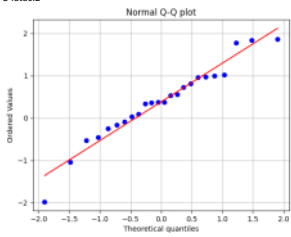
U4-2.2



U4atac



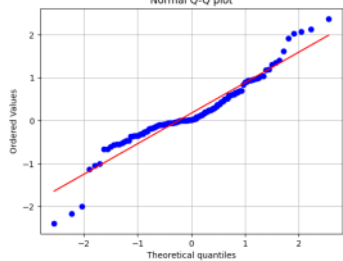
U4atac.2



USA.1

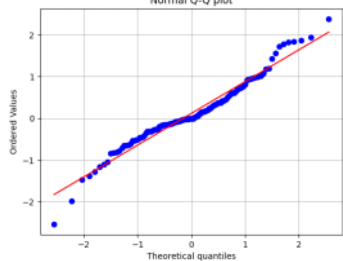
From https://colab.research.google.com/drive/169pU0c843cDpa3_Umy6vMazQ15iE7uagchrome_nipabcoIT?user=67e3b8

Normal Q-Q plot



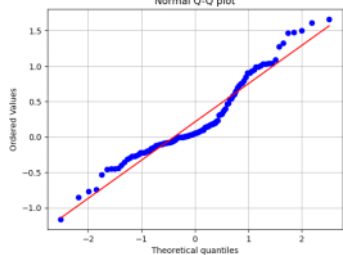
U4atac2

Normal Q-Q plot



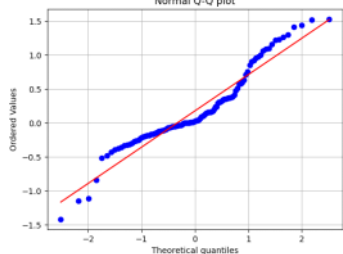
USA.1

Normal Q-Q plot



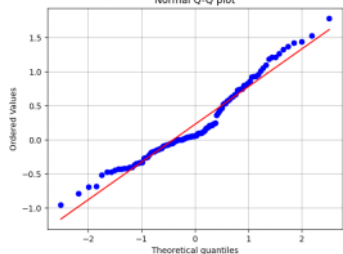
US8.1

Normal Q-Q plot



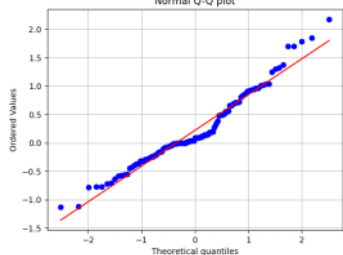
USA.2

Normal Q-Q plot



US8.2

Normal Q-Q plot



From https://velob-research.google.com/drive/16uH4ZocR41G-agA_UnygbvMapQ15fT-agu-chrome_mq#scrollTo=ae9672db

Model	Test Score	Val Score	Pred Time (test, s)	Fit Time (s)	Fit Order
LightGBM	0.9898	0.500	0.009	0.265	2
LightGBMXt	0.9898	0.500	0.013	7.534	3
NeuralNetFastAI	0.7733	0.750	0.780	2.707	8
ExtraTreesEnr	0.7102	0.625	0.316	0.666	7
ExtraTreesGini	0.6898	0.625	0.212	0.654	6
RandomForestEnr	0.6873	0.750	0.175	0.918	4
RandomForestGini	0.6873	0.750	0.176	1.106	5
CalBos	0.6791	0.750	0.018	1.023	5
NeuralNetTorch	0.6125	0.625	0.081	5.988	10
LightGBMLarge	0.6014	1.000	0.130	0.434	11
XGBos	0.5608	1.000	0.043	0.134	9
WeightedEnsemble_L2	0.5608	1.000	0.057	0.279	12

Installation

[AutoGlue \(GitHub\)](#) supports Python 3.9 to 3.12. Installation is available for Linux, MacOS, and Windows. [More installation options](#).

```
pip install autogluon
```

AutoGluon accepts `pandas` DataFrames as inputs, where each row stores an example, with columns as features. We use the `Kaggle Titanic` dataset to demonstrate how to use AutoGluon.

```
import pandas as pd
train_data = pd.read_csv('titanic/train.csv')
```

AutoGluon works with raw data. Little or no data preprocessing, such as removing obvious non-predictive columns, is needed.

```
train_data = train_data.drop(columns=['PassengerId'])
```

Train models to predict the values in the column 'Survived'. The training log will tell you how AutoGluon extracts features, selects, trains, and ensembles models.

```
from autogluon.tabular import TabularPredictor

predictor =
TabularPredictor(label='Survived').fit(train_data)
```

More options to construct a `TabularPredictor` instance ([docs](#))

```
verbosity=3 # Logging (1=warnings, 2=default, 3=verbose)
# The metric used to tune models. All available metrics
eval_metric='roc_auc'
```

More options for the `fit` method ([docs](#), [presets](#)):

- Limit the training time, in seconds
`time_limit=0.0`
- Better model ensemble for a better accuracy, but longer training time. All available options:
`presets='best_quality'`
- Use a separate data set to tune models.
`tuning_data_val=data`
- Explore less models. You can fully control the model search space. All available options:
`hyperparameters='very_light'`
- Ignore some models.
`excluded_model_types='NNN', 'NN_TORCH'`
- Speed up inference
`infer_limit=0.0005` At most 0.05 ms per row

Understand the contribution of each model ([docs](#)).

```
predictor.leadboard(display=True)
```



More options for Leaderboard:

```
# Report metrics on a separate test dataset.
data=test_data
# Evaluate more metrics.
extra_metrics=['accuracy', 'log_loss']
```

Understand more about the trained models ([docs](#))

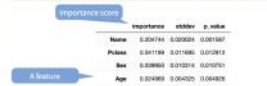
```
predictor.fit_summary(show_plot=True)
```



You need bokeh to use `show_plot=True` (e.g. `pip install bokeh`). It will generate performance vs speed results in `SummaryOfModels.html`.

Understand the importance of each feature ([docs](#)).

```
predictor.feature_importance(test_data)
```



```
test_data = pd.read_csv('titanic/test.csv')
# Predict for each row
predictor.predict(test_data)
# Return the class probabilities for classification
predictor.predict_proba(test_data)
# Evaluate various metrics, it needs test_data to have
# the label column
predictor.evaluate(test_data)
```

AutoGluon predicts with the final ensemble model. You can also [predict](#) using an individual model.

```
# Get a list of string names
models = predictor.model_names()
# Predict with the 2nd model. Both predict_proba and
# evaluate also accept the model argument
predictor.predict(test_data, model=models[1])
```

AutoGluon models are saved to disk automatically. You can check logs to find where it is saved, or by checking `predictor.path`

```
# Load saved model from disk.
```

If the inference speed matters, there are multiple ways to accelerate the speed. First, you can force all models in memory.

`predictor.persist()`

During training, you can use presets for the `fit` method optimized

newstan/'nu:nd nʌstɪn/, 'optimism for development')

Finally, clone a minimal artifact for inference.

```
# Will create a minimal artifact for deployment.
predictor.clone_for_deployment(path='prod_model_dir')
# Load the optimized predictor and persist models in mem.
predictor_opt = TabularPredictor.load('prod_model_dir')
predictor_opt.persist()
predictions = predictor_opt.predict(test_data)
```

- Click [here](#) for detailed Tabular tutorials.
- For data involving text and images, try out [HuisModelPredictor](#).
- For data involving forecasting, try out [TimeSeriesPredictor](#).
- For effortless production deployments, use [AutoGluon Cloud](#).
- Check the latest version of this cheat sheet at <https://autogluon.ai/tabular/cheatsheet.html>
- Any questions? Ask [here](#)
- Like what you see? Consider starring AutoGluon on [GitHub](#) and following us on [twitter](#) to get notified of the latest updates!